DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE

The Faculty of Science

University of Southern Denmark

Predicting week-ahead electricity prices in the Danish bidding zones and cross-border

---

Christine Sandager Søgaard & Nikolaj Kjellerup Andersen
chrso19@student.sdu.dk & nande24@student.sdu.dk

Master Thesis - 30 ECTS
Course code: SPDS800 - Speciale i Data Science
SDU Supervisor: Christian Damsgaard Jørgensen
Hand-in date: 01.06.2026

Abstract

The purpose of this thesis was to test the performance several machine learning models regarding their ability to predict electricity prices for two Danish bidding zones, DK1 and DK2, using a number of explanatory features. The importance of these explanatory features for the model performance was also investigated along with the development of prediction error across the 7-day, or 168-hour, prediction horizon. Several studies have already been performed in this area and for the DK1 bidding zone, but none of these have been based on current trends. The Danish electricity grid has been under steady development with the introduction of more renewable electricity sources, which increases the volatility of the prices.

The project tested three baseline models, which were Naïve Persistence, Moving Average, and Seasonal Naïve, to be used as benchmarks for the more advanced models. Seven different shallow learners were examined, which were ARIMA, ARIMAX, Linear Regression, LightGBM, Random Forest, SVR, and XGBoost, along with four deep learners; Simple RNN, GRU, LSTM, and LSTM AE. Each model was hyperparameter tuned using walk forward cross validation with a prediction horizon of 168 hours to simulate a real use-case. Once the hyperparameters of each model were determined, a final evaluation was performed, where the test set consisted of the entire year of 2025. The performance of the models was determined based on three evaluation metrics: SMAPE, RMSE, and MAE.

To determine the overall best performing model for each bidding zone, a set of 6 evaluation metrics were used. These were an hourly SMAPE, an hourly RMSE, an hourly MAE, an average error quartile 1, an average error quartile 2, and an average error quartile 3. Each of these metrics were weighted equally in order to find the best performing model. The best overall model for DK1 was XGBoost, while the best baseline model was Seasonal Naïve, and the best deep learner was RNN. For DK2, the best performing model was Seasonal Naïve, and the best shallow learner was SVR. This conclusion is based on the code architecture and models examined, but it should be taken with a grain of salt. The models also showed an improved performance with the addition of explanatory features, and that noise added to the weather data during prediction periods did not have a note-worthy impact on their performance.

Contents

List of Figures

List of Tables

Symbols and Abbreviations

Symbols

Symbol	Definition	Unit
t	Time, a given hour	Hour
m	Municipality	N/A
z	Price zone, either DK1 or DK2	N/A
M_z	Set of municipalities in zone z	N/A
$v_{m,t}$	Wind speed in municipality m at hour t	meter/second, m/s
$w_{m,month(t)}$	Weight (capacity) in municipality m in the month at hour t	N/A
$\bar{v}_{z,t}$	Normalized weighted average wind speed in zone z at hour t	meter/second, m/s
n	Number of observations	N/A
y_i	Actual value	DKK/MWh
$\hat{y}_i$	Predicted value	DKK/MWh
C	Regularization term for shallow learners	N/A
p	Number of autogressive terms	N/A
d	Degree of differencing	N/A
q	Number of lagged forecats or terms to be included	N/A

Abbreviations

Abbreviation	Definition
AE	Autoencoder
ANR	Adaptive Noise Reduction
ANN	Artificial Neural Networks
ARIMA	AutoRegressive Integrated Moving Average
ARIMAX	AutoRegressive Integration Moving Average with eXogeneous inputs
ARMA	AutoRegressive Moving Average
CNN	Convolutional Neural Network
DE	Differential Evolution or Germany
DK	Denmark
DK1/DK2	The Danish electricity bidding zones
DKK	Danish Kroner
DMI	Danish Meteorological Institute
EUR	Euro
GB	Great Britain
GPT	Pre-Trained Transformers

Continued on next page

– Continued from previous page –

Abbreviation	Definition
GRU	Gated Recurrent Units
IQR	Interquartile Range
KELM	Kernel Extreme Learning Machine
KNN	k-Nearest Neighbor
LightGBM	Light Gradient Boosting Machine
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
MILET	Multimodal Integration and Linear Enhanced Transformer
MLP	MultiLayer Perceptron
MOO	Multi-Objective Optimization
MW	Mega Watt
MWh	Mega Watt per hour or Mega Watt hours
NL	Netherlands
NO	Norway
PSO	Particle Swarm Optimization
RBF	Radial Basis Function
RF	Random Forest
RNN	Recurrent Neural Network
RPE	Recursive Feature Elimination
SAE	Stacked Auto Encoder
SAPSO	Self-Adapting Particle Swarm Optimization
SARIMA	Seasonal AutoRegressive Integrated Moving Average
SE	Sweden
SGD	Stochastic Gradient Descent
SHAP	SHapley Additive exPlanations
SMAPE	Symmetric Mean Absolute Percentage Error
SNforecast	Seasonal Naïve Forecast

Continued on next page

– Continued from previous page –

Abbreviation	Definition
SSA-DELM	Deep Extreme Learning Machine optimized by Sparrow Search Algorithm
STD	Standard Deviation
SVR	Support Vector Regression
TBATS	Trigonometric seasonal Box-Cox transformation with ARMA residuals Trend and Seasonal Components
TFM-iTransformer	Temporal Featured Mixed inverted Transformer
TSO	Transmission System Operator
UTC	Coordinated Universal Time
Val	Validation
WT	Wavelet Transform
XGB	XGBoost
XGBoost	eXtreme Gradient Boosting

1 Introduction

Energinet is the Danish transmission system operator (TSO), responsible for operating and developing the electricity transmission grid and ensuring security of supply. In practice, this is achieved by ensuring that there is a balance between consumption and production in the electricity system [1].

The Danish power grid is divided into two zones DK1 and DK2, due to transmission constraints between the regions. DK1 and DK2 are only connected by the Great Belt transmission, which has a limited capacity. As illustrated on Figure 1.1, DK1 covers Jutland and Funen and the surrounding islands, while DK2 covers Zealand and the surrounding islands. Both Danish zones are connected to zones in neighboring countries. These connections enable electricity to flow across borders based on price differences and available transmission capacity. If one of the Danish zones produces more power than what is consumed, excess power is exported to the other zone or neighboring countries. If a zone produces less power than demanded by the consumers, power will be imported either from the other zone or from neighboring countries [2]. Figure 1.2 shows the zones in the neighboring countries closest to Denmark and the amount of power exchange between them on 2026-02-17.

Figure 1.1: Source: batteribanken.dk. Danish bidding zones DK1 and DK2 and the border between them.

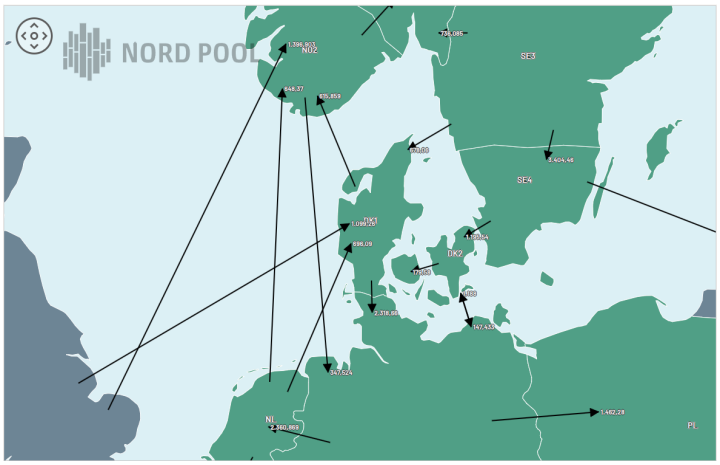


Figure 1.2: Source: Nord Pool. Bidding zones in the neighboring countries surrounding Denmark and the amount of exchange between the zones on 2026-02-17.

On the online Nordic electricity exchange, Nord Pool, the price of electricity is determined on a 15-minute basis based on supply and demand. Market participants submit their buy and sell bids to the exchange, which then matches all buy and sell bids while taking into account the transmission capacity constraints between the different electricity price areas. The Great Belt transmission between the two Danish zones, DK1 and DK2, is an example of such a bottleneck. Having an electricity exchange ensures that the power generation comes from the cheapest available production unit [3].

As the authority responsible for developing the market framework for the Danish electricity market, Energinet has an interest in monitoring developments in electricity prices and the underlying factors in order to ensure effective competition in the electricity market, forecast developments in production capacity, and plan infrastructure to secure a cost-efficient and high level electricity supply security [4].

As renewable energy sources like wind turbines and solar panels constitute an increasing share of the combined electricity production in Denmark and neighboring countries, the volatility in supply increases. As volatility increases, the prices become harder to predict. As these variable renewable energy sources have a large impact on the prices, weather data forecasts like wind speed and sun radiation is necessary in order to predict future prices.

One way of predicting electricity prices is to use machine learning. Machine learning models need to be trained on historic data containing both prices and weather statistics. The goal of this project is to predict electricity prices using machine learning models and in doing that we will experiment with different types of models to learn which models perform best on a task like this.

1.1 Objectives

Main question:

- Which models are best suited for week-ahead forecasts in a context with high volatility?

  Sub-questions:

  1. How does the inclusion of explanatory variables beyond historical electricity prices affect the model's performance for week-ahead forecasting?
  2. How does the forecast error develop over a 7-day interval?

2 Literature Review

The desire to forecast electricity prices or the general interest in the electricity sector is not a new phenomenon. One such example of an early study was published in 1951. The paper focused on analyzing demand and consumption in Great Britain [5], and it is credited as being part of the early work within the area of forecasting aspects of the electricity sector [6, 7]. An early example of artificial neural networks (ANN) being used for energy forecasting was published in 1998. They used Bayesian Information Criterion as a means of determining the model specifications. For the modelling, they used a dataset containing several explanatory variables split into three categories; weather including temperature and humidity, calendar including day of the week and sunrise/sunset, and 4 lagged load features. Rather than using backpropagation, the authors used a conventional nonlinear least squares algorithm in order to find parameter solutions. They also estimated a linear regression model to be used as a baseline for the ANN model. As part of their model specification tuning, they examined how the number of hidden layer nodes impacted the network's performance. They tested networks with 1 to 5 nodes and found that there was no definite best node amount. Ultimately, they settled on 3 nodes, as it has the best test Mean Absolute Percentage Error (MAPE) of 1.71 %, which was better than the baseline, which achieved 1.90 % [8].

The use of machine learning in a forecasting context has evolved substantially since 1998. Easier access to both shallow and deep learners has made it more convenient for studies to be made focusing on the prediction of electricity prices. A paper from 2021 compared the performance of shallow and deep learners for predicting electricity prices in Germany. They used Random Forest (RF) and XGBoost (XGB) as their shallow learners and Long Short-Term Memory (LSTM) as their deep learner. They began the modelling by hyperparameter tuning in order to determine the best combination of hyperparameters for each model. They also included two baseline models to use as a benchmark for the performance of the machine learners. They performed two types of predictions: Single Step Prediction and Multi-Step Forecast Extension. Single step predictions means that only the next upcoming values is predicted. LSTM performed best with a Root Mean Squared Error (RMSE) of 1.9422 followed by XGB at 1.9461 and RF at 1.9957. During the multi-step forecast extension, only the two best performing models, LSTM and XGB, were examined. During this forecast, the forecast window was extended to include the next 4 values, where the model had to predict electricity prices during the first 15 minutes (00:00-15:00), the

second 15 minutes (15:00-30:00), the third 15 minutes (30:00-45:00), and the last 15 minutes (45:00-60:00). XGB performed best during the first two timeslots, while LSTM performed best during the last two [9]. Since the focus of this paper is on the prediction of German electricity prices, it is assumed that the unit of the evaluation metric, RMSE, is in EUR per MWh.

LSTM or LSTM hybrid models are frequently used for the purpose of forecasting electricity prices. A paper from 2022 set out to predict electricity prices within the DK1 bidding zone, which is one of the Danish electricity bidding zones, which was described in section 1. They developed a deep extreme learning machine optimized by sparrow search algorithm (SSA-DELM) to perform these price predictions, where SSA would be able to optimize random input weights and biases, which would help improve the accuracy of the model. They compared the performance of the SSA-DELM to LSTM using Mean Absolute Error (MAE) and RMSE. Each model had to predict the hourly price for each hour within a 24 hour timeframe. The MAE and RMSE was then averaged across the 24 hours. SSA-DELM had an MAE of 3.8 and an RMSE of 4.7 for a test set consisting of all prices from 2020 compared to LSTM, which achieved an MAE of 11.8 and an RMSE of 13.8. When tested on prices for 2021, SSA-DELM achieved an MAE of 9.3 and an RMSE of 11.9, while LSTM had an MAE of 11.1 and RMSE of 14.4. Based on their RMSE, SSA-DELM performed 65.9 % better than LSTM. The unit of the evaluation metrics, MAE and RMSE, is in EUR per MWh [10].

A study from 2019 also developed an LSTM hybrid. Their proposed model was a wavelet transform and Adam optimized LSTM neural network (WT-Adam-LSTM). They used the model to forecast electricity prices for two different markets; France and the New South Wales region of Australia. They used multiple evaluation metrics to determine the performance, but to keep it simple only the MAPE will be highlighted. They set up four cases to determine the best model and parameters. Case 1 focused on comparing different LSTM optimizers, where they concluded that Adam outperformed Stochastic Gradient Descent (SGD) and RMSprop. Case 2 focused on comparative experiments for New South Wales. They compared the performance of their proposed hybrid model with a hybrid model created in a different study referenced to as "Yang's model" [11], which combined wavelet transform, kernel extreme learning machine (KELM), self-adapting particle swarm optimization (SAPSO), and autoregressive moving average (ARMA) [12]. This comparison was possible, since they both focused on data from New South Wales for 2014. The WT-Adam-LSTM hybrid model performed better. In case 3, they compared their hybrid model to a publication using differential evolution (DE) LSTM, where WT-Adam-LSTM once again performed better. Case 4 was used to compare the performance of the model on the French market with other deep learning methods, where the hybrid model outperformed

traditional back propagation, regular LSTM, and a general regression neural network [11]. In case 2, the WT-Adam-LSTM model yields an overall average MAPE of 2.34 %, while Yang's model yields an overall average MAPE of 3.74%. In case 3, the WT-Adam-LSTM has a MAPE of 2.15 %, while the DE-LSTM model has a MAPE of 3.65 % and a regular Adam-optimized LSTM yields a MAPE of 3.64 % [11].

In 2018, a paper was published that focused on the use of Gated Recurrent Units (GRU) as a means of forecasting electricity prices in the Turkish day-ahead market. They trained the model on data from 2013, 2014, and 2015 in order to forecast electricity prices in 24 hour windows throughout 2016. To evaluate the performance, they used MAE. They tested the performance of several models in order to determine the overall performance of the GRU model such as seasonal autoregressive integrated moving average (SARIMA), convolutional neural network (CNN), ANN, and LSTM. A one-layered GRU network outperformed the other models and achieved an MAE of 5.71 EUR/MWh, while LSTM, ANN, SARIMA, and CNN achieved an MAE of 5.91 EUR/MWh, 6.37 EUR/MWh, 7.29 EUR/MWh, and 9.82 EUR/MWh, respectively. Since GRU, LSTM, and ANN performed better than the other models, they were each constructed as networks with three stacked layers along with a final fully connected layer for prediction. Using all 11 features of the dataset, the more advanced GRU model improved beyond the initial performance and achieved an MAE of 5.36 EUR/MWh. LSTM and ANN also improved and reached an MAE of 5.47 EUR/MWH and 6.20 EUR/MWh, respectively [13].

The GRU forecasting model was developed further into a hybrid model in a study from 2022. They proposed a so-called ANR-SAE-GRU method, where they added Adaptive Noise Reduction (ANR) and a Stacked Auto Encoder (SAE). The SAE is used to extract features from the dataset. To evaluate the performance, they used both RMSE and MAE. They compared the performance of the ANR-SAE-GRU method with an ANR-SAE-LSTM method. The GRU method achieved an RMSE of 6.48 and an MAE of 5.00 with a time horizon of 12 hours, while the LSTM method achieved an RMSE of 7.5 and an MAE of 5.42. The currency of electricity prices is not stated in the article, but it is still clear to see that the GRU-based method outperforms the LSTM method [14].

These studies are just a fraction of the work that has been put into developing and analyzing various electricity price forecasting models. Most of these forecasting models appear to utilize some form of neural network approach, whether it is a feedforward network [15], a sequential wavelet-ANN with particle swarm optimization (WT-ANN-PSO) [16], or a hybrid CNN LSTM (CNN-LSTM) model [17]. Transformers are also being utilized more frequently within the elec-

tricity price forecasting area. Several different types of transformers have been studied and show a high performance such as generative pre-trained transformers (GPT) [18], temporal feature mixed inverted transformers (TFM-iTransformer) [19], and multimodal integration and linear enhanced transformer (MILET) [20].

The LSTM study focused on predicting electricity prices within the DK1 bidding zone discussed previously [10] is not the only electricity price forecasting study focusing on Denmark. Most of these studies focus on forecasting the price for just DK1. A paper from 2019 compared the performance of a number of forecasting models for DK1 and with a forecasting period of 24 hours. The study used a seasonal naïve forecast as a benchmark and compared it to autoregressive integrated moving average (ARIMA), trigonometric seasonal box-cox transformation with ARMA residuals Trend and Seasonal Components (TBATS), and ANN. They used MAE and RMSE as their evaluation metrics and a training dataset of the entirety of 2016, while the prices for the first 212 days of 2017 were forecasted as an expanding forecast. Their seasonal naïve benchmark achieved an overall mean RMSE of 65.95 DKK/MWh and MAE of 52.53 DKK/MWh. Out of the models examined, the non-seasonal ARIMA performed best with a mean RMSE of 39.53 DKK/MWh and a mean MAE of 33.24 DKK/MWh. TBATS was the second best performed with an overall mean RMSE of 45.01 DKK/MWh and MAE of 37.51 DKK/MWh. ANN was the worst of the three model and yielded an overall mean RMSE of 48.41 DKK/MWh and MAE of 41.41 DKK/MWh. All three of the explored models outperformed the benchmark. To further improve upon the results, they also combined the forecasts of each of the models using simple averaging. This combined model achieved the best performance with an overall mean RMSE of 36.44 DKK/MWh and MAE of 30.06 DKK/MWh [21].

A third study focused on the DK1 bidding zone of Denmark also examined LSTM as a possible forecasting model. They collected data for four months from 2019; January, February, March, and April, where 60 % was used as the training set, 20 % was used as a validation set, and the last 20 % was used as the test set. Although they were mainly interested in the performance of the LSTM model, they also examined several other models to compare the performance. These models were XGBoost, Random Forest, LightGBM, Support Vector Regression (SVR) using the radial basis function (RBF), k-nearest neighbor (KNN), SARIMA, and CNN. XGBoost performed worst with an RMSE of 26.62 EUR/MWh and an MAE of 15.13 EUR/MWh. LSTM achieved the best performance with an RMSE of 8.41 EUR/MWh and an MAE of 4.64 EUR/MWh, followed close by CNN, which yielded an RMSE of 9.49 EUR/DKK and an MAE of 5.60 EUR/MWh [22].

Many studies have already been conducted to develop a model that accurately forecasts electricity prices, some of these studies have already been done for Denmark. However, the data used for these studies does not reflect the current state of the Danish electricity grid, which has changed substantially since 2021. The evolution of the Danish electricity grid is also described in further detail from a data point-of-view in section 3.1. Due to the increased production of electricity from renewable energy sources and therefore the increased volatility of the Danish electricity prices, there is scientific justification to conduct an analysis on the performance of different machine learning models in the current state of the Danish electricity production.

3 Methodology

The following sections will describe the methodology used for this project. This includes describing, analyzing, and preprocessing the data used, how the data was split and used for validation, and different evaluating models. It will also describe overall method of training, hyperparameter tuning, validation, and final evaluation for both shallow and deep learners.

3.1 Data Analysis

In this project, we collected data from Energinet's data portal Energi Data Service and DMI (Danish Meteorological Institute). From Energi Data Service, we collected the datasets "Elspot Prices" and "Day-Ahead Prices" for data on Danish electricity prices, "Generation per Production Type and Exchange" for production amounts and exchange with neighboring countries, and "CapacityPerMunicipality" for data on production capacity for each municipality. From DMI, we collected weather data containing wind speed and sun radiation intensity.

First, we produced visualizations of the data to examine seasonal trends, development and periods with extreme values. We examined the data in both hourly, daily, weekly, and monthly resolutions, and for each price zone and both zones combined. In the section below, all visualizations are shown in monthly resolution with the production of both zones combined into a total production value for the entire country.

First, we looked at the development in total power production, total consumption and the production capacity in offshore wind, onshore wind and solar power. This development is shown in Figure 3.1. The figure shows that both production and consumption follow a seasonal pattern that stays on a relative steady level through the period. It shows that during summer the production matches consumption, but during winter, production is lower than consumption. We see a steady slow increase in onshore capacity, a slightly faster increase in offshore capacity, and a faster increase in solar capacity.

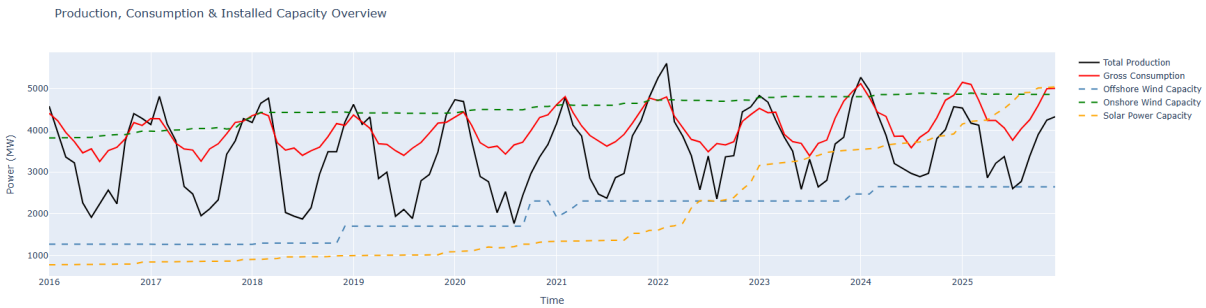


Figure 3.1: Development in total power production (black line), total consumption (red line) and the production capacity for offshore wind (blue line), onshore wind (green line) and solar power (yellow line) in Denmark.

Figure 3.2 shows the development in the production of solar and wind power in Denmark. It shows an increase in offshore wind and solar power production and a more steady level of production of onshore wind energy. The production of solar power reaches a more steady level from 2023 to 2025.

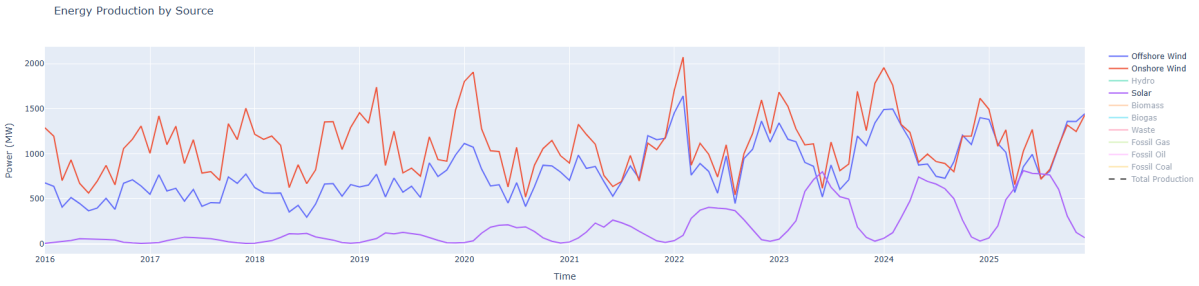


Figure 3.2: Development in production of solar and wind power in Denmark (MW). The red line shows onshore production, the blue line shows offshore production and the purple line shows solar power production.

The opposite trend is seen when inspecting the power production of fossil fuels as Figure 3.3 shows. Production level decreases for all three sources; fossil gas, fossil oil, and fossil coal.

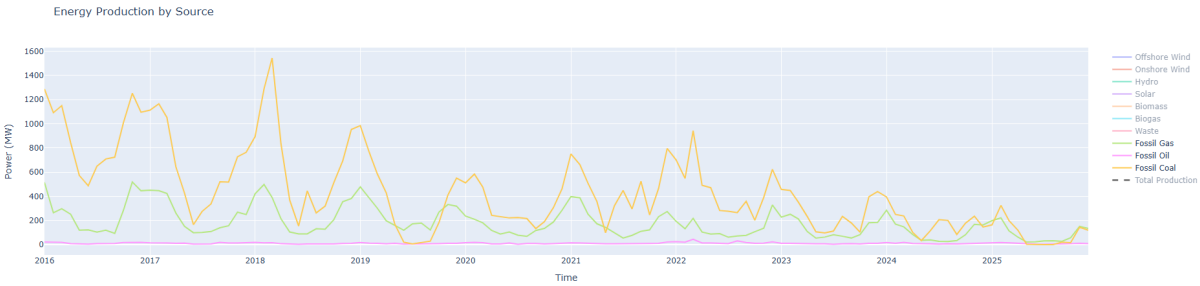


Figure 3.3: Development in production of power from fossil fuels in Denmark (MW). The green line shows production from fossil gas, the pink line from fossil oil and the yellow line from fossil coal.

The production of power from hydro, biomass, biogas and waste is more steady as can be seen Figure 3.4. Biomass energy production, on the other hand, is at a steady level from 2016 to 2021 and then jumps to a higher level for the remainder of the period.

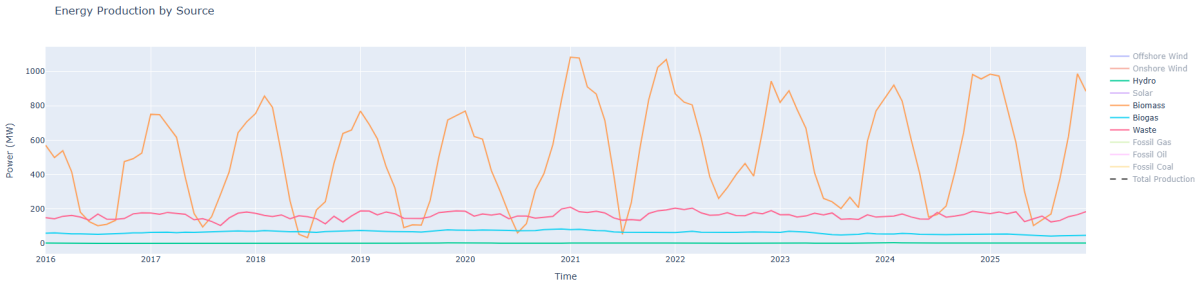


Figure 3.4: Development in production of power from hydro, biomass, biogas, and waste in Denmark (MW). Green line is hydro power, orange is biomass, blue is biogas and red is energy from waste.

When looking at the levels of energy exchange with neighboring countries, Sweden, Norway, and Germany, shown on Figure 3.5, we also see some important trends. In the first half of the period, there seems to be a seasonal variation in whether we import or export energy to each country. All three countries vary within the same range. In the second half of the period, we see different developments for the three countries. We mostly import what seems to be an increasing amount of energy from Norway and Sweden with larger values during summer, where our domestic production is lowest and smaller values during winter, where domestic production is lower than our consumption as we saw in Figure 3.1. We mostly export energy to Germany and the exported amount increases during the period.

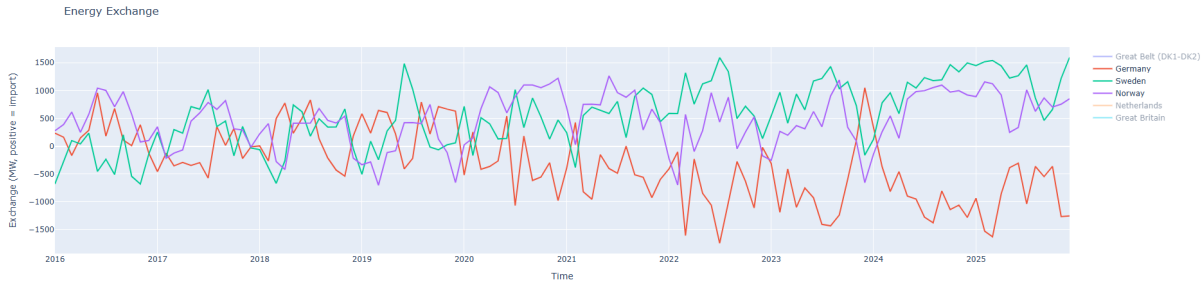


Figure 3.5: Development of exchange of energy with three neighboring countries; Sweden, Norway, and Germany (MW). The green line shows exchange with Sweden, the purple line shows exchange with Norway, and the red line shows exchange with Germany. Positive values indicate import to DK and negative values indicate export from DK.

For the Netherlands (NL) and Great Britain (GB), we don't have data for the entire period. Exchange with the NL started in July 2019 and exchange with GB started in November 2023. This yields two years of data for GB and 6.5 years for NL as shown on Figure 3.6. Because of this limited data, it is difficult to see clear developments for GB. However, for NL it seems that there is a large variation in the beginning with mostly export of energy from DK to NL, and later in the period, we see smaller variations with both power trade going both ways.

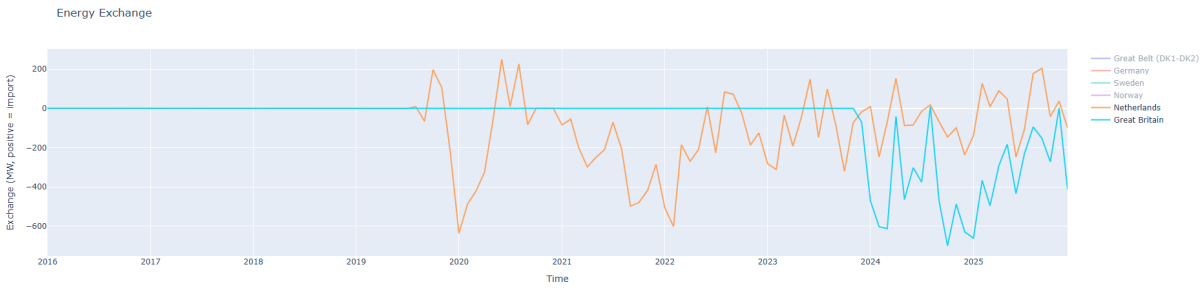


Figure 3.6: Development of exchange of energy with neighboring countries GB and NL (MW). Orange is NL and blue is GB. Positive values indicate import to DK and negative values indicate export from DK.

Visualizing the development in electricity prices in price zones of neighboring countries also reveals important trends. Figure 3.7 shows steady prices before 2021, increased volatility from 2021 to 2023 with a peak of extremely high prices in August 2022, and then from 2023 more stabile and normal levels even though volatility is still higher than before the price peak and difference between the countries is larger. This also coincides with the start of the war in Ukraine, which began in February 2022 [23].

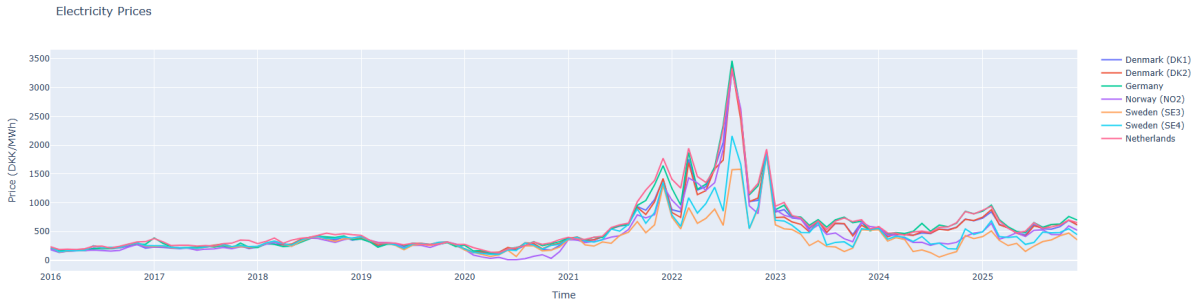


Figure 3.7: Development in electricity prices in price zones of neighboring countries (DKK). The graph uses the following color:country pairs; dark blue:DK1, red:DK2, green:Germany, purple:NO2, orange:SE3, and light blue:SE4.

Figure 3.8 shows the trend in radiation intensity and wind speed averaged across both Danish price zones. We see clear seasonal variations with higher levels of radiation in the summer and higher levels of wind speed in the winter. We also see varying levels per year in wind speed that match varying year levels in the wind production graphs in Figure 3.2.

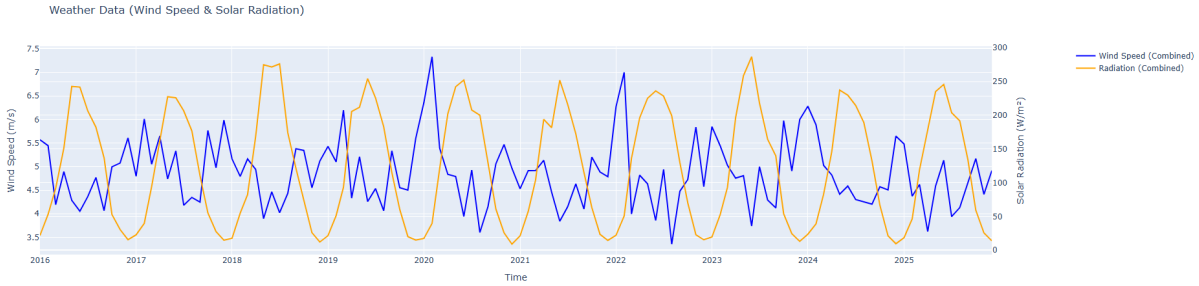


Figure 3.8: Trend in radiation intensity (W/m^2) and wind speed (m/s) in Denmark. Blue line is wind speed and orange line is sun radiation intensity.

3.2 Data Preprocessing

Based on the data analysis, we decided to discard the data on exchange with GB as it only covers two years. We did this, because we would have to split the data into a training and a testing set, which would leave only 1 year of data for training, which we did not expect to be enough for the models to learn patterns on a yearly basis. There were also no prices available for GB, which makes the data even less valuable.

The data on exchange with NL was also limited, but since there was data for 6.5 years, which would mean several seasons worth of data, and there were significantly large values of exchange, we chose to include the data for exchange with the NL.

The data from Energinet contained both UTC time and DK time for each hour. The DK time included daylight saving time which meant that we had duplicates every October and a missing hour every March. To avoid confusing the prediction models, we decided to only use the UTC time.

The price data from Energinet consisted of two separate datasets: old prices from 2016-01-01 to 2025-09-30 (Elspot Prices) with hourly resolution and new prices from 2025-10-01 to 2025-12-31 (Day-Ahead Prices) with 15 minute resolution. To avoid mismatch in resolution, we downscaled the new prices to hourly resolution by calculating hourly averages. The datasets originally contained prices in both Euros and DKK, but we only used the DKK prices as these are the prices we wish to predict.

The data from Energinet contained data values for each of the two price zones and each hour, which meant that there were two rows per timestamp; one for each price zone. Meanwhile, the data from DMI contained data values for each municipality in Denmark and for each timestamp, which meant that for each hour, there were 98 weather records; one for each municipality. This mismatch in dimensionality meant that we needed to reduce the dimensionality of the DMI data, so we would only have one value per price zone per hour in order to merge the two datasets. A simple average value across all municipalities in each price zone would not make the wind speed a very useful explanatory variable for the wind power production as the average wind speed of an entire price zone might differ a lot from the wind speed at the big offshore wind farms or from the wind speed in municipalities with many wind turbines. Instead, we used a weighted average, so the wind speeds in municipalities with large production capacities had a larger influence on the average than the wind speeds in municipalities with smaller capacities. The same goes for the sun radiation and the solar power capacities. The capacities used as weights are the actual, installed capacities within the given municipality, so no normalization is utilized on the capacities.

From Energinet, we had data on production capacity per municipality for each of the three renewable power sources; offshore wind, onshore wind, and solar power, for each month in the period 2016-2025. We used these capacity values as weights to calculate the weighted average of wind speed and radiation across all the municipalities for each price zone. To make sure the weighted average was in a normal range of wind speeds and sun radiation, we used a normalized weighted average given by the following formula:

$$\bar{v}_{z,t} = \frac{\sum_{m \in M_z} v_{m,t} \cdot w_{m,month(t)}}{\sum_{m \in M_z} w_{m,month(t)}} \tag{3.1}$$

where

- t is a specific hour
- m is a specific municipality
- z is a specific price zone (DK1 and DK2)
- M_z is the set of municipalities in zone z
- $v_{m,t}$ is the wind speed in municipality m at hour t
- $w_{m,month(t)}$ is the weight (capacity) in municipality m in the month at hour t
- $\bar{v}_{z,t}$ is the normalized weighted average in zone z at hour t.

For instance, we calculated the weighted average of wind speed by computing the product of the wind speed and the installed wind turbine capacity (onshore and offshore combined) for each municipality and computed the sum of all these products for the entire price zone. Then we divided the sum with the sum of all the capacities in all the municipalities in the price zone. The same was done with sun radiation and the installed capacities of solar panels to find the weighted average of sun radiation for each price zone.

The reason that we included data on wind speeds and sun radiation in the dataset was the assumption that they could provide the models with indications of how much power would be produced by wind turbines and solar panels. We assumed that measurements of wind speeds and sun radiation, at locations with large production capacities, would contribute with more reliable information about the power production than measurements at locations with smaller capacities. The purpose of using the weighted averages for wind speed and sun radiation was to reduce the 98 values to one value for each weather feature in a way that would be more informative for the models than a simple average. In this method, however, laid an assumption that the relevance of weather data in a given municipality would have a linear relationship with the installed capacity in that municipality. This assumption was further more based on the assumption that installed capacity was proportional to power production. These linear relationships were not necessarily true and therefore might provide the model with some degree of inaccurate information. Even

though we normalized the average by dividing by the sum of the capacities to make sure the values were inside the range of normal wind speeds and sun radiation, we still ended up with statistical proxies for wind and sun data, that might not always be good indicators of the production of renewable energy. This is discussed further in section 6.2.2.

Another important detail was that the installed capacity for wind power also included the offshore capacity belonging to each municipality, while the wind speed was only measured on land. This meant that the wind speed in municipalities with a large offshore capacity might not be very representative of the wind speed at the location of the offshore wind farms. With that said, our assumption was still that the wind speed on land in the coastal municipalities with large offshore capacities was more relevant for the electricity price than the wind speeds in municipalities far in land with almost no wind power capacity.

While we assumed the weighted average to be more informative for the models than a simple average, it was important to keep in mind, that it might introduce some inaccuracies and might be the source of some degree of error in the models predictions.

As the capacity dataset contained a capacity record for each month, the weights were calculated for each month during the 10-year period, meaning the capacities were constant for one month at a time, but changed from month to month. This meant that the contribution of each municipality changed over time. Figure 3.9 shows the capacities of each municipality and thus how much each municipality affected the average in December 2025.

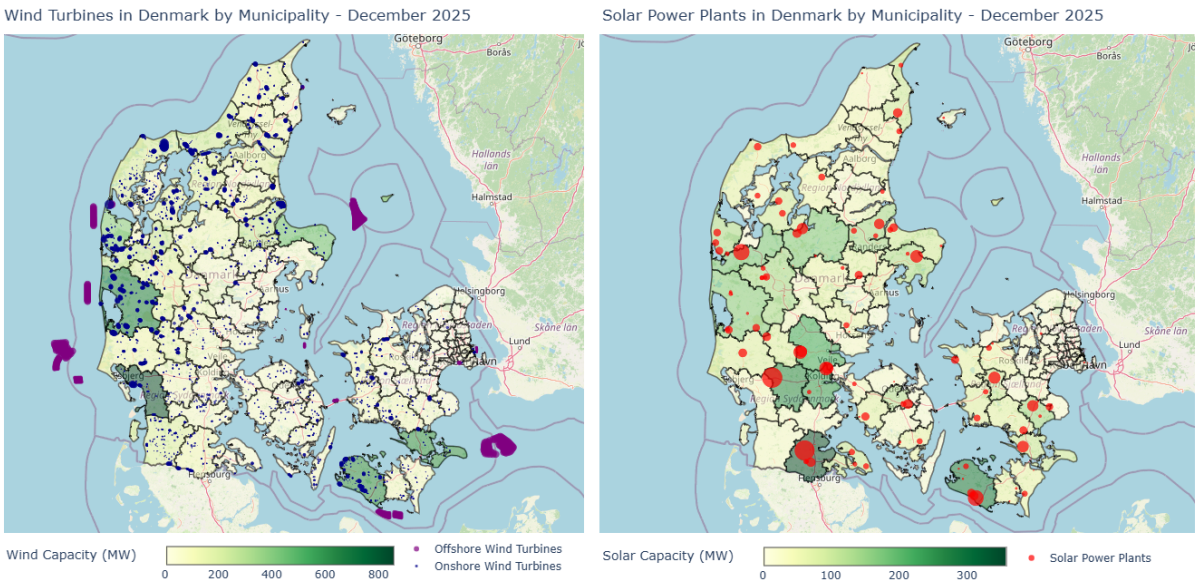


Figure 3.9: Left side: Wind power capacity per municipality in December 2025 (MW). The purple points are offshore wind turbines, the blue points are onshore wind turbines and the saturation of green color per municipality shows the power capacity. Right side: Solar power capacity per municipality (MW). The red points are solar power plants, the size of the red points shows the power capacity for the plant and the saturation of green color per municipality shows the total power capacity for that municipality.

Using these averages in the weather data, the two datasets matched each other in dimensionality.

We also wanted to add the total capacity for each price zone to the project's dataset. The dataset "CapacityPerMunicipality" contained a power production capacity for each municipality and each month, meaning that for each month of the 10-year period, there were 98 records of capacities. To be able to add this data, we calculated the total capacity for each price zone DK1 and DK2 for each of the three power sources; offshore wind, onshore wind, and solar power.

In order to allow the models to learn any cyclic patterns in daily, weekly and yearly seasonality, we split the time column into separate parts, so we also had features for the hour of the day, the day of the week and the month of the year.

Prices for the Netherlands were not included in the dataset from Energinet. The prices were instead found through the Transparency platform, which is a platform used to show the electricity generation, transportation, and consumption for the European market [24]. Prices were downloaded from 2016-01-01 to 2025-12-31. These prices were in EUR/MWh, so it was necessary to convert the currency from EUR to DKK. Exchange rates were downloaded from 2016-01-01 to 2025-12-31 [25]. However, exchange rates were only available for bank days, so an average of all exchange rates from the period was taken, which was 7.44 DKK/EUR. Using this rate, all electricity prices from the Netherlands were converted to DKK/MWh.

All data was combined into a single dataset consisting of 10 years from 2016-01-01 to 2025-12-31 with hourly records for both price zones DK1 and DK2. This resulted in 24 hours $\cdot$ 365 days $\cdot$ 10 years = 87.600 records per price zone The 10 years of records included 3 leap years, 2016, 2020 and 2024, which added another 3 years $\cdot$ 24 hours = 72 records per zone, resulting in a total of **87.672** records per zone. The data contained 35 features listed below with their measured units:

1. TimeUTC: year-mm-dd hh:mm:ss
2. DKZone: DK1 or DK2
3. OffShoreWindPower: MWh (Mega Watt hours)
4. OnShoreWindPower: MWh
5. HydroPower: MWh
6. SolarPower: MWh
7. Biomass: MWh
8. Biogas: MWh
9. Waste: MWh
10. FossilGas: MWh
11. FossilOil: MWh
12. FossilHardCoal: MWh

13. ExchangeGreatBelt: Exchange between zones DK1 and DK2 in MWh
14. ExchangeGermany: MWh
15. ExchangeSweden: MWh
16. ExchangeNorway: MWh
17. ExchangeNetherlands: MWh
18. WindSpeed: m/s (meters per second)
19. Radiation: W/m^2 (spectral range: 305-2800nm)
20. DKPrice: DKK per MWh
21. DEPrice: DKK per MWh
22. NO2Price: DKK per MWh
23. SE3Price: DKK per MWh
24. SE4Price: DKK per MWh
25. NLPrice: DKK per MWh
26. GrossCon: MWh
27. TotalProduction: MWh
28. Year: integer
29. Month: integer
30. Day: integer
31. WeekDay: integer
32. Hour: integer
33. OffshoreWindCapacity: MW
34. OnshoreWindCapacity: MW
35. SolarPowerCapacity: MW

3.2.1 Preprocessing for Shallow Learners

When tuning and training the shallow learners, the dataset was split up in two sets; one for DK1 and one for DK2. The Zone column was removed and the DKPrice column was moved to the first column as the target variable. Then four new features were calculated and added; 1-hour lag and 24-hour lag for both the price and the GrossCon features. These four lag features were added because shallow learners do not remember past values and since the data had strong cyclic patterns with both daily and weekly cycles, these lags were important to take into account when predicting future prices. The GrossCon lags were added because consumption is a measure of demand and demand usually has a high impact on prices on any market. The result was that the dataset for the shallow learners contained the following 38 features:

1. DKPrice
2. Time
3. OffshoreWindPower
4. OnshoreWindPower
5. HydroPower
6. SolarPower
7. Biomass
8. Biogas
9. Waste
10. FossilGas

11. FossilOil
12. FossilHardCoal
13. ExchangeGreatBelt
14. ExchangeGermany
15. ExchangeSweden
16. ExchangeNorway
17. ExchangeNetherlands
18. WindSpeed
19. Radiation
20. DEPrice
21. NO2Price
22. SE3Price
23. SE4Price
24. NLPrice
25. GrossCon
26. TotalProduction
27. Year
28. Month
29. Day
30. WeekDay
31. Hour
32. OffshoreWindCapacity
33. OnshoreWindCapacity
34. SolarPowerCapacity
35. GrossCon_lag1
36. GrossCon_lag24
37. Price_lag1
38. Price_lag24

3.3 Data Splitting

In machine learning, the dataset is typically split into a training set and a test set. The training set is used to train the model, while the test set is used to evaluate the performance of the model. To increase the performance of a given machine learning model, it is also necessary to adjust its hyperparameters, which means that the training of the model may be repeated several times, until the best combination of hyperparameter values is found. In order to determine which combination is best, the performance of each combination needs to be evaluated. However, if the hyperparameters are adjusted while evaluating on the test set repeatedly, the model will be increasingly specialized to perform well on that specific test set. This may reduce the model's generalization ability, as it is adapted to the characteristics of a small test set instead of learning the overall trends of the problem domain. When this happens the model's performance on unseen data may be reduced and the test set can no longer be used to evaluate the generalization performance of the model. To avoid this, the training set is further divided into two subsets; a training set and a validation set. The validation set is used for evaluating the model during

hyperparameter tuning, while the test set is reserved for the final evaluation. This practice preserves the test sets ability to evaluate how well the model generalizes to unseen data. This also means that the actual test set should only be utilized once, when the training of the model is complete [26].

In this project, we used time series data from a 10-year period from 2016-01-01 to 2025-12-31, as described in the previous sections. All data from year 2025 was used as the test set. Using the latest year for evaluation would provide the best picture of the model performance on present day data trends. By evaluating on a full year, we got a measurement of how well the model performs across all seasons.

This left 9 years for hyperparameter tuning and training. Due to the increased capacity and thereby production of electricity from renewable energy sources such as wind- and solar power in recent years, these recent years were important for model training. Thus, it would not be preferable to reserve more of the latest years for validation than necessary. This would not fit the model to an accurate image of the current capacities in Denmark, and therefore predictions would have large errors. The hyperparameter search must also reveal the best parameter values for fitting the model to recent years trends, which meant that the hyperparameter search must also include the most recent data.

One way to prevent loosing training data is by using cross validation. Cross validation means that the training data is split into several subsets, which are in turn used for training and validation [26]. Several different types of cross validation exist, but in an effort to keep the temporal dependencies, it is important to use a cross validation method that preserves the order of the data, unlike k-fold cross validation. In this project, walk forward cross validation could be a suitable method.

3.3.1 Walk Forward Cross Validation

Walk forward cross validation is a validation method that trains on a period of the time series and then validates on a period that follows the training period. This process is repeated while moving both the training and validation periods forward each fold. If using expanding window walk forward cross validation, the training period expands every fold while always starting from the same point, which pushes the validation period forward. If using rolling window walk forward cross validation, the training periods' starting point moves forward, while the training period is of fixed length, which also pushes the validation period forward.

The advantage of using walk forward cross validation was that we could evaluate on the entire year of 2024 while still using most of it for tuning hyperparameters. Take for example, the setup illustrated in Figure 3.10. Here we could have the first fold train up until the end of 2023 and validate on the first 4 months of 2024, then on the second fold train until the end of the first 4 months of 2024 and validate on months 5-8 of 2024, and so on. This would give 3 folds until 2024 was fully evaluated while only 4 months would be excluded from training the models. If we used validation periods of 1 month instead of 4, then only 1 month would be excluded from training the models.

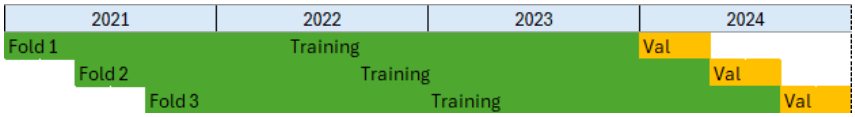


Figure 3.10: Example of a rolling window walk forward cross validation setup. The green areas represent the training periods and the orange areas represent the validation periods. The setup contains 3 folds with a training period of 3 years, a validation period of 4 months and a stride of 4 months.

The drawback of walk forward cross validation is that it takes a lot of time. Take for example a rolling window walk forward cross validation setup with a validation period of $4 \cdot 1$ week, a stride $= 4$ weeks and an initial training period starting from 2016-01-01 and ending at 2023-12-31. This gives a training set of 8 years $\cdot 365$ days per year $\cdot 24$ hours per day $+ 2$ leap years $\cdot$ 24 hours per leap year $= 70,128$ hours and a validation set of 4 weeks $\cdot 7$ days per week $\cdot$ 24 hours per day $= 672$ hours. Fitting and evaluating an SVR model on these sets takes about 1 hour. With a stride of 4 weeks, we would have 52 weeks per year $\div 4$ weeks per fold $= 13$ folds. This gives at least 13 hours of training and evaluation for each combination in the search grid. We preferred to have at least 60 to 100 combinations in the search grid, which would result in 780 to 1300 hours per model, which is 32 to 54 days for just one model. Of course, this would not be feasible for a project spanning 4 months, where we wanted to compare several models. But luckily, initial experiments proved that training periods could be reduced without loss of performance.

In the beginning of the project, we tested the different training periods with a fixed hyperparameter setup on a LightGBM model and a Random Forest, to see which period of training gave the best performance on the validation set (the full year of 2024). The experiments showed that training on only the most recent 3 years gave the best performance. Only one combination of hyperparameter values was tested on two different models in this experiment, so we could not

know for sure if the result would have been the same with other combinations. Ideally, we should have tested more combinations, but that would have been very time consuming. Table 3.1 shows the performance of the LightGBM models trained on different periods of the DK1 train data, evaluated with the SMAPE metric. All models were evaluated on the full year of 2024.

Table 3.1: This table shows the results of using different training periods for the LightGBM model on the DK1 train data. The performance of the models are evaluated using the SMAPE metric to compare the models. They are all validated on the full year of 2024.

Train years	Train start	Train end	SMAPE
2	2022-01-01	2023-12-31	56.88
3	2021-01-01	2023-12-31	59.12
4	2020-01-01	2023-12-31	61.04
5	2019-01-01	2023-12-31	63.14
6	2018-01-01	2023-12-31	63.39
7	2017-01-01	2023-12-31	64.26
8	2016-01-01	2023-12-31	65.06
1	2023-01-01	2023-12-31	67.37

The experiment was conducted with the aim to get a quick picture of how large a dataset, we would need for training, and it showed that for the two models, Random Forest and LightGBM, training on the most recent 3 years of data gave the best performance. Based on this, we decided to only use 2021 to 2023 for training during hyperparameter tuning.

Another important detail is that validation periods did not need to come right after the training period, because of the way we decided to feed data to the model when evaluating. In the evaluation process, when the models made predictions, we fed them with forecasted data, price lags, and consumption lags. These values were forecasted and computed based on real historic data up until the hour before the prediction hour. This meant the model always predicted based on input values that were realistic for that specific hour of the day, week, and year. One of the most important factors was that the model had been trained on data from a similar part of the cycles: same hour of the day, same day of the week, and same month of the year. The model would then from the features Year, Month, Day, WeekDay, and Hour know exactly where in the cycle the prediction hour was and use that information combined with the price lags, consumption lags, and the forecasted feature values for the current hour to make the prediction. This evaluation process is further explained in Section 3.4. The point is, that validation periods did not need to come right after the training period, because the validation process provided the models with recent data needed to make predictions for the next time step.

Because of the two important details mentioned above, we decided to use a more time saving validation method. In order to evaluate the performance of each model in the hyperparameter tuning process across all seasons and under present day circumstances, we used the latest full year of the training set, 2024, as the validation set. To reduce tuning time to a feasible time frame, we used a validation method with only one training fold of 3 years from 2021-01-01 to 2023-12-31 and four periods of 4 validation weeks with a stride of 13 weeks between. This meant that January, April, July, and October of 2024 were used for validation. This way the validation time was reduced to a third of the full year validation time, while still validating on all seasons. Exactly how the validation process was conducted is described in section 3.4.

The resulting data split is shown in Table 3.2 and visualized in Figure 3.11. This split was used for all baseline models and shallow learners. The data split for deep learners is almost identical and is described in section 3.6.1.

Table 3.2: The data split used throughout the project.

Dataset	Span	hours per DK zone
Training	2016-01-01 00:00:00 to 2023-12-31 23:00:00	70128
Validation	2024-01-01 00:00:00 to 2024-12-31 23:00:00	8784
Test	2025-01-01 00:00:00 to 2025-12-31 23:00:00	8760

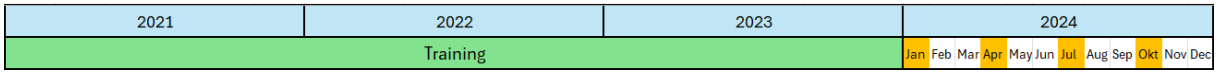


Figure 3.11: The training and validation split used for hyperparameter tuning of baseline models and shallow learners. The green area represents the training period and the orange areas represent the validation periods.

3.4 Evaluating Models

When evaluating the performance of the models, whether it was during hyperparameter tuning, training the final model, or testing the final model, the same method was used every time. We wanted to simulate a real use-case as realistic as possible. This meant that we had to feed the model with realistic data. We couldn't just feed it with real historic data, because in a real prediction scenario, we would not have access to the future values of e.g., the production, consumption, exchange, and prices in neighboring countries. We would only have access to the current and past values of these features. In a real scenario, it would be necessary to forecast the future values of all unknown explanatory features and then predict the prices based on these forecasts.

3.4.1 Forecasting Explanatory Features

In order to figure out which features to forecast, we divided the features into the following categories:

- Truly known values: Time, Year, Month, Day, WeekDay and Hour.
- Known with high certainty: OffshoreWindCapacity, OnshoreWindCapacity and SolarPowerCapacity (as these usually don't change because they are planned and known in advance unless a unit fails).
- Externally forecasted inputs: WindSpeed and Radiation (forecasted by DMI with advanced models).
- Internally forecasted inputs: Foreign prices, production, consumption, and exchange (forecasted by us).

For both the truly known values and the known with high certainty, we used the real historic values as these would be known with very high accuracy and certainty in a real scenario. The weather data would be difficult to forecast, and we couldn't get real historic forecasts, so we had to use the real historic values for these as well. Forecasted weather values with simple forecast models would be too inaccurate. However, this meant that we used weather data that was more accurate than the forecasted values, which would be used in a real scenario. Especially towards the end of the 168-hour prediction horizon, the difference would become more significant as forecasted data tends to become less accurate for longer prediction horizons. This meant that one effect of this decision was, that the test results of the model's performance would be affected in a more optimistic direction. Another effect of using real weather data was, that we would not see the effect that an increasing uncertainty in weather forecasts during the 7-day period would have on the model's performance. Section 6.2.2 describes how we handled this issue. For all the 22 internally forecasted features, we tested 5 different simple forecast models: Lag 1 as baseline, lag 24, lag 168 (1 week), Random Forest and Seasonal Naïve Forecast, SNforecast, from Skforecast. The algorithms behind the Random Forest and Seasonal Naïve forecast is described in further detail in Section 4. The performances of the forecast models are shown in Figure 3.12.

Feature	Model	DK1 SMAPE	Model	DK2 SMAPE	Diff DK1	Diff DK2
OffshoreWindPower	Random Forest	44.80%	Random Forest	51.80%	40.50%	46.90%
OnshoreWindPower	Random Forest	28.10%	Random Forest	36.00%	58.90%	55.80%
HydroPower	Lag24	4.94%	Lag168	0.18%	3.16%	0.00%
SolarPower	Random Forest	29.80%	Random Forest	26.80%	97.40%	99.10%
Biomass	Lag24	38.20%	Lag24	38.50%	4.50%	20.30%
Biogas	SNforecast	5.86%	SNforecast	13.65%	5.04%	15.65%
Waste	Lag24	13.80%	Lag24	12.60%	8.70%	0.80%
FossilGas	SNforecast	57.70%	SNforecast	76.54%	35.10%	38.56%
FossilOil	Lag24	46.90%	Lag24	32.90%	19.10%	13.90%
FossilHardCoal	Lag24	50.50%	Lag24	52.80%	18.60%	28.90%
ExchangeGreatBelt	SNforecast	101.18%	skforeacst	101.26%	27.42%	27.54%
ExchangeGermany	Lag1	108.80%	Lag24	73.00%	0.00%	5.70%
ExchangeSweden	Lag24	68.10%	Lag1	43.70%	2.60%	0.00%
ExchangeNorway	Lag1	58.80%			0.00%	
ExchangeNetherlands	SNforecast	82.98%			50.32%	
DEPrice	SNforecast	37.30%	SNforecast	37.30%	19.90%	19.90%
NO2Price	SNforecast	26.76%	SNforecast	26.76%	2.24%	2.24%
SE3Price	SNforecast	47.46%	SNforecast	47.46%	67.54%	67.54%
SE4Price	SNforecast	47.01%	SNforecast	47.01%	65.79%	65.79%
NLPrice	SNforecast	28.51%	SNforecast	28.51%	25.69%	25.69%
GrossCon	SNforecast	6.32%	SNforecast	4.72%	12.08%	14.98%
TotalProduction	Random Forest	19.70%	Random Forest	23.10%	33.60%	24.00%
Sum	Sum	953.52%		774.60%	598.18%	573.28%
Avg	Avg	43.34%		38.73%	27.19%	28.66%

Figure 3.12: The performance of the models used to forecast the future values of the internally forecasted features, when validating the trained models. Their performance is assessed by the SMAPE metric. The Diff-columns show how much better they perform compared to the baseline lag 1 model. Note: DK2 SMAPE for ExchangeNorway and ExchangeNetherlands is empty because DK2 has no exchange with Norway and the Netherlands.

Figure 3.12 shows that for DK1 the average forecast SMAPE is 43.34% and 17 out of 22 forecasted features have a SMAPE above 25%. For DK2, the average forecast SMAPE is 38.73% and 15 out of 20 forecasted features have a SMAPE above 25%. This means that most of the 22 the explanatory features have significantly different values than the real historic values. Some of our early experiments showed that when models used forecasted values for prediction, the optimal hyperparameter values were different from the optimal values, when models used real historic values for prediction. As our goal was to make models, that performed well on forecasted data input we decided to use forecasted values in the validation process when tuning the hyperparameters as well as training the final models.

3.4.2 The Forecast Models

In this section, we provide a more detailed description of how the different models forecasted the explanatory features. It's important to note that these models are called "forecast models", while the models that predict the price are called "prediction models". The forecast models would always forecast periods of 168 hours at a time, with the first hour noted as t_1. To simulate real forecast scenarios, the forecast models would always have the historical data available up until t_1, meaning the last known historical value would be at t_0.

First, let's look at how the **Lag 1 model** worked. This model would forecast the first hour in the validation period t_1 by copying the last value from the historical data t_0 and then forecast the second hour by copying the first and so on. So we would have $t_0 = t_1 = t_2 = ... = t_{167} = t_{168}$. This means the last known value is used as a constant for the next 168 hours.

The Lag 24 model would forecast t_1 by copying the historical value 24 hours earlier, $t_{1-24} = t_{-23}$ and t_2 by copying t_{-22} and so on until $t_{24} = t_0$. When forecasting t_{25}, no more historical data would be available, so the model would start copying the forecasted values, so $t_{25} = t_1$, $t_{26} = t_2$, and so forth.

The Lag 168 model always copies the historical value from 168 hours earlier, so it does not run out of historical data in the 168-hour forecast period.

The SNforecast model takes the average of the last 3 historical values one week apart, so $t_1 = \frac{t_{-167}+t_{-335}+t_{-503}}{3}$. This means that SNforecast doesn't run out of historical data during the forecast period either. This model is also described in further detail in section 4.1.3. The forecast models described so far have been univariate models predicting the target feature based on earlier values of itself.

The Random Forest models are multivariate forecast models forecasting the target feature based on the values of explanatory features. This model is described in further detail in section 4.2.4. There are 4 Random Forest models, one for each of the four target features: OffshoreWindPower, OnshoreWindPower, SolarPower, and TotalProduction. Table 3.3 shows the explanatory features used to forecast each of the 4 targets. Table 3.3 show the explanatory features used for each of the 4 Random Forest forecast models.

Table 3.3: This table shows the input features used for each of the four Random Forest forecast models. The column headers are the target features and the rows in each column contain the explanatory features used to forecast the target. Note that some feature names has been shortened: OffshoreWindPower -> OffshorePower, OnshoreWindPower -> OnshorePower, TotalProduction -> TotalProd.

OffshoreWindPower	**OnshoreWindPower**	**SolarPower**	**TotalProduction**
WindSpeed	WindSpeed	Radiation	WindSpeed
OffshoreCapacity	OnshoreCapacity	SolarCapacity	Radiation
OffshorePower_lag24	OnshorePower_lag24	SolarPower_lag168	OffshoreCapacity
OffshorePower_lag168	OnshorePower_lag168	Month	OnshoreCapacity
Month	Month	WeekDay	SolarCapacity
WeekDay	WeekDay	Hour	TotalProd_lag24
Hour	Hour		TotalProd_lag168
			Month
			WeekDay
			Hour

The Random Forest needs data from the prediction hour to create the forecasts for that hour, but these data are either lags, truly known values, values known with high certainty, or external forecasts, which is all data that would be available for the prediction hour in a real scenario. As mentioned earlier, WindSpeed, Radiation, and the three capacity features always have true historical values as they do not belong to the Internally forecasted values category. The Offshore- and OnshoreWindPower_lag168, SolarPower_lag168 and TotalProduction_lag168 will not run out of historical data as they look as far back in time as the length of the forecast period (168 hours). The Month, WeekDay and Hour features belong to the Truly known values category, so they will also have true historical values. The last features, the lag24 features, use historical data in the beginning until t_{24} (which is equal to t_0), and from then on they use their own forecasted values recursively. Thus, like the other forecast models, the Random Forests are designed to forecast future values based on historical data, available up until t_1 and known future data. The Random Forests are trained on the period 2021-01-01 to 2023-12-31 just like the prediction models. This means, we have no information leakage from 2024 into the forecast models.

In conclusion, all forecast models are designed to simulate a real forecast scenario, where true historical data only are available until the prediction period begins and from then on, it uses either last known values, external forecasts (weather features), truly known values, values known with high certainty or its own earlier forecast values to forecast the explanatory features for the prediction hour needed for the prediction models.

3.4.3 The Evaluation Process

As shown in Figure 3.11, the validation window (the entire year of 2024) consisted of 4 periods of 4 weeks with a stride of 13 weeks between the beginning of each period. Each of the 4 weeks were evaluated separately, one hour at a time. This meant that for the first week (or 168 hours) in 2024, the first hour was predicted by starting with the forecast models creating the forecasted values for explanatory features for that hour. In order to create these forecasts, some of them needed real historical data from the hour before (the last hour of 2023), some needed data from 24 hours before, some 168 hours before, and some needed data from 3 weeks (504 hours) prior. All of this data was available as real historical data for the first hour in 2024, as we simulated a prediction scenario where the present was the hour before the prediction hour and all information about the present and the past was known as well as the future values of the weather features, capacity features, and calendar features.

When the forecasted values for the first hour in 2024 were created, this data was used as input for the prediction model, which then created a price prediction for that hour. Then for the second hour in 2024, the Lag 1 forecast model used the forecasted values from the first hour to forecast the second hour and the rest of the forecast models still used historical data or known values for the second hour. When all forecasted values were created for the second hour, the prediction model predicted the price based on the explanatory feature values. This process was repeated for all 168 hours. When every hour in the first week was predicted all the predictions were evaluated by computing the metrics described in section 3.4.4.

Then the evaluation process started over for the second week in 2024, where the present hour was the 168th hour in 2024 (the last hour of the first week) and the prediction hour was the 169th hour of 2024. Here the first week was considered known as past historical and present data with which the forecast models could create the forecasts for the first hour of the second week. This way we could choose any 168 hour blocks across 2024 for validation, as forecast models would always have the necessary data available to create the data needed for the prediction models.

The evaluation process is very similar for the deep learners, but with small differences. These are described in Section 3.6.

3.4.4 Evaluation Metrics

To ensure that the results could be compared to the existing literature, three evaluation metrics were used throughout the hyperparameter tuning and the final test. They were; Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and SMAPE (Symmetric Mean Absolute Percentage Error). We found that the majority of literature used RMSE as an evaluation metric. A large number also used MAE. SMAPE has been included as it is a percentage and puts the error into a context that does not require domain knowledge. Since SMAPE uses a percentage rather than absolute values, it is clearer how big a deviation or error is.

3.4.4.1 Root Mean Squared Error

Root Mean Squared Error (RMSE) calculates the average difference between a model's predicted value and the actual values from the data. Essentially, it calculates the standard deviation of the residuals. RMSE can take any value from 0 up to positive infinity, which means that it has no upper bound, and it takes on the same unit as the output. If RMSE is equal to 0, then the predicted values are the same as the actual values, which means that a lower RMSE is an indicator that the model is fitting the data well and making precise predictions. A larger RMSE

means that the model is not very precise. The RMSE of a model shows the overall difference between the predicted values and the accurate values. The equation for calculating RMSE can be seen in equation 3.2, where n is the number of observations, y_i is the actual value, and $\hat{y}_i$ is the predicted value [27].

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{3.2}$$

3.4.4.2 Mean Absolute Error

Mean Absolute Error (MAE) is used to evaluate the accuracy of a model. It calculates the average of the absolute differences between the predicted values and the actual values. Just as with RMSE, MAE uses the same unit as the output variable. Because it finds the absolute error, MAE can take on any value from 0 to positive infinity, which means that it does not have an upper bound. A lower MAE means that the model is very accurate and that the predicted values are very similar to the actual values. A large MAE is an indicator that a model is not very accurate. The equation for calculating MAE can be seen in equation 3.3 where n is the number of observations, y_i is the actual value, and $\hat{y}_i$ is the predicted value [28].

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{3.3}$$

3.4.4.3 Symmetric Mean Absolute Percentage Error

Symmetric Mean Absolute Percentage Error (SMAPE) is a relative of MAE and Mean Absolute Percentage Error (MAPE). Just like MAE, MAPE is a measure of the accuracy of a model, except that this is expressed as a percentage. MAPE calculates the absolute difference between the predicted and actual values for each observation, which is then divided by the actual value in order to obtain a percentage. Each of the percentage differences are then averaged and multiplied by 100 % to obtain the final MAPE. MAPE has a lower bound of 0 %, but no upper bound. A MAPE of 0 % means that the predicted values and the actual values are the same, which means that the model is completely accurate. As MAPE increases, it indicates that the model is becoming less accurate [29].

SMAPE uses the same principles as MAPE, but rather than just dividing the difference by the actual values, it divides the difference by the average of the prediced and the actual value. This functions as a normalization of the error, which makes it possible to compare SMAPE values across different datasets. This normalization also means that there is an upper bound of 200 %, while the lower bound remains the same. The presence of both a lower- and an upper bound is where "Symmetric" comes from. The equation for calculating SMAPE can be seen in equation 3.4, where n is the number of observations, y_i is the actual value, and $\hat{y}_i$ is the predicted value [30].

$$\text{SMAPE} = \frac{100\%}{n} \sum_{i=1}^{n} \frac{|y_i - \hat{y}_i|}{(|y_i| + |\hat{y}_i|)/2} \tag{3.4}$$

3.4.4.4 Application

The evaluation metrics were computed across different lengths of time. During the cross validation, the three evaluation metrics were calculated for each week (168 hour period) within a fold and for each day (24 hour period) within that week. Based on the weekly metrics within the same fold, an average value for each metric was calculated on a fold basis. An overall weekly average across all folds for each metric was also calculated, along with an overall daily average across all weeks and folds. An overall average for each forecasted day for each 168 forecasting period was also calculated, however this was only done for SMAPE.

When the results of hyperparameter tuning or final evaluation were saved, only some of the calculated metrics were saved to a csv file. The metrics saved were:

- The average weekly RMSE
- The average weekly MAE
- The average weekly SMAPE
- The average daily RMSE
- The average daily MAE
- The average daily SMAPE
- The average SMAPE for day 1 during the forecasting period
- The average SMAPE for day 2 during the forecasting period
- The average SMAPE for day 3 during the forecasting period

- The average SMAPE for day 4 during the forecasting period
- The average SMAPE for day 5 during the forecasting period
- The average SMAPE for day 6 during the forecasting period
- The average SMAPE for day 7 during the forecasting period

3.5 Shallow Learners

This section describes the details of the methodology that only applies to the shallow learners.

When tuning the hyperparameters of the shallow learners, we usually searched over a grid mostly containing 3 values for each of the most important hyperparameters, resulting in about 60 to 100 combinations for each model in each price zone. Normally when tuning hyperparameters, multiple searches would be conducted, narrowing down the search grid each time in specific promising areas of the search space to find the optimal values. As we only had very limited time we only conducted one search for each model in each price zone. This means there are parameters that we did not include in the search and the best values we found are probably not the globally optimal values, but we do have a picture of the potential of each model and a relatively optimized version of each model to compare to the others.

For each combination, the model was trained once on the period from 2021-01-01 00:00:00 to 2023-12-31 23:00:00 and validated on 4·1 week periods with a stride of 13 weeks across the full year of 2024, which means January, April, July and October, as illustrated in Figure 3.11. During the validation process, the models were evaluated on 1 week at a time, or 168 hours, by predicting one hour at a time, so the Price_lag1, Price_lag 24, GrossCon_lag1 and GrossCon_lag24 features and all the forecasted features could be updated each hour during the process.

The predicted prices were then compared to the true prices and the SMAPE, MAE, and RMSE were calculated for each of the 168 hours. Then the average of each metric was calculated for each day in the week and for the whole week. This was done for every week in all the validation folds. Then for each metric the average of the weekly averages was computed to get an overall average performance. For SMAPE, we also computed the average of day 1 across all weeks, and for day 2 and so on, so we could see how the performance changed on average across a 168 hours prediction horizon.

When the best hyperparameter values were identified for a model, we trained the final model. As described in Section 3.3, we found out that training a model on the 3 most recent years of data gave the best performance on the following year, so we trained the final models on the full years of 2022, 2023, and 2024. The model was then tested on the full year of 2025 by using the same evaluation method as we did during hyperparameter tuning; making the model predict one hour at a time based on forecasted and known explanatory features and restarting the process every 168 hours. The performance of the final models was evaluated with the same metrics as during hyperparameter tuning.

And finally, a SHAP-analysis was conducted to see which features had the highest impact on the predictions.

3.6 Deep Learners

The following sections describe, how the method was adjusted from being used for shallow learners to working for deep learners.

3.6.1 Data Splitting

The second half of the project focused on the deep learners and at this point we realised that time was more scarce than we had expected. Up until now we had been working with both price zones DK1 and DK2, but the process of tuning, training and testing for DK2 was always a repetition of the process for DK1. While the results of hyperparameter tuning for the same model across both bidding zones yielded different combinations, essentially running the same code twice would limit the number of deep learners examined. For these reasons, we decided to only work with DK1 for the deep learners.

Early in the project we found out that training the models on the most recent two to three years of data gave the best performance. Based on this, we decided to train all final shallow learners on the 3 most recent years of training data from 2022-01-01 00:00:00 to 2024-12-31 23:00:00. In order to be able to compare the shallow learners and deep learners on equal terms, we wanted to train the deep learners on the same data as the shallow learners. However, deep learners need to be validated during training to avoid overfitting and to be able to use early stopping, meaning that we still needed a separate validation set during training of the final model. All of 2024 had so far been reserved for validation during hyperparameter tuning, but for shallow learners we only used 4 months of 2024 for validation.

With the aim to use the most recent data for training while still validating the models during training, we decided to validate the deep learners on the same 4 months of 2024 as we did for the shallow learners and include the rest of 2024 in the training data. This decision resulted in the train-validation split shown in Figure 3.13.

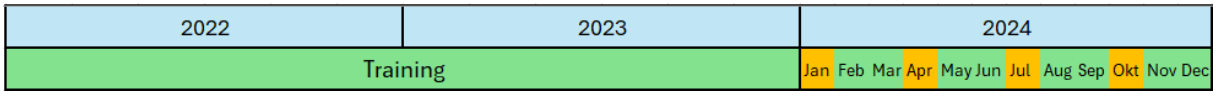


Figure 3.13: The training and validation split first considered for the deep learners. The green areas represent the training periods and the orange areas represent the validation periods.

This split ensured validation across yearly seasons, while still using most of 2024 for training. However, a disadvantage of this split was that validation periods were mixed into the training window. Hewamalage et al. pointed out that such a split would break the temporal order of the data, which could have multiple negative effects on the performance of the final model [31]. For our project, we considered the potential consequences to be:

1. If the temporal order was broken, trends and patterns in the data might be disrupted, which would make it more difficult for the model to learn these.
2. The model would be trained on future data and thus it would know the price distribution, seasonal behavior, and volatility of 2024 before validating on earlier periods in the same year. This is data leakage and might lead to optimistic validation results, which might be misleading for early stopping and model selection.
3. The input sequences for the first validation hours of the training periods that follow immediately after the validation periods would contain data from the validation period, which is also information leakage.

These were significant issues, so we decided to use a padding of 168 hours after each validation period. The padding meant that we excluded these hours from the training data, but still used these hours for the input sequence and forecast models. This way the input sequence did not contain data from the validation period and the trends in the input sequence were not disrupted in the middle of an input sequence. However, the issue of training on future data and predicting on past data resulting in information leakage still remained. As mentioned above the potential consequences of this could be optimistic validation results, that might affect early stopping and model selection. In order to assess if the tradeoff was worth this risk, we tested the effect of including the remaining months of 2024 in the training data on the performance of the final model. Because we had limited time we only tested it for the RNN and LSTM models, which meant we could not know for sure if the results would apply to the other deep learners as well,

but it gave us a good indication of the effect of using most recent data for training. For both the RNN and LSTM, we tuned the hyperparameters and trained the final model once with 2024 included in the training data and once with 2024 excluded from the training data. The results are listed in Table 3.4 and as they show, the performance of both models was slighty worse on the validation set and significantly worse on the training set, when 2024 was not included in the training data. We used early stopping so the final models were picked based only on the validation SMAPE. The models with significant worse train SMAPE were underfitted and improved their performance when trained for more epochs, but then the validation SMAPE would decrease, leading to an overall worse performance.

Table 3.4: This table shows the results of including and excluding the remaining months of 2024, not used for validation, in the training data. Both the RNN and LSTM are tested and the performance of the models are evaluated using the validation SMAPE to compare the models.

Model	2024	Train SMAPE	Val SMAPE
RNN	Incl	50.58	**50.48**
RNN	Excl	77.09	52.43
LSTM	Incl	33.36	**51.43**
LSTM	Excl	72.75	52.29

If the results only showed a better performance on the validation data, when 2024 was included in the training data, they would not tell us much about the models' ability to generalize to unseen data. The results might just indicate data leakage. But the fact that the models trained on data including 2024 performed better on both the training data, containing 2022, 2023, and the validation data, suggested that the models were actually better at generalizing. These results did not guarantee a better performance on all unseen future data, but they suggested that the models might actually be better fit to the entire period of 2022-2024, which was the most recent data and therefore the data most similar to future data. Based on these results, we decided to include the remaining months of 2024 in the training data for the deep learners. Thus the final training-validation split for the deep learners including the remaining months of 2024 in the taining data and having 168 hours of padding after each validation period is illustrated in Figure 3.14.

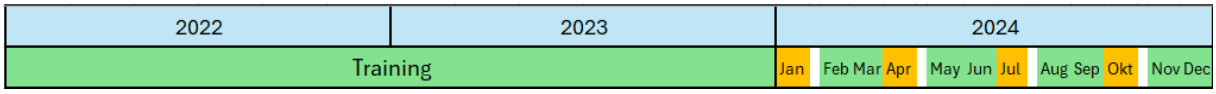


Figure 3.14: The training and validation split used for training of the deep learners. The green areas represent the training periods and the orange areas represent the validation periods. The white areas are periods used for padding for the input sequence and forecast models.

3.6.2 Input Sequence Structure

In this project, we tested different types of recurrent neural networks for our time series data, such as Recurrent Neural Networks (RNN), Gated Recurrent Units (GRU) and Long Short Term Memory (LSTM). These models all function by receiving a sequence of data as input, learning patterns and trends in the sequence and using this information to make a prediction for the next hour that follows right after the sequence in temporal order. These models are also described in further detail in section 4.3. For our project, the input sequence could have been structured like the sequence illustrated in Figure 3.15, where the sequence length was 24 time steps, containing past and present values of n explanatory features and the target feature. Time steps t-23 to t-2 belonged to the past and time step t-1 belonged to the present, containing present values of the explanatory features and the current target value. Time step t was the next hour in the future, where the target value was unknown. With this setup, the model would predict the price based on trends of both explanatory features and the target features on past and present data values.

	Sequence (24)						
	Past					Present	
Time steps	T-24	T-23	T-22	...	T-2	T-1	Prediction
Feature 1	x_t-24,1	x_t-23,1	x_t-22,1	...	x_t-2,1	x_t-1,1	
Feature 2	x_t-24,2	x_t-23,2	x_t-22,2	...	x_t-2,2	x_t-1,2	
...	...	...	...	...	...	...	
Feature n	x_t-24,n	x_t-23,n	x_t-22,n	...	x_t-2,n	x_t-1,n	
Target feature	y_t-24	y_t-23	y_t-22	...	y_t-2	y_t-1	→ y_t

Figure 3.15: What the input sequence would usually look like, when using an RNN on a time series dataset. The sequence length in this example is 24 time steps, there are n features and past values of the target feature are included in the input sequence.

In our project, it was very important that the models would also receive the values of the explanatory features for the future prediction hour as input, because they described the conditions that affected the price in that hour. This required the model learning the relationship between the explanatory features and the target feature, DKPrice. This was the reason the model needed to receive a sequence that contained both the target feature and the explanatory features, and for the last time step in the sequence, the explanatory features needed to be for the prediction hour. In this setup of the input sequence, the first 22 time steps, t-23 to t-2, belonged to the past, time step t-1 belonged to the present, and time step t was the next hour in the future. This setup is illustrated in Figure 3.16.

	Sequence (24)							
	Past					Present	Future	
Time steps	T-23	T-22	T-21	...	T-2	T-1	T	Prediction
Feature 1	x_t-23,1	x_t-22,1	x_t-21,1	...	x_t-2,1	x_t-1,1	x_t,1	
Feature 2	x_t-23,2	x_t-22,2	x_t-21,2	...	x_t-2,2	x_t-1,2	x_t,2	
...	...	...	...	...	...	...	...	
Feature n	x_t-23,n	x_t-22,n	x_t-21,n	...	x_t-2,n	x_t-1,n	x_t,n	
Target feature	y_t-23	y_t-22	y_t-21	...	y_t-2	y_t-1		→ y_t

Figure 3.16: What the input sequence would look like, with explanatory features and target values in the historical and present part of the sequence and forecasted explanatory feature values for the prediction hour. The sequence length in this example is 24 time steps, there are n features and past values of the target feature are included in the input sequence.

The problem with this setup was that the last time step in the input sequence did not have the same shape as the other time steps in the sequence, as it was missing a value for the target feature. We would need to fill that empty space with a value as a placeholder for the target value. We tested two different ways of solving this problem. The first solution was to set the future target value to 0 and in an attempt to avoid that the model would think the DKPrice was actually 0, we added a binary feature to the input sequence taking the value 1 for all time steps except the last one, where it took the value 0. This binary feature was intended as a mask to tell the model if it should consider the target value as known or unknown. This setup is illustrated in Figure 3.17.

	Sequence (24)							
	Past					Present	Future	
Time steps	T-23	T-22	T-21	...	T-2	T-1	T	Prediction
Feature 1	x_t-23,1	x_t-22,1	x_t-21,1	...	x_t-2,1	x_t-1,1	x_t,1	
Feature 2	x_t-23,2	x_t-22,2	x_t-21,2	...	x_t-2,2	x_t-1,2	x_t,2	
...	...	...	...	...	...	...	...	
Feature n	x_t-23,n	x_t-22,n	x_t-21,n	...	x_t-2,n	x_t-1,n	x_t,n	
Target_mask	1	1	1	...	1	1	0	
Target feature	y_t-23	y_t-22	y_t-21	...	y_t-2	y_t-1	0	→ y_t

Figure 3.17: One of the input sequence setups used for the deep learners, with explanatory features and target values in the historical and present part of the sequence and forecasted explanatory feature values for the prediction hour. The sequence length in this example is 24 time steps, there are n explanatory features, a target mask and a 0 as a placeholder for the target value at the prediction hour.

The intention with this setup was that the model would learn the relationship between the explanatory features and the target value, while learning when to consider the target value as known or unknown. We had one concern with this setup though. The mask value would always be 1 at all time steps in the sequence except for the last one, so it might be difficult for the

model to learn the meaning of the mask. For this reason, we also tested a second solution. In the second solution, we set the target feature to lag 1 hour behind the other features during the input sequence. This way we could fill out the target value for the prediction hour with the last known price. This setup is illustrated in Figure 3.18.

	Sequence (24)							
	Past					Present	Future	
Time steps	T-23	T-22	T-21	...	T-2	T-1	T	Prediction
Feature 1	x_t-23,1	x_t-22,1	x_t-21,1	...	x_t-2,1	x_t-1,1	x_t,1	
Feature 2	x_t-23,2	x_t-22,2	x_t-21,2	...	x_t-2,2	x_t-1,2	x_t,2	
...	...	...		...	...	...	...	
Feature n	x_t-23,n	x_t-22,n	x_t-21,n	...	x_t-2,n	x_t-1,n	x_t,n	
Target feature	y_t-24	y_t-23	y_t-22	...	y_t-3	y_t-2	y_t-1	→ y_t

Figure 3.18: The other input sequence setup used for the deep learners, with explanatory features and target values in the historical and present part of the sequence and forecasted explanatory feature values for the prediction hour. The sequence length in this example is 24 time steps, there are n explanatory features and the target value at each hour is set to lag 1 hour behind the other features.

Both of these setups were tested for both the RNN and the LSTM models with a full hyperparameter search for each setup followed by a training of a final model with early stopping. The results are listen in Table 3.5. The results show that the lag 1 setup performed better than the mask setup for both the RNN and the LSTM model evaluated with the validation SMAPE. Even though the price mask setup had a better training SMAPE on the RNN than the lag 1 setup, its validation SMAPE was worse, which suggested that the mask setup model was overfitting on the training data. The difference was very small on the validation set for the LSTM but the difference was significant on the training set. This suggested that the lag 1 model was better on both training and validation and thus it might be better at generalizing. Based on these results, we decided to use the lag 1 setup for all the deep learners.

Table 3.5: This table shows the results of using the two different input sequence setups for the RNN and LSTM models. The performance of the models are evaluated using the SMAPE metric to compare the models.

Model	Setup	Train SMAPE	Val SMAPE
RNN	DKPrice mask	35.42	53.29
RNN	DKPrice lag1	50.58	**50.48**
LSTM	DKPrice mask	81.53	51.56
LSTM	DKPrice lag1	33.36	**51.43**

3.6.3 Training and Evaluation

As the deep learners are more complex and customizable than the shallow learners, the hyperparameter search for the deep learners was a bit more extensive than it was for the shallow learners. The number of combinations tested for each model was around 100 to 160 combination in the first search followed by 1 or more smaller grid searches narrowing down the search space. Some of the models were also tested with both input sequence setups doubling the number of combinations. Some were also tested with most of 2024 included in the training data and without 2024 as described in Section 3.6.1.

During the hyperparameter search for each of the deep learners, the run time for each epoch was a couple of minutes with only 2-3 seconds spent on training and the remaining time spent on validation. This run time combined with the high number of combinations made the hyperparameter search a very time-consuming process. As the validation part of each epoch was the most time consuming part by far, we tested the effect of reducing the validation time from 4 times 4 weeks to 4 times 1 week. Figure 3.19 shows the validation curves for two identical models trained on the exact same hyperparameters, but validated on different fractions of the validation data. The grey model was only validated on the first week of every validation period and the red one was validated on all 4 weeks of each validation period. The two curves are very similar as they have similar shapes and only have a vertical distance of less than 10 in SMAPE, while run time for an epoch was reduced to one fourth of the original time. If all models' validation curves were displaced in the same way by reducing the validation period, the hyperparameter search would still yield the same best hyperparameters. The experiment was only conducted for one model, which is a very weak basis for making a decision like this, but under the time pressure we felt at that point, we decided to take the risk in order to have time to test more models. The consequences of this decision are further examined in the discussion section.

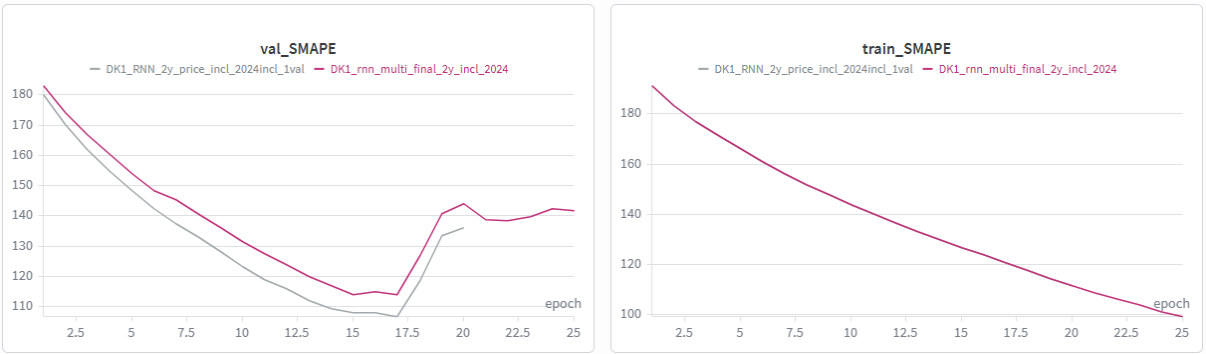


Figure 3.19: The comparison of the validation curves for two identical models validated on different fractions of the validation data. The grey curve is validated on 1 week periods and the red curve is validated on 4 week periods. The curves of the training SMAPE are on top of each other.

Based on this little test, we decided to only validate on 4 times 1 week during the hyperparameter search. The training set was unchanged, so the resulting split for the hyperparameter search was as shown in Figure 3.20.

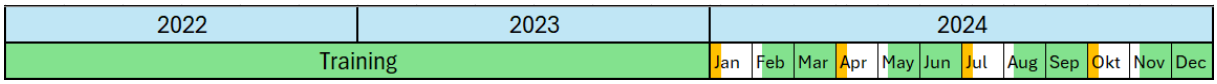


Figure 3.20: The training and validation split used for hyperparameter tuning of deep learners. The green areas represent the training periods and the orange areas represent the validation periods. The white areas are periods used for padding for the input sequence and forecast models.

With most of the models peaking between epoch 15 and epoch 50 the maximum number of epochs for the hyperparameter search was set to around 60 to 100 and the patience was set between 10 and 30 depending on the behavior of the models.

When the hyperparameter search was completed, the best combination was used for training the final model. Here we used the training-validation split illustrated in Figure 3.14 with 4 times 4 weeks of 2024 used for validation, one week of padding after each validation period and the rest of 2024 used for training in addition to 2022 and 2023. Here we also used early stopping to select the best model based on the validation SMAPE.

The model was then tested on the full year of 2025; from 2025-01-01 00:00:00 to 2025-12-31 23:00:00. During testing, the model would receive forecasted explanatory feature values just like during validation for the hyperparameter tuning and training of the final model. Based on the input sequence and explanatory features, the model would predict one hour at a time. Just as with tuning the hyperparameters, the forecasted features would be made in periods of 168 hours and every new period would be forecasted with the preceding historical data and continue recursively through the 168 hours. The model would be evaluated with the same three metrics calculated the same way and for the same time horizons as for the shallow learners.

After testing, the model would also be analyzed with a SHAP analysis to examine which features had the highest impact on the predictions.

4 Models

The following sections contain descriptions of the three overall model categories: baseline, shallow learners, and deep learners. Each model in each of these categories will also be described along with the hyperparameter tuning process of these models.

4.1 Baseline Models

To ensure that the forecasting models perform well, a baseline model is used as a reference to evaluate their performance. Such a baseline model does not use any advanced algorithms, but relies on simple calculations to predict values. The baseline model is used as a benchmark for the more advanced models. If one of these advanced models does not perform better than the baseline, then it may be an indication that there is something wrong with the model, or that the model is not appropriate for the given data set. Many different baseline models exist, but three of the most commonly used models are Last Observed Value also called Naïve Persistence, Moving Average, and Last Equivalent Date also called Seasonal Naïve Forecasting [32]. Each of these models only require the price as input.

4.1.1 Naïve Persistence

Naïve Persistence is a very simple model that assumes that the forecasted value is equivalent to the most recent actual value. This means that the algorithm does not take any trends into consideration. There are no hyperparameters to adjust within the model itself, since it is solely based on the values of the dataset, which also makes it very simple to implement [33].

4.1.2 Moving Average

Moving Average, specifically Simple Moving Average, is an algorithm that calculates the average of a specified number of data points. This model has one hyperparameter to adjust, which is the look-back period (the number of data points to average). As the look-back period increases, it will provide a smoother average, since it considers more data points. It still does not consider trends, since it calculates a simple average for a number of data points [34].

For this project, the look-back period was set to 168 hours. Which means that each forecasted value is the average of the previous 168 values.

4.1.3 Seasonal Naïve Forecasting

Seasonal Naïve forecasting is a slightly more advanced version of the Naïve Persistence algorithm. Naïve persistence only considers the most recent data point, whereas Seasonal Naïve forecasting takes seasonal fluctuations and trends into consideration [35]. Seasonal Naïve forecasting has three main hyperparameters; offset, number of offsets, and the aggregation function. Offset is

a parameter that determines how many steps back in time the model needs to look to find the most recent equivalent date. Number of offsets determines how many of these equivalent dates need to be considered. This will also be dependent on the size of the dataset. If the number of offsets is 1, then only the most recent equivalent date is considered. If it is 2, then the two most recent equivalent dates are considered, which means that the size of the dataset needs to be at least 2 times the offset. The aggregation function is used to predict the forecasted value based on the equivalent dates. This function could be mean or median, depending on what suits the dataset best [32].

As one of the baselines in this project, the offset was set to 168 hours, since it is assumed that there are strong weekly trends within the data. The number of offsets was set to 3 and the aggregation function used was mean. This meant that the predicted value was the mean of the value 168 hours ago, 336 hours ago, and 504 hours ago. In other words, the prediction is the mean of the same hour of the day and same day of the week for the three most recent weeks, which means it should fit into both a daily and a weekly cycle.

4.2 Shallow Learners

The field of machine learning is typically split into two categories; shallow learning and deep learning. Deep learning will be described further in section 4.3. But in simple terms, shallow learners generally require less data and computational power compared to deep learners. However, they are not as strong when it comes to automatically learning complex or abstract features, as is the case with deep learners [36]. The shallow learners examined are Random Forests, XGBoost, Support Vector Regression (SVR), AutoRegressive Integrated Moving Average both with (ARIMAX) and without eXogenous inputs (ARIMA), LightGBM, and Linear Regression. These shallow learners were primarily chosen due to their presence in the studies presented in section 2, but also due to our pre-existing knowledge of each of them.

The shallow learners are modelled and evaluated first to serve as a rough idea of the type of performance that may be expected with the deep learners. While the shallow learners are less complex, they may still achieve a performance similar to the deep learners, which would be beneficial in a computational context. The shallow learners modelled using the dataset described in section 3 are detailed in the following sections.

4.2.1 Support Vector Regression

Support Vector Regression (SVR) is a supervised machine learning algorithm. It works by finding decision planes that separate the data into different classes. The goal is to maximize the margin between these classes. A larger margin means that the model will perform better on new, unseen data compared to a smaller margin. The margin is calculated as the distance from the decision plane to the nearest data points on each side. These data points are the support vectors. SVR is able to handle both linearly separable and non-linear data. This is due to the use of kernels [37].

Kernels are used to transform data into a higher feature space, which will enable SVR to separate the data, even if it is not linear. This allows SVR to draw irregular separation lines, while keeping the computation relatively low. Some of the most used kernels are linear, polynomial, and radial basis function (RBF). The linear kernel is best for linearly separable data and has a fast computation. The polynomial kernel is able to capture more complex relationships than the linear, since it raises the feature interactions to a power. This also allows for a curved decision plane similar to a polynomial. The RBF kernel maps data into infinite dimensions. This kernel is particularly useful, when the boundaries are highly irregular [38].

There are other important parameters for SVR than kernels. C is a regularization term that balances maximizing margin and penalizing misclassifications. A higher C-value means a stricter misclassification penalty [37]. Gamma is a parameter that has a significant influence on the model's performance, particularly when non-linear kernels are used. When gamma is a large value, then the influence of each training example is limited to a small region. In this case, the decision plane will be very tight around the data points, which may lead to overfitting. Instead, if gamma is a smaller value, the decision plane becomes smoother and less complex, which can lead to underfitting [39]. Epsilon is a parameter that is specific for SVR. It is referred to as a margin of tolerance and defines the width of the epsilon-sensitive tube around the regression function. Within this tube, errors are not penalized [40].

4.2.1.1 Hyperparameter Tuning

Using the method described in section 3.5, a number of combinations of parameters were tested. To avoid any "illegal" or useless combinations, the hyperparameter values were selected based on each kernel. The parameters chosen for RBF were:

- C: [0.1, 1, 10, 100]
- Gamma: [scale, auto, 0.01, 0.1]

- Epsilon: [0.01, 0.1, 0.5]

The parameters chosen for the linear kernel were:

- C: [0.1, 1, 10, 100]
- Gamma: [scale]
- Epsilon: [0.01, 0.1, 0.5]

The parameters chosen for the polynomial kernel were:

- C: [0.1, 1, 10, 50]
- Gamma: [scale, auto]
- Epsilon: [0.01, 0.1, 0.5]
- Degree: [2]

C for the polynomial kernel had to be adjusted from having the largest value be 100 to 50. This was due to issues with running the code. The large value made the validation process take a very long time and also caused NaN values to emerge that were not an issue with the previous combinations.

Table 4.1 shows the top 5 best performing hyperparameter tuning combinations according to their overall SMAPE for DK1. The performance of the top 4 combinations are nearly identical, so the first listed combination was selected as the hyperparameters for the final evaluation. Which means that the selected hyperparameters are:

- Kernel: poly
- C: 0.1
- Gamma: auto
- Epsilon: 0.1
- Degree: 2

Table 4.1: The top 5 best performing combinations from the SVR hyperparameter tuning for DK1 based on their overall average SMAPE. All of the best combinations used the polynomial kernel and a C of 0.1.

kernel	C	gamma	epsilon	degree	avg_smape
poly	0.1	auto	0.1	2	49.71
poly	0.1	scale	0.1	2	49.71
poly	0.1	scale	0.5	2	49.71
poly	0.1	auto	0.5	2	49.71
poly	0.1	auto	0.01	2	49.71

The top 5 best performing hyperparameter combinations for DK2 can be seen in Table 4.2. Just as for DK1, the combinations are sorted according to their overall SMAPE. Unlike DK1, where the performance was nearly identical, there is for DK2 an obvious best performer. This means that the hyperparameter combination used for the final evaluation is:

- Kernel: rbf
- C: 1.0
- Gamma: auto
- Epsilon: 0.5
- Degree: default = 3

Table 4.2: The top 5 best performing combinations from the SVR hyperparameter tuning for DK2 based on their overall average SMAPE. All of the best combinations used the rbf kernel and a C of 1.0.

kernel	C	gamma	epsilon	degree	avg_smape
rbf	1	auto	0.5	default	49.58
rbf	1	auto	0.1	default	49.59
rbf	1	auto	0.01	default	49.59
rbf	1	scale	0.1	default	49.60
rbf	1	scale	0.01	default	49.60

4.2.2 XGBoost

XGBoost, also called eXtreme Gradient Boosting, is a gradient boosting algorithm that uses decision trees as base learners. These decision trees are built sequentially, which means that each tree corrects the errors of the previous tree. This is referred to as boosting, which is an ensemble technique. When the model is training, it begins by building a decision tree. After this tree has been trained, it calculates the errors between the actual and predicted values. The next tree is then built and trained using the errors of the previous tree. This process is then repeated, until a stopping criterion is met. The final prediction of the model is the sum of all predictions from all trees. Regularization techniques are also utilized to improve the model's generalization [41].

XGBoost also uses a number of hyperparameters to help improve the model's performance. One of the most important hyperparameters is the learning rate, which may also be referred to as the step size. The learning rate should be a value from 0 up to and including 1. The learning rate determines the rate that the boosting algorithm should learn from each iteration. So if the learning rate is smaller, then the learning will be slower, but this can also prevent overfitting.

Whereas if the learning rate is larger, then the model will learn faster, but it can also lead to overfitting. Max_depth is a hyperparameter that determines how deep each tree can grow during training. This depth is the number of levels from the root node to the leaves. A larger max_depth will lead to a more complex model and there is a risk of overfitting [42].

n_estimators is used to determine the number of trees that the model should build. During each boosting round, a new tree is added to the ensemble, which allows the model to correct the errors that were made by previous trees. This parameter also influences the overall complexity of the model and impacts the time spent training. As the value of n_estimators increases, as does the complexity of the model. This will also allow the model to capture more complicated data patterns, however, it may also lead to overfitting. A general rule of thumb is that if the value of n_estimators increases, then the learning rate should be decreased [42].

Subsample and colsample_bytree are parameters that refer to the number of data used in each tree. Subsample refers to the number of samples (or rows) that should be used in each tree. While colsample_bytree refers to the number of features (or columns) that should be used in each tree. Both parameters have a lower bound of 0 and an upper bound of 1. If they are set to 1.0, then it means that all samples or features should be used [43].

4.2.2.1 Hyperparameter Tuning

Using the method described in section 3.5, the following hyperparameters were tested to determine the optimal combination according to the dataset:

- Max_depth: [3, 5, 7]
- Learning_rate: [0.05, 0.1, 0.2]
- n_estimators: [200, 400, 600]
- subsample: [0.7, 0.9, 1.0]
- colsample_bytree: [0.7, 0.9, 1.0]

The best performing combinations from the hyperparameter tuning for DK1 are shown in Table 4.3. They are sorted according to their overall SMAPE calculated during the hyperparameter tuning. The best performing hyperparameter combination has been selected as the combination for the final evaluation with the parameters listed below:

- Max depth: 5
- Learning rate: 0.05

- Num estimators: 400
- Subsample: 1.0
- Colsample_bytree: 1.0

Table 4.3: The top 5 best performing combinations from the XGBoost hyperparameter tuning for DK1 based on their overall average SMAPE. All of the best combinations have a max depth of 5 and a subsample and colsample_bytree of 1.0.

max_depth	learning_rate	n_estimators	subsample	colsample_bytree	avg_smape
5	0.05	400	1	1	62.09
5	0.1	600	1	1	62.2
5	0.1	400	1	1	62.3
5	0.1	200	1	1	62.3
5	0.05	600	1	1	62.37

Table 4.4 shows the 5 best performing hyperparameter combinations from the hyperparameter tuning of DK2. The best performing combination is selected based on its SMAPE, which means that the hyperparameters used for the final evaluation are:

- Max depth: 7
- Learning rate: 0.05
- Num estimators: 400
- Subsample: 1.0
- Colsample_bytree: 0.7

Table 4.4: The top 5 best performing combinations from the XGBoost hyperparameter tuning for DK2 based on their overall average SMAPE. All of the best combinations have a max depth of 7, a learning rate of 0.05, and a colsample_bytree of 0.7.

max_depth	learning_rate	n_estimators	subsample	colsample_bytree	avg_smape
7	0.05	400	1	0.7	62.07
7	0.05	200	1	0.7	62.17
7	0.05	200	0.9	0.7	62.18
7	0.05	600	1	0.7	62.22
7	0.05	400	0.9	0.7	62.33

4.2.3 Linear Regression

Linear Regression is a relatively simple supervised machine learning model that assumes that there is a linear relationship between the input and output. It predicts the output values by fitting a straight line that best represents the input data. This best fitting line should minimize the difference between the actual output values and the predicted output values. Linear Regression also assumes independence of errors, meaning that errors in predictions should not affect one

another [44]. Ordinary Linear Regression uses very few parameters, and only one of these is relevant in this case, which is fit_intercept. If this parameter is set to True, then the model will calculate the intercept. If it is set to False, then no intercept will be calculated and the model will assume that the data is centered [45].

Two other versions of Linear Regression are Lasso Regression and Ridge Regression, which both utilize regularization to improve the performance of the Linear Regression. Lasso regression adds an L1 penalty term to the loss function, which puts a constraint on the size of the regression coefficients. Using the regularization parameter, it controls the complexity of the model, which helps reduce overfitting and improves prediction stability [46]. The main parameter of Lasso Regression is alpha, which is a constant that multiplies the L1 term and thereby controls the regularization strength. Alpha must be a non-negative number, but it does not have an upper bound. If alpha is set to 0, then no regularization is utilized, and the model effectively functions as a regular Linear Regression [47].

Ridge Regression adds an L2 penalty to the cost function of Linear Regression. This helps it control large coefficient values, which makes the model more stable and reduces overfitting. It also helps the model generalize better on new data. During training, Ridge Regression deliberately introduces some bias by shrinking the coefficients. This means that the model may perform worse during training, but it also reduces variance, which can lead to a better performance on new, unseen data [48]. Just as with Lasso Regression, the main parameter to adjust for Ridge Regression is alpha. Once again, alpha must be a non-negative number that is used to control the regularization strength. If it is set to 0, no ridge regularization will be added and the model will run just like regular Linear Regression [49].

4.2.3.1 Hyperparameter Tuning

Three different types of Linear Regression are run; regular Linear Regression, Lasso, and Ridge. Only one parameter is tested for each of three regression types. Their hyperparameter tuning still follows the method that has been described in section 3.5. Their parameter grids can be seen below.

The parameters chosen for regular Linear Regression are:

- fit_intercept: [True, False]

The parameters chosen for both Lasso and Ridge Regression are:

- Alpha: [0.0001, 0.001, 0.01, 0.1, 1, 10, 50]

The results of the hyperparameter tuning for DK1 featuring the top 5 best performing combinations can be seen in Table 4.5. The best performing combinations are selected based on the overall average SMAPE, which all happen to use Lasso regularization. The hyperparameters selected based on SMAPE for DK1 to be used for the final evaluation are listed below:

- Model: Lasso regression
- Fit intercept: NA
- Alpha: 0.0001

Table 4.5: The top 5 best performing combinations from the Linear Regression hyperparameter tuning for DK1 based on their overall average SMAPE. All of the best combinations used Lasso regularization as part of the Linear Regression.

model	fit_intercept	alpha	avg_smape
Lasso	NA	0.0001	62.42
Lasso	NA	0.001	62.43
Lasso	NA	0.01	62.46
Lasso	NA	0.1	62.84
Lasso	NA	10	65.23

Table 4.6 shows the best 5 performing hyperparameter combinations from the hyperparameter tuning of DK2. The hyperparameter combination with the lowest overall SMAPE has been selected to be used for the final evaluation. These parameters are listed below:

- Model: Lasso regression
- Fit intercept: NA
- Alpha: 50

Table 4.6: The top 5 best performing combinations from the Linear Regression hyperparameter tuning for DK2 based on their overall average SMAPE. Most of the best combinations used Lasso regularization as part of the Linear Regression.

model	fit_intercept	alpha	avg_smape
Lasso	NA	50	68.26
Lasso	NA	10	68.66
Lasso	NA	1	74.94
Ridge	NA	1	78.13
Lasso	NA	0.1	78.40

4.2.4 Random Forest

Random Forest can be used for both classification and regression problems. It is an ensemble learning method, which means that multiple models, in this case trees, are trained in order to make predictions. Utilizing multiple trees means that it is able to make more accurate and stable predictions. When it is used for regression problems such as in this case, it takes an average of the predictions from the tree and uses that as the final prediction. During training, multiple trees are trained on different subsets of the original training set. These subsets are created using replacement, which means that one data point can be present in multiple subsets. During the splitting of nodes, the tree uses a random subset of features. This also means that each tree will be trained using different parts of the data and features, which diversifies the overall model [50].

There are a number of hyperparameters to tune for random forest. One of these is n_estimators, which is the number of trees that should be trained [51]. As the number of trees increases, the model performance is likely to improve, but this will also increase the computational cost [52]. Another is max_depth, which limits the maximum depth of the trees. It is possible to set this parameter to None, which means that nodes will continue to be expanded upon, until all leaves in the tree are pure, or until all leaves contain less than min_samples_split [51]. If the trees are too shallow, there is a risk of underfitting. However, very deep trees may lead to overfitting. It is therefore important to find the appropriate balance between the two [52]. Min_samples_split is a measure of the minimum number of samples required to split an internal node [51]. Min_samples_leaf is used to determine the minimum number of samples required to be at a leaf node. This means that in order to perform a split, there needs to be at least min_samples_leaf training samples in each of the left and the right branches [51].

4.2.4.1 Hyperparameter Tuning

The method for hyperparameter tuning for Random Forest is described in section 3.5. The parameters chosen for Random Forest are:

- n_estimators: [50, 200, 500]
- max_depth: [None, 10.0, 20.0]
- min_samples_split: [2, 20, 50]
- min_samples_leaf: [1, 10, 50]

This search grid results in 81 tested combinations for each price zone. For DK1 the best performing combinations according to the average SMAPE are listed in Table 4.7.

Table 4.7: The top 5 best performing combinations from the Random Forest hyperparameter tuning for DK1 based on their overall average SMAPE.

n_estimators	max_depth	min_samples_split	min_samples_leaf	avg_smape
200	None	2	1	64.47
200	20	2	1	64.51
500	None	2	1	64.53
500	20	2	1	64.69
50	None	20	1	64.70

The parameter values chosen for the final model for DK1 are:

- n_estimators: 200
- max_depth: None
- min_samples_split: 2
- min_samples_leaf: 1

The best performing combinations for DK2 are listed in Table 4.8.

Table 4.8: The top 5 best performing combinations from the Random Forest hyperparameter tuning for DK2 based on their overall average SMAPE.

n_estimators	max_depth	min_samples_split	min_samples_leaf	avg_smape
50	None	2	10	73.72
50	None	2	50	73.80
50	10	20	50	73.86
200	10	20	10	74.24
50	None	20	50	74.26

The parameter values chosen for the final model for DK2 are:

- n_estimators: 50
- max_depth: None
- min_samples_split: 2
- min_samples_leaf: 10

4.2.5 LightGBM

LightGBM, also called Light Gradient Boosting Machine, is an ensemble learning method. It utilizes gradient boosting to construct a strong learner by sequentially adding weak learners. It uses decision trees that are iteratively built while adjusting the tree parameters to optimize the overall performance. By using techniques such as leaf-wise growth and histogram-based learning, it is suitable for large datasets, while keeping computational costs low and accuracy high [53]. Histogram-based learning is used by LightGBM to convert continuous data into discrete bins. This process helps reduce memory and increase training speed. During leaf-wise growth, nodes are split in terms of maximum gain as opposed to splitting nodes level by level. This means that trees may become deeper, but they are also more optimized [54].

To help improve the performance of the model, there are many hyperparameters to adjust. n_estimators, also called num_iterations, determines how many boosting iterations the model should perform. Just as with XGBoost, it is also possible to adjust the learning rate, which is a measure for the step size that the model should use to update its parameters. A small learning rate means that small steps are taken, which means that learning is slow, but it might also be more accurate. A larger learning rate means larger steps and therefore faster training, but there is a risk of missing the optimal solution. Num_leaves is the maximum number of leaves that can be present in a single decision tree. A larger number of leaves yields more complex trees, but if the dataset is small, then it can also lead to overfitting. If the number of leaves is small, then the risk of overfitting decreases, but there is instead a risk of losing some details [55]. Min_child_samples, also called min_data_in_leaf, is the minimum number of samples that a leaf can include. Subsample functions just as it did for XGBoost. It is a measure of how many features should be included in training. If it is set to 1.0, then all features are used. If it is set to 0.8, then 80 % of all features are used during training [56].

4.2.5.1 Hyperparameter Tuning

Hyperparameter tuning for LightGBM followed the method described in section 3.5. The parameters chosen for LightGBM are:

- n_estimators: [50, 200, 500]
- learning_rate: [0.01, 0.1]
- num_leaves: [15, 63, 127]
- min_child_samples: [1, 20, 50]

- subsample: [0.8, 1.0]

With the following parameters being kept consistent during all combinations:

- max_depth: -1
- subsample_freq: 1
- colsample_bytree: 1
- reg_alpha: 0
- reg_lambda: 0

The search grid contained 108 combinations, which were all tested for each price zone. For DK1, the best performing combinations are listed in Table 4.9.

Table 4.9: The top 5 best performing combinations from the LightGBM hyperparameter tuning for DK1 based on their overall average SMAPE.

n_estimators	learning_rate	num_leaves	min_child_samples	subsample	avg_smape
200	0.01	15	1	0.8	56.51
200	0.01	15	20	0.8	56.60
200	0.01	15	50	0.8	56.60
200	0.01	15	20	1.0	56.69
200	0.01	15	50	1.0	56.71

The best combination of values was chosen for the final model for DK1, which were:

- n_estimators: 200
- learning_rate: 0.01
- num_leaves: 15
- min_child_samples: 1
- subsample: 0.8

For DK2, the best performing combinations are listed in Table 4.10.

Table 4.10: The top 5 best performing combinations from the LightGBM hyperparameter tuning for DK2 based on their overall average SMAPE.

n_estimators	learning_rate	num_leaves	min_child_samples	subsample	avg_smape
200	0.01	127	50	0.8	56.68
200	0.01	63	50	0.8	56.86
200	0.01	63	50	1.0	57.08
200	0.01	127	50	1.0	57.22
200	0.01	15	1	1.0	57.27

The combination that performed best was selected for the final model for DK2, which were:

- n_estimators: 200
- learning_rate: 0.01
- num_leaves: 127
- min_child_samples: 50
- subsample: 0.8

4.2.6 ARIMA/ARIMAX

ARIMA (AutoRegressive Integrated Moving Average) utilizes three parameters to predict outputs for time-series data; autoregression, integration, and moving average. Autoregression is a measure of the dependence of the current observation based on its previous values. It is denoted by **p**, and it is possible to have more than one autoregressive term. Integration makes the time series stationary by using differencing of the observations. Differencing means to subtract the current observation from the previous values. It is denoted by **d**, which represents the number of differences that should be performed to make the data stationary. The moving average aspect is similar to the method described in section 4.1.2. Here it is used to depict the error of the model as a combination of the previous error terms, which means that it shows how dependent the current observations is on the previous errors. It is denoted by **q**, which represents the number of lagged forecasts or terms that should be included [57, 58].

ARIMAX (ARIMA with eXogenous inputs) is a more advanced version of ARIMA. It utilizes the same three parameters as ARIMA does, but it also expands upon it by integrating exogenous variables. Exogenous variables are external factors that can impact the data. Using this integration means that the model can utilize additional information, which can improve the accuracy of the model. This additional information is data that is not a part of the time series, but it may still have an impact on it [59].

4.2.6.1 Hyperparameter Tuning

The hyperparameter tuning for ARIMA and ARIMAX follows the method described in section 3.5. The main parameter to tune for these models is order, which is a tuple of (p, d, q). This means that it includes the degree of autoregression, integration/differencing, and moving average to include. The parameters, or orders, chosen for both ARIMA and ARIMAX are:

- order: [(0, 1, 1), (1, 1, 0), (1, 1, 1), (2, 1, 0), (2, 1, 1), (3, 1, 1)]

Each order has $\mathbf{d} = \mathbf{1}$, which means that each combination is differenced once before fitting. **p** has four possible options: 0, 1, 2, and 3. $\mathbf{p} = \mathbf{0}$ means that no autoregressive terms are used, which means that no lagged values are used as predictors, while $\mathbf{p} = \mathbf{3}$ means that three autoregressive terms are used. **q** has two possible values; 0 and 1. When $\mathbf{q} = \mathbf{0}$, no moving average terms are included, which means that no lagged forecasts errors are used. When $\mathbf{q} = \mathbf{1}$, a single moving average term is included.

The 5 best performing combinations for DK1 are listed in Table 4.11 and the 5 best combinations for DK2 are listed in Table 4.12. Since ARIMAX was not featured among the top 5 combinations for neither DK1 nor DK2, it has been left out of the results section.

Table 4.11: The top 5 best performing combinations from the ARIMA hyperparameter tuning for DK1 based on their overall average SMAPE.

Order	avg_smape
(2,1,1)	94.63 %
(3,1,1)	97.74 %
(0,1,1)	132.18 %
(2,1,0)	141.28 %
(1,1,1)	143.08 %

Table 4.12: The top 5 best performing combinations from the ARIMA hyperparameter tuning for DK2 based on their overall average SMAPE.

Order	avg_smape
(2,1,1)	75.92 %
(3,1,1)	76.66 %
(1,1,0)	89.02 %
(1,1,1)	89.03 %
(2,1,0)	89.04 %

The single best performing combination for each price zone were selected for the final models, which were:

- DK1: ARIMA with order (2, 1, 1)
- DK2: ARIMA with order (2, 1, 1)

For ARIMAX the 5 best performing combinations for DK1 are listed in Table 4.13 and the 5 best combinations for DK2 are listed in Table 4.14.

Table 4.13: The top 5 best performing combinations from the ARIMAX hyperparameter tuning for DK1 based on their overall average SMAPE.

Order	avg_smape
(2,1,0)	104.11 %
(0,1,1)	104.37 %
(1,1,0)	104.37 %
(1,1,1)	112.35 %
(2,1,1)	130.83 %

Table 4.14: The top 5 best performing combinations from the ARIMAX hyperparameter tuning for DK2 based on their overall average SMAPE.

Order	avg_smape
(1,1,0)	189.42 %
(0,1,1)	189.43 %
(1,1,1)	189.43 %
(2,1,0)	189.43 %
(3,1,1)	189.43 %

The single best performing combination for each price zone were selected for the final models, which were:

- DK1: ARIMAX with order (2, 1, 0)

- DK2: ARIMAX with order (1, 1, 0)

4.3 Deep Learners

While shallow learners have been developed to handle medium-scale datasets, deep learners were developed to process large, complex datasets. Deep learners are particularly useful in situations, where complex pattern recognition and hierarchical feature extraction are necessary [36]. A number of different types of deep learners exist, where each is particularly useful in certain instances. This project utilizes sequential data, which is handled particularly well by Recurrent Neural Networks (RNN), which are able to handle each input sequence separately. In this process, the RNN stores information about past elements of the sequence in a state vector within its hidden units. This particular feature enables the RNN to understand the context of past data and utilize this understanding to predict future outputs [36]. And it is this feature that also makes RNNs incredibly useful for forecasting electricity prices. The specific RNNs examined are a Simple RNN, Gated Recurrent Units (GRU), Long Short-Term Memory (LSTM), and an LSTM-autoencoder (LSTM AE). These models were selected due to their presence in existing literature as is described in section 2.

Due to time constraints, only the DK1 dataset was examined using the deep learning models. This means that the hyperparameter combinations determined in the following sections are only applicable for DK1, and that the process should be repeated for DK2 to find its best set of hyperparameters for each model.

4.3.1 Simple Recurrent Neural Network

As described above, Recurrent Neural Networks (RNN) are excellent for handling sequential data. They are built with integrated loops, which allows them to retain information from previous steps. This feature is particularly useful, when dealing with language modeling or regression problems. The basic structure of RNNs consists of three layers: Input layer, hidden layer, and output layer. The input layer receives the sequential data, which is then sent to the hidden layer. The hidden layer processes the information from the input layer and maintains information from earlier time steps through recurrent connections. The information from the hidden layer is sent to the output layer, which generates a prediction based on the information. While RNNs are good at predicting short term sequences, they struggle as the time horizon increases due to the vanishing gradient problem [60]. Vanishing gradients occur due to very small gradients during backpropagation. This causes early layers to learn very slowly, but it can also cause them to stop learning completely. This greatly impacts the training, as it can cause the model to forget about earlier information, because the gradients do not carry the learning signals backward. If

the gradients do not properly convey information, then the model will not be able to pick up on the complex patterns within the data. One way this has been mitigated has been through the development of more advanced models such as Gated Recurrent Units (GRU) and Long Short-Term Memory (LSTM) [61].

RNNs consist of several hyperparameters that are tuned in order to improve the overall performance of the final model. The hidden size is the number of features in the hidden state. The number of layers refers to the number of recurrent layers. If this value is 1, then there is only one recurrent layer. If it is two or greater, then two (or more) RNNs are stacked on top of each other. This means that the output from the first RNN is used as input for the second RNN, and so on if there are more layers. Dropout is a measure of how much dropout should be added to the output of each RNN layer, apart from the last layer, which creates a dropout layer. The default value is 0, which means that no dropout is added. If the value is non-zero, then it defines the dropout probability [62]. The batch size is a measure of how large a subset of the training data should be used to train the model. E.g., if the batch size is 64, then the model begins by training the first 64 samples. It then takes the next 64 samples and trains the network again. This process continues, until it has trained on all samples. The learning rate, just as for XGBoost described in section 4.2.2, is a measure of how quickly the model learns. If the learning rate is small, then training is more reliable, but optimization will take longer due to the model having taken smaller steps. Whereas if the learning rate is very high, then the model will take larger steps in order to learn, and it may miss the minimum loss [63]. The sequence length determines the number of past inputs that the model considers when making its predictions. If the sequence length is too small, then the model may miss long-term patterns. But if the sequence length is too big, then memory and computation needs are increased [64].

The RNN defined for the project had to work with the existing pipeline with a few adjustments. The existing pipeline was developed using several functions from Scikit, so the RNN Regressor had to wrap the PyTorch RNN function in order for it to work. The RNN model itself takes advantage of the described architecture in the PyTorch module, which means that it was not necessary to design it from scratch. The most important aspect was to perform the hyperparameter tuning in order to find the optimal combination yielding the best performance.

4.3.1.1 Hyperparameter Tuning

Using the method described in section 3.6.3, hyperparameter tuning for the RNN model was performed. Since RNN is not a very complex or time-consuming model to train, a more comprehensive examination was performed using multiple smaller hyperparameter search grids. The first grid was also the largest to get an overall idea of which parameters yielded the best performance. This initial grid tested 48 combinations, which can be seen below:

- Hidden size: [16, 32]
- Layers: [1, 2]
- Learning rate: [0.001, 0.0005]
- Batch size: [32, 64]
- Sequence length: [24, 168]
- Dropout: [0.0, 0.2]

The grid did not yield any conclusive results, but a batch size of 32 was generally better than 64. The best performing combination used a learning rate of 0.0005. However, several models found their best epoch at epoch 100, which was the total number of epochs. So the maximum number of epochs was also increased to 300 to give the models more time to reach their lowest SMAPE. A new grid was established with 36 combinations:

- Hidden size: [32, 64, 128]
- Layers: [2]
- Learning rate: [0.0003, 0.0005, 0.0007]
- Batch size: [32, 64]
- Sequence length: [24, 168]
- Dropout: [0.0]

In this grid, a hidden size of 64 was the best performer. There was still some variation within best learning rate, so this would be useful to examine further. It would also be useful to test a hidden size between 64 and 128 such as 96. A new grid with 36 combinations were tested to narrow the search field further:

- Hidden size: [64, 96, 128]
- Layers: [2]

- Learning rate: [0.0001, 0.0003, 0.0005]
- Batch size: [64]
- Sequence length: [24, 168]
- Dropout: [0.0, 0.2]

This grid showed that a hidden size of 96 performed best. There was still some variety with learning rate, but a sequence length of 24 was generally better than 168. Dropout of 0.2 was also better than 0.0. So, the next grid focused on a hidden size of 96 and sequence length of 24 to establish the best learning rate and dropout using the following 9 combinations:

- Hidden size: [96]
- Layers: [2]
- Learning rate: [0.0001, 0.0002, 0.0003]
- Batch size: [64]
- Sequence length: [24]
- Dropout: [0.1, 0.2, 0.3]

Based on this grid, the best combination for a hidden size of 96 and sequence length of 24 used a learning rate of 0.0003 and a dropout of 0.2. One final grid of 2 combinations was tested to double-check whether a hidden size of 96 or 128 was the best option:

- Hidden size: [96, 128]
- Layers: [2]
- Learning rate: [0.0003]
- Batch size: [64]
- Sequence length: [24]
- Dropout: [0.2]

This grid confirmed that a hidden size of 96 yielded a lower SMAPE, which meant that the best combination, which should be used for the final evaluation was:

- Hidden size: [96]
- Layers: [2]
- Learning rate: [0.0003]

- Batch size: [64]
- Sequence length: [24]
- Dropout: [0.2]

4.3.2 Gated Recurrent Units

Gated Recurrent Units (GRU) is a more advanced version of a simple RNN model, and they are also well-suited for handling sequential data. The concept behind GRU is to use gating mechanisms to selectively update the hidden state at each time step. This allows the model to remember important information, while discarding what it deems irrelevant. The two main gates are; the update gate and the reset gate. The update gate decides how much information from the previous hidden state should be retained for the next time step, while the reset gate determines how much of the information should be forgotten [65]. These gating mechanisms are the reason why GRU does not have the same risk of vanishing gradients as RNN did. This is due to the gates controlling the flow of information and keeping the gradients stable during training [61]. Some of the advantages of GRU include its ability to handle long-term dependencies within the data due to the update and reset gates, it is also less computationally expensive than more advanced RNNs, and it is useful for both regression and classification problems. Some disadvantages include greater risks of overfitting than with more advanced networks especially when datasets are on the smaller side and it may not perform as well as more advanced models if the task requires modeling very long-term dependencies or complex sequential patterns [66].

GRU uses the same hyperparameters described for RNN in section 4.3.1, and it is also based on the same principle of functioning in a pre-defined pipeline. This means that the GRU Regressor was also structured to be a wrapper of the PyTorch GRU function in order to work with the established cross validation pipeline. The GRU model itself is mainly influenced by the selected hyperparameters, which is why hyperparameter tuning was performed.

4.3.2.1 Hyperparameter Tuning

The hyperparameter tuning for GRU follows the method described in section 3.6.3. To ensure that the hyperparameter tuning was as comprehensive as possible, multiple parameter grids were tested. The initial parameter grid contained a total of 96 combinations, but because it is not possible to apply dropout, when there is only one layer, the number of actually tested combinations is 72. The original parameter grid can be seen below:

- Hidden size: [32, 64, 128]

- Layers: [1, 2]
- Learning rate: [0.001, 0.0005]
- Batch size: [32, 64]
- Sequence length: [24, 168]
- Dropout: [0.0, 0.2]

This grid did not show anything conclusive, but it did show some clear trends. Such as a dropout of 0.2 outperforming 0.0 and 2 layers performing better than 1 layer. Generally, a hidden size of 128 performed better than 64 and 64 performed better than 32, so it may be useful to test 256 as well. A sequence length of 168 was better than 24, so it may also be worthwhile to test 336. This led to the following search grid with 48 combinations:

- Hidden size: [64, 128, 256]
- Layers: [2, 3]
- Learning rate: [0.0005]
- Batch size: [64]
- Sequence length: [168, 336]
- Dropout: [0.2, 0.3, 0.4, 0.5]

In this grid, a hidden size of 256 performed best, but it was dependent on the appropriate dropout value. Generally, 2 layers also performed better than 3 layers. A new grid focusing on hidden sizes of 256 and 512 was established, which would test 24 combinations:

- Hidden size: [256, 512]
- Layers: [2]
- Learning rate: [0.0005, 0.001]
- Batch size: [64]
- Sequence length: [168, 336]
- Dropout: [0.3, 0.4, 0.5]

This grid confirmed that a hidden size of 256 is the best option, it also performed best with a learning rate of 0.0005. A sequence length of 336 did not outperform 168, but it is worth testing a middle ground such as 240. A new grid of 12 combinations used to figure out the best learning rate, sequence length, and dropout can be seen below:

- Hidden size: [256]
- Layers: [2]
- Learning rate: [0.0001, 0.0002, 0.0005]
- Batch size: [64]
- Sequence length: [168, 240]
- Dropout: [0.35, 0.4]

This grid confirmed that the optimal combination uses a sequence length of 168 and a dropout of 0.4. Once again, a learning rate of 0.0005 outperformed the others. But there are still other learning rates that would be worth testing. A small grid of just 4 combinations is set up to ensure what the appropriate learning rate for the model is:

- Hidden size: [256]
- Layers: [2]
- Learning rate: [0.0005, 0.0006, 0.0007, 0.0008]
- Batch size: [64]
- Sequence length: [168]
- Dropout: [0.4]

This grid confirmed that the optimal learning rate for GRU is 0.0008, which means that the hyperparameters used for the final evaluation are:

- Hidden size: [256]
- Layers: [2]
- Learning rate: [0.0008]
- Batch size: [64]
- Sequence length: [168]
- Dropout: [0.4]

4.3.3 Long Short-Term Memory

Long Short-Term Memory (LSTM) is an improved version of RNN and a more advanced version of GRU. It has been designed to capture long-term trends and dependencies within the data. While GRU is controlled by two gates, LSTM utilizes three gates; input gate, forget gate, and output gate. It also uses a so-called memory cell, which is used to store and update important information over time. The input gate is in charge of selecting the information sent to the memory cell. The forget gate controls what information should be removed from the memory cell. And the output gate determines what information from the memory cell should be used as output. This enables the LSTM to selectively retrain or discard any information, which allows it to learn long-term dependencies. This means that the memory cell and the three gates can be seen as the long-term memory of the model. It also has a hidden state, which can be thought of as the model's short-term memory. This memory is updated using the current input, the previous hidden state, and the current state of the memory cell [67]. Just like GRU, the introduction of the three gates in LSTM means that the issues of vanishing gradients have been mitigated. The gates are used to control the flow of information and also to keep gradients stable throughout training. This means that they are much less likely to suddenly become very small, which can cause the model to stop learning [61]. Due to the increased complexity of LSTM, they are much slower to train than RNN and GRU, and they also require more available memory. They are well-suited for longer sequences, but they still face issues with very long-range dependencies [60].

LSTM uses the same hyperparameters described for RNN in section 4.3.1, and just as for both RNN and GRU, the LSTM model is defined using the PyTorch library, which means that it needed a wrapper in order to work with the existing pipeline. This also meant that no major architectural choices were made in the development of the model and that its performance will be dependent on determining the right combination of hyperparameters.

4.3.3.1 Hyperparameter Tuning

After finding out which input sequence setup had the best performance and that lag 1 performed better than the mask setup, a more comprehensive hyperparameter search was conducted. A total of 311 combinations were tested divided into multiple grids. The first grid had 160 combinations, but subtracting the combinations where dropout was applied to a single layer, the number of actually tested combinations was 120:

- Hidden size: [16, 28, 32, 38, 48]

- Layers: [1, 2]
- Learning rate: [0.001, 0.0005]
- Batch size: [32, 64]
- Sequence length: [24, 168]
- Dropout: [0.0, 0.2]

The first search revealed that 2 layers and a dropout rate of 0.2 were generally better than 1 and 2 layers with no dropout. A larger hidden size also generally performed better than a smaller one. Another conclusion was that the high learning rate generally performed better than the low learning rate. The second search examined the effect of a higher dropout rate and a larger hidden size, by searching this grid of 64 combinations:

- Hidden size: [32, 38, 48, 64]
- Layers: [2, 3]
- Learning rate: [0.001]
- Batch size: [32, 64]
- Sequence length: [24, 168]
- Dropout: [0.2, 0.3]

The second search revealed that 3 layers performed better than 2, and batch size 64 performed better than 32. As even larger hidden sizes had performed well on RNN, we tested if this was the case for LSTM as well. Thus the third search grid contained these 32 combinations:

- Hidden size: [96, 128]
- Layers: [2, 3]
- Learning rate: [0.001]
- Batch size: [32, 64]
- Sequence length: [24, 168]
- Dropout: [0.2, 0.3]

The third search did not increase performance. The fourth search tested a larger hidden size and 3 - 4 layers. This grid had 16 combinations:

- Hidden size: [256]

- Layers: [3, 4]
- Learning rate: [0.001, 0.0005]
- Batch size: [64]
- Sequence length: [24, 168]
- Dropout: [0.2, 0.3]

The fourth search did not improve performance either, but 4 layers showed potential. The fifth search tested a smaller hidden size with 4 layers with 4 combinations:

- Hidden size: [192]
- Layers: [4]
- Learning rate: [0.0005]
- Batch size: [64]
- Sequence length: [24, 168]
- Dropout: [0.2, 0.3]

The fifth search did not improve performance, but it did perform very closely to the best one so far, which was an indication that 4 layers was as good or maybe better than 3 layers. So we tested a larger number of layers with 4 combinations:

- Hidden size: [128]
- Layers: [6]
- Learning rate: [0.0005]
- Batch size: [64]
- Sequence length: [24, 168]
- Dropout: [0.2, 0.3]

The performance of the sixth search was the same as the fifth search, so 6 layers was not better than 4 layers. In the seventh search, we went back to the best hidden size, kept 4 layers and tested other learning rates with 4 combinations:

- Hidden size: [128]
- Layers: [4]
- Learning rate: [0.00075, 0.00025]

- Batch size: [64]
- Sequence length: [168]
- Dropout: [0.2, 0.3]

The seventh search did not improve performance, so we ended the search.

4.3.4 LSTM Autoencoder

An LSTM Autoencoder (AE) is the implementation of an autoencoder within an encoder-decoder LSTM architecture [68]. An autoencoder is a type of neural network that compresses input data into a smaller representation and then reconstructs it. This helps the model learn these patterns more efficiently. It can also help remove any unwanted noise and improve the overall quality of the data. This process can also extract the important features which can lead to a better model performance. The architecture of an AE consists of three main components: an encoder, a bottleneck, and a decoder. The encoder is in charge of compressing the input data into a smaller and more manageable form. This is done by reducing its dimensionality while still preserving important information. The encoder consists of three parts: an input layer, hidden layers, and an output layer. The input layer is where the data enters, it then moves on to the hidden layers. Here a series of transformations are performed on the input data by the hidden layers. In each of these layers, weights and activation functions are applied to capture any important patterns and thereby reducing the size and complexity of the data. The result of these transformations moves on to the output layer, also called the latent space. All the information is compressed to a vector known as the latent representation, which contains the important features of the input data [69].

The bottleneck, or latent space, holds the latent representation of the data, which is then sent to the decoder. The decoder consists of hidden layers and an output layer. The hidden layers progressively expand the latent representation, or vector, back to a higher-dimensional space. By performing several transformations, it attempts to restore the original data shape and details. The result of these transformations moves to the output layer, where it attempts to make a final reconstructed output, which ideally should be identical to the original input [69].

Encoder-decoder LSTM was designed to address sequence-to-sequence problems, also called seq2seq. They were designed to handle problems, where the number of items in the input and output sequences might vary. The encoder-decoder LSTM consists of two models. The first of these models is in charge of reading the input data and encoding it to a fixed-length vector. The second model then decodes this vector and outputs the predicted sequence [70]. LSTM AE can be seen as an extension of the encoder-decoder LSTM and can be built in many different ways [68].

The LSTM AE developed for this project uses a feature autoencoder, an encoder LSTM, and a decoder LSTM. The feature autoencoder can be considered as two connected multilayer perceptrons (MLP). An MLP is a fully connected feedforward neural network that consists of multiple layers counting at least an input layer, one or more hidden layers, and an output layer. An MLP can learn complex relationships in the input data. In the input layer, each node corresponds to an input feature. Each of these nodes are connected to each node in the hidden layer. If there are multiple hidden layers, then the nodes of each hidden layer are connected to all nodes in the subsequent layer. The hidden layer processes the information that it has received from the input layer or from another hidden layer. The output layer is in charge of generating the final prediction or result. The output layer has one node for each output feature it needs to generate [71]. The feature autoencoder consists of an encoder and a decoder. The encoder compresses the input data to a lower-dimensional latent representation. The lower-dimensional latent representation output from the encoder is then given as input to the decoder, which then tries to reconstruct the input data given to the encoder. The decoder will only succeed if the encoder has managed to capture all essential information in the latent representation. Thus by making a good autoencoder, we get an encoder that knows how to extract all essential information from the data and represent it in a compressed way.

When making an encoder-decoder LSTM, we are only interested in using the encoder part of the autoencoder as a feature encoder. The purpose of the feature encoder is to compress a sequence of input features to a lower-dimensional latent representation sequence. This sequence of compressed data is then fed to the encoder LSTM, which condenses it to a single hidden state. This hidden state is fed to the decoder LSTM, which also receives the forecasted features for the prediction hour. It uses this compressed information of the past sequence and the forecasted future values to predict the DKPrice for the next hour. The DKPrice and forecasted features for the predicted hour are added to the input sequence for the next prediction, which means the input sequence is a walk forward rolling window sequence. This prediction process continues until all 168 hours have been predicted.

This process also means that the feature autoencoder is trained prior to training the encoder and decoder LSTMs. The feature autoencoder is hyperparameter tuned for its latent dimensions, dense layers, learning rate, batch size, and dropout rate. The latter four have all been described in section 4.3.1. The number of latent dimensions determines what dimensionality the input data should be transformed to [72]. The number of dense layers is equal to the number of linear transformations in each of the two parts, encoder and decoder, of the feature autoencoder. If the autoencoder only has one dense layer, then the full autoencoder consists of an input layer, a bottleneck layer, and an output layer. This would mean there would be no hidden layers when only one dense layer is used and thus no dropout would be applied in such cases. For each search grid the combinations where dense_layers = 1 and dropout > 0.0 are skipped and not included in the count of total number of combinations. Once the set of parameters for the feature autoencoder have been selected, the parameters, weight, and biases of the feature encoder are frozen, while the hyperparameter tuning of the encoder and decoder LSTMs can begin, which use the same hyperparameters as described in section 4.3.1. When the best combination of hyperparameter values for the LSTMs are determined, the weights and biases of the feature encoder are unfrozen and all three models are trained together to get the final LSTM AE model.

4.3.4.1 Hyperparameter Tuning

The hyperparameter search for the LSTM AE started with a search for the feature encoder. A total of 141 combinations were tested for the feature encoder divided into the grids described below. The first grid had 36 combinations, when combinations with dense_layers = 1 and dropout rates > 0.0 are excluded. The hyperparameter search for the feature autoencoder aimed to find a lower dimensional latent representation, that would still capture the essential information in the data. The largest latent dimension of 34 was selected because that was the number of features in the input data, which means we can compare the reduced representations with an unreduced representation. The first grid consisted of the following combinations:

- Latent dimensions: [8, 16, 34]
- Dense layers: [1, 6]
- Learning rate: [0.001, 0.0005]
- Batch size: [32, 64]
- Dropout: [0.0, 0.2]

The first search revealed that the largest number of latent dimensions were the best, no dropout was better than a dropout rate of 0.2, and 1 layer was clearly better than 6 layers. The second search tested both a slightly smaller latent dimension and a latent dimension larger than the input size, while also testing the effect of using 2 layers. The second grid consisted of 40 combinations:

- Latent dimensions: [32, 38]
- Dense layers: [1, 2, 6]
- Learning rate: [0.001, 0.0005]
- Batch size: [32, 64]
- Dropout: [0.0, 0.2]

In the second search, both latent dimensions of 38 and 32 performed better than 34 with 32 being the best so far. 1 layer was still the best number of layers and no dropout was still better than a dropout rate of 0.2. The third search tested two smaller latent dimensions than 32, while also testing if a smaller dropout rate than 0.2 would perform better. Third grid consisted of 24 combinations:

- Latent dimensions: [24, 28]
- Dense layers: [1, 2]
- Learning rate: [0.001, 0.0005]
- Batch size: [32, 64]
- Dropout: [0.0, 0.1]

For the third search the best number of latent dimensions was 28, the best number of layers was still 1 and no dropout was still better than a dropout rate of 0.1. But no combination was better than the best combination in the second search with 32 latent dimensions. The fourth search a latent dimension of 30 was tested with both 1 and 2 layers resulting in 12 combinations:

- Latent dimensions: [30]
- Dense layers: [1, 2]
- Learning rate: [0.001, 0.0005]
- Batch size: [32, 64]
- Dropout: [0.0, 0.2]

30 latent dimensions did not outperform 32, so for the fifth search latent dimensions of 31 and 33 were tested without dropout and only 1 layer, resulting in the following 8 combinations:

- Latent dimensions: [31, 33]
- Dense layers: [1]
- Learning rate: [0.001, 0.0005]
- Batch size: [32, 64]
- Dropout: [0.0]

Number of latent dimensions = 33 proved to be the best so far, but we still wanted to test a larger number as large hidden sizes had proved to be better for the recurrent neural networks, so for the sixth search we tested 48 latent dimensions resulting in 12 combinations:

- Latent dimensions: [48]
- Dense layers: [1, 2]
- Learning rate: [0.001, 0.0005]
- Batch size: [32, 64]
- Dropout: [0.0, 0.2]

As 33 latent dimensions still had the best performance, we ended the search here and moved on to the next step. Step 2 was to freeze the weights and biases of the feature encoder trained on the best combination of hyperparameter values from step 1 and then perform hyperparameter tuning on the encoder and decoder LSTMs. This hyperparameter search had a search grid of 648 combinations:

Encoder parameters:

- Encoder hidden size: [28, 33, 48]
- Encoder layers: [1, 2]
- Encoder dropout: [0.0, 0.2]

Decoder parameters:

- Decoder hidden size: [28, 33, 48]
- Decoder layers: [1, 2]
- Decoder dropout: [0.0, 0.2]

Shared parameters:

- Learning rate: [0.001, 0.0005]
- Batch size: [32, 64]
- Sequence length: [24, 168]

5 Results

These sections will present the results of the final evaluation of each model examined. The final evaluation period covers all of 2025.

5.1 Baseline

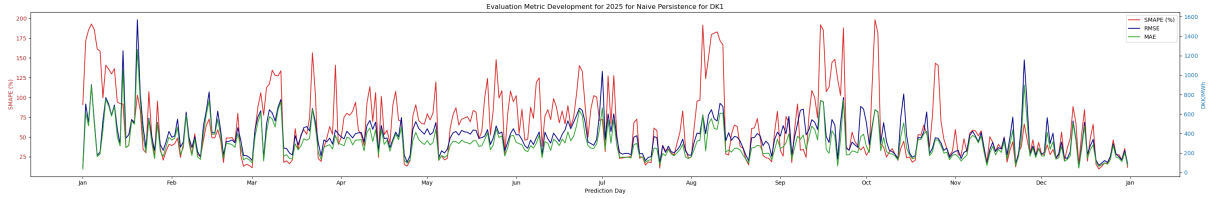
The three baseline models were examined for both DK1 and DK2. The results for each bidding zone are presented and visualized in the following sections.

5.1.1 Naïve Persistence

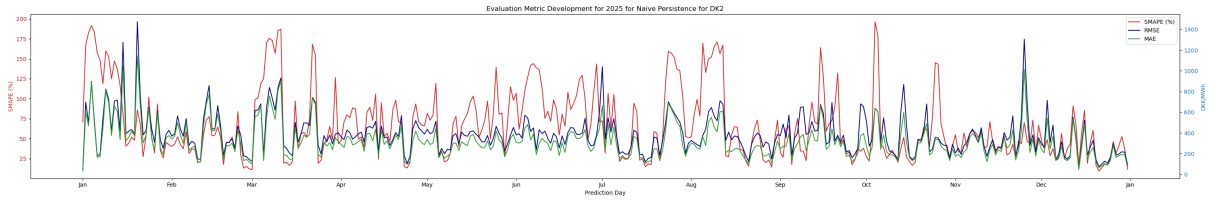
The results of the final evaluation of the Naïve Persistence model described in section 4.1.1 can be seen in Table 5.1. For DK1, the model had an overall average SMAPE of 63.50 %, an overall average RMSE of 392.85 DKK/MWh, and an overall average MAE of 302.81 DKK/MWh. The results for DK2 look a little bit different, here the performance yielded an overall average SMAPE of 66.36 %, RMSE of 405.85 DKK/MWh, and MAE of 316.25 DKK/MWh. The development of these metrics can be seen in Figure 5.1 for each of the two bidding zones, where Figure 5.1a shows the development for DK1 and Figure 5.1b shows the development for DK2.

Table 5.1: Final evaluation results for the Naïve Persistence model for DK1 and DK2 for the evaluation metrics SMAPE, RMSE, and MAE.

Bidding Zone	Test SMAPE	Test RMSE	Test MAE
DK1	63.50 %	392.85 DKK/MWh	302.81 DKK/MWh
DK2	66.36 %	405.85 DKK/MWh	316.25 DKK/MWh



(a) Development of SMAPE, RMSE, and MAE for Naïve Persistence for DK1 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).



(b) Development of SMAPE, RMSE, and MAE for Naïve Persistence for DK2 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).

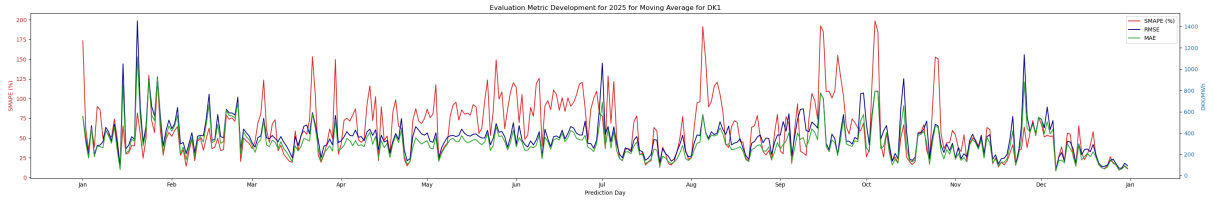
Figure 5.1: Development of SMAPE, RMSE, and MAE for Naïve Persistence for the entire test period. The top figure shows this development for DK1, while the bottom figure shows it for DK2. In each figure, the left y-axis shows the SMAPE (%) and the right y-axis shows the RMSE and MAE (DKK/MWh)

5.1.2 Moving Average

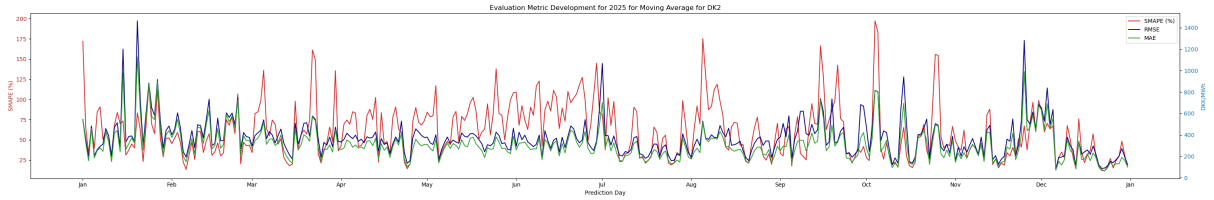
As described in section 4.1.2, the Moving Average model used a window size of 168 for both DK1 and DK2. The results of the final evaluation for each bidding zone can be seen in Table 5.2. DK1 has an overall average SMAPE of 60.31 %, while it is 59.46 % for DK2. The overall average RMSE and MAE for DK1 are 379.09 DKK/MWh and 303.34 MWh/DKK, respectively. For DK2, the overall average RMSE and MAE are 387.59 DKK/MWh and 308.62 DKK/MWh, respectively. Figure 5.2 shows the development of these metrics for both bidding zones, where Figure 5.2a shows the development for DK1 and Figure 5.2b shows it for DK2.

Table 5.2: Final evaluation results for the Moving Average model for DK1 and DK2 for the evaluation metrics SMAPE, RMSE, and MAE.

Bidding Zone	Test SMAPE	Test RMSE	Test MAE
DK1	60.31 %	379.09 DKK/MWh	303.34 DKK/MWh
DK2	59.46 %	387.59 DKK/MWh	308.62 DKK/MWh



(a) Development of SMAPE, RMSE, and MAE for Moving Average for DK1 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).



(b) Development of SMAPE, RMSE, and MAE for Moving Average for DK2 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).

Figure 5.2: Development of SMAPE, RMSE, and MAE for Moving Average for the entire test period. The top figure shows this development for DK1, while the bottom figure shows it for DK2. In each figure, the left y-axis shows the SMAPE (%) and the right y-axis shows the RMSE and MAE (DKK/MWh)

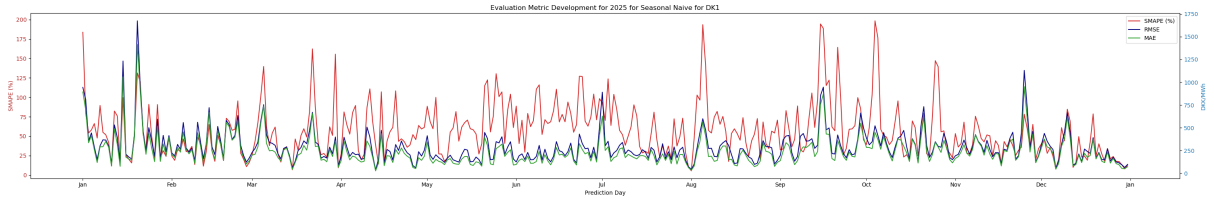
5.1.3 Seasonal Naïve Forecasting

The Seasonal Naïve forecasting model used an offset of 168, a number of offsets of 3, and an aggregation function of mean, which was also described in section 4.1.3. These parameters were also used for its final evaluation, where the results of this can be seen in Table 5.3. The results of the two bidding zones are quite similar; DK1 yielded an overall average SMAPE of 56.59 %, an average RMSE of 318.04 DKK/MWh, and an average MAE of 243.73 DKK/MWh, while DK2

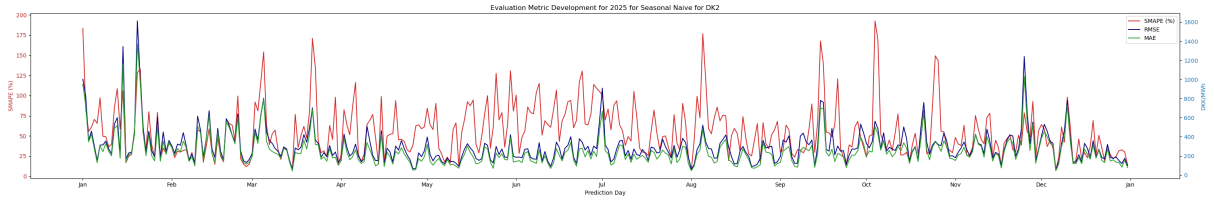
achieved an overall average SMAPE of 55.17 %, an average RMSE of 324.59 DKK/MWh, and an average MAE of 240.42 DKK/MWh. The development of these metrics can be seen for both bidding zones in Figure 5.3. Figure 5.3a shows this development for DK1, while this development for DK2 can be seen in Figure 5.3b.

Table 5.3: Final evaluation results for the Seasonal Naïve Forecasting model for DK1 and DK2 for the evaluation metrics SMAPE, RMSE, and MAE.

Bidding Zone	Test SMAPE	Test RMSE	Test MAE
DK1	56.59 %	318.04 DKK/MWh	243.73 DKK/MWh
DK2	55.17 %	324.59 DKK/MWh	240.42 DKK/MWh



(a) Development of SMAPE, RMSE, and MAE for Seasonal Naïve for DK1 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).

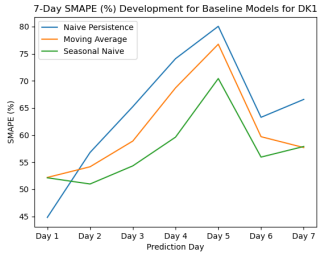


(b) Development of SMAPE, RMSE, and MAE for Seasonal Naïve for DK2 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).

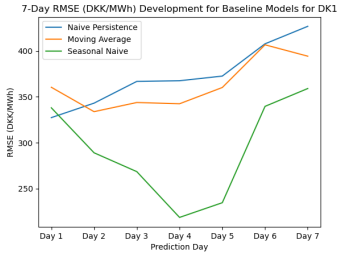
Figure 5.3: Development of SMAPE, RMSE, and MAE for Seasonal Naïve for the entire test period. The top figure shows this development for DK1, while the bottom figure shows it for DK2. In each figure, the left y-axis shows the SMAPE (%) and the right y-axis shows the RMSE and MAE (DKK/MWh)

5.1.4 Consolidated Visualizations

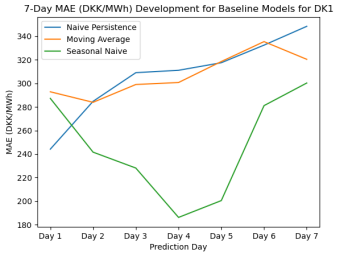
Since the purpose of this project has been to examine the performance of each model in predicting the electricity price across a 168-hour prediction horizon, the results of each baseline model have also been visualized with their 7-day evaluation metrics. To obtain these results, the daily errors were averaged across each day of the prediction horizon. The development of the evaluation metrics across the 7-day prediction horizon can be seen for DK1 in Figure 5.4, where Figure 5.4a shows the development of SMAPE (%), Figure 5.4b shows the development of RMSE (DKK/MWh), and Figure 5.4c shows the development of MAE (DKK/MWh). These same results can be seen for DK2 in Figure 5.5. The development of SMAPE (%) for DK2 can be seen in Figure 5.5a, and the development of RMSE (DKK/MWh) and MAE (DKK/MWh) can be seen in Figure 5.5b and Figure 5.5c, respectively.



(a) Development of SMAPE (%) for each of the three baseline models for DK1 for the 7-day prediction horizon. These values are averages of each 7-day prediction horizon evaluated for the entire test set.

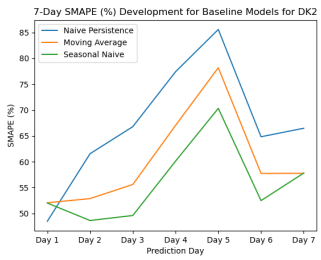


(b) Development of RMSE (DKK/MWh) for each of the three baseline models for DK1 for the 7-day prediction horizon. These values are averages of each 7-day prediction horizon evaluated for the entire test set.

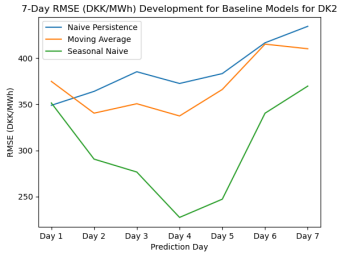


(c) Development of MAE (DKK/MWh) for each of the three baseline models for DK1 for the 7-day prediction horizon. These values are averages of each 7-day prediction horizon evaluated for the entire test set.

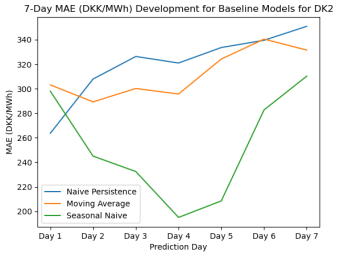
Figure 5.4: Development of the evaluation metrics for each of the three baseline models for DK1 for the 7-day prediction horizon. These values are averages of each 7-day prediction horizon evaluated for the entire test set. The top figure shows the development of SMAPE (%), the middle figure shows the development of RMSE (DKK/MWh), and the bottom figure shows the development of MAE (DKK/MWh).



(a) Development of SMAPE (%) for each of the three baseline models for DK2 for the 7-day prediction horizon. These values are averages of each 7-day prediction horizon evaluated for the entire test set.



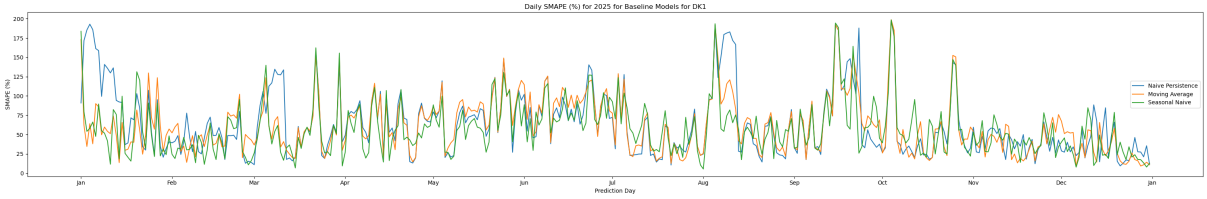
(b) Development of RMSE (DKK/MWh) for each of the three baseline models for DK2 for the 7-day prediction horizon. These values are averages of each 7-day prediction horizon evaluated for the entire test set.



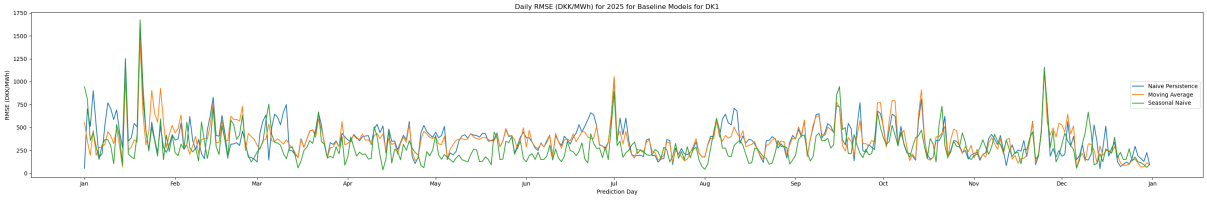
(c) Development of MAE (DKK/MWh) for each of the three baseline models for DK2 for the 7-day prediction horizon. These values are averages of each 7-day prediction horizon evaluated for the entire test set.

Figure 5.5: Development of the evaluation metrics for each of the three baseline models for DK2 for the 7-day prediction horizon. These values are averages of each 7-day prediction horizon evaluated for the entire test set. The top figure shows the development of SMAPE (%), the middle figure shows the development of RMSE (DKK/MWh), and the bottom figure shows the development of MAE (DKK/MWh).

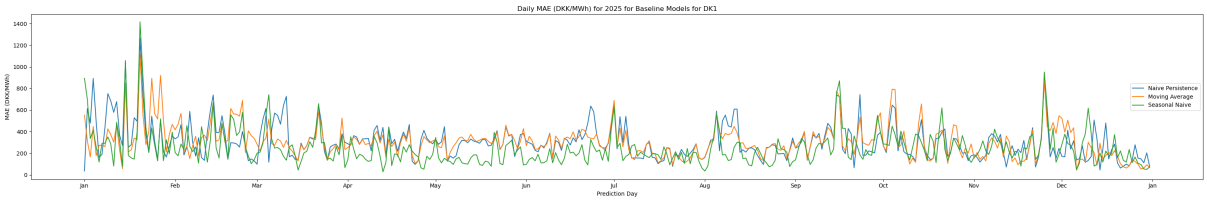
The various evaluation metric developments for each baseline model have also been summarized into consolidated visualizations making it possible to compare their individual development. These consolidated visualizations can be seen for DK1 in Figure 5.6, where Figure 5.6a shows the development of SMAPE (%), Figure 5.6b shows the development of RMSE (DKK/MWh), and Figure 5.6c shows the development of MAE (DKK/MWh). The same consolidated visualizations have also been made for DK2, which can be seen in Figure 5.7. The development of SMAPE (%), RMSE (DKK/MWh), and MAE (DKK/MWh) can be seen in Figure 5.7a, Figure 5.7b, and Figure 5.7c, respectively.



(a) Development of SMAPE (%) for each of the three baseline models for DK1 for the entire test period.

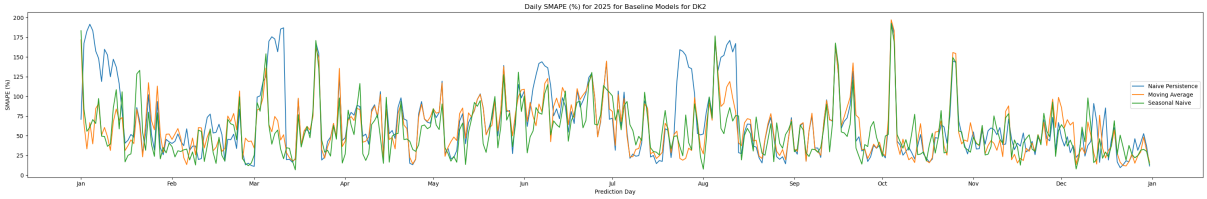


(b) Development of RMSE (DKK/MWh) for each of the three baseline models for DK1 for the entire test period.

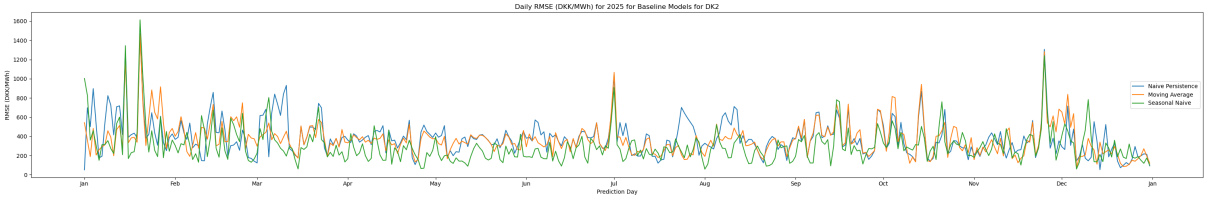


(c) Development of MAE (DKK/MWh) for each of the three baseline models for DK1 for the entire test period.

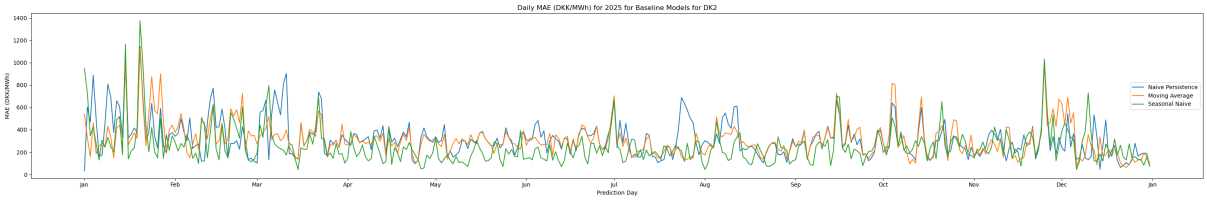
Figure 5.6: Development of SMAPE, RMSE, and MAE for each baseline model for the entire test period for DK1. The top figure shows this development of SMAPE (%), the middle model shows the development of RMSE (DKK/MWh), and the bottom figure shows the development of MAE (DKK/MWh).



(a) Development of SMAPE (%) for each of the three baseline models for DK2 for the entire test period.



(b) Development of RMSE (DKK/MWh) for each of the three baseline models for DK2 for the entire test period.



(c) Development of MAE (DKK/MWh) for each of the three baseline models for DK1 for the entire test period.

Figure 5.7: Development of SMAPE, RMSE, and MAE for each baseline model for the entire test period for DK2. The top figure shows this development of SMAPE (%), the middle model shows the development of RMSE (DKK/MWh), and the bottom figure shows the development of MAE (DKK/MWh).

5.2 Shallow Learners

The results of the final evaluation of each shallow learner are presented in the sections below. Each shallow learner was examined for both bidding zones.

5.2.1 Support Vector Regression

The final evaluation of the SVR models used the combination of hyperparameters found in section 4.2.1. The hyperparameters used for DK1 were:

- Kernel: poly
- C: 0.1
- Gamma: auto
- Epsilon: 0.1
- Degree: 2

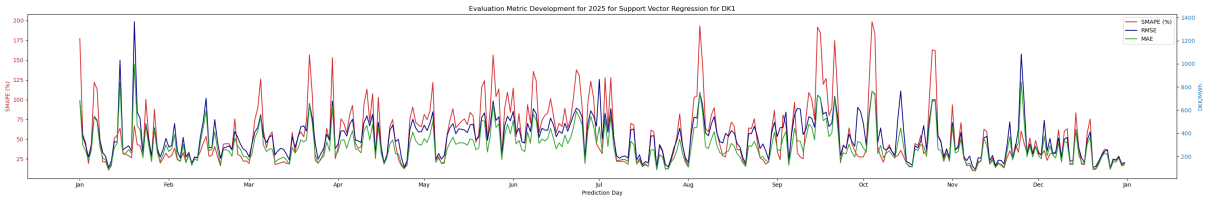
And the hyperparameters used for DK2 were:

- Kernel: rbf
- C: 1.0
- Gamma: auto
- Epsilon: 0.5
- Degree: default = 3

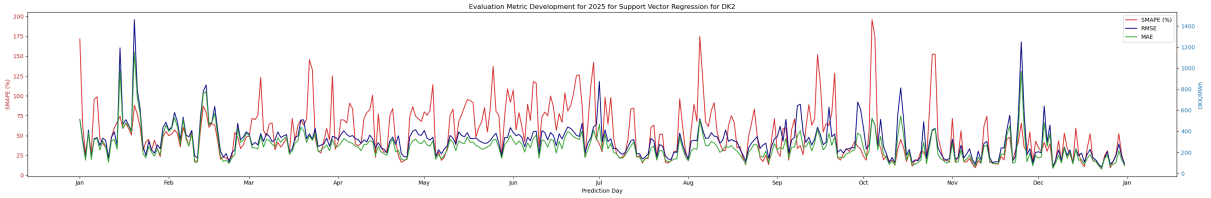
Using these hyperparameters, the results of the final evaluation can be seen in Table 5.4, where DK1 achieved an overall average SMAPE of 54.12 %, an average RMSE of 383.98 DKK/MWh, and an average MAE of 292.18 DKK/MWh. Meanwhile, DK2 yielded an overall average SMAPE of 52.24 %, an average RMSE of 330.48 DKK/MWh, and an average MAE of 255.29 DKK/MWh. The development of these metrics can be seen in Figure 5.8, where Figure 5.8a shows the development for DK1 and Figure 5.8b shows the development for DK2.

Table 5.4: Final evaluation results for the Support Vector Regression model for DK1 and DK2 for the evaluation metrics SMAPE, RMSE, and MAE using the hyperparameters found during the respective hyperparameter tuning.

Bidding Zone	Test SMAPE	Test RMSE	Test MAE
DK1	54.12 %	383.90 DKK/MWh	292.18 DKK/MWh
DK2	52.24 %	330.48 DKK/MWh	255.29 DKK/MWh



(a) Development of SMAPE, RMSE, and MAE for Support Vector Regression for DK1 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).



(b) Development of SMAPE, RMSE, and MAE for Support Vector Regression for DK2 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).

Figure 5.8: Development of SMAPE, RMSE, and MAE for Support Vector Regression for the entire test period. The top figure shows this development for DK1, while the bottom figure shows it for DK2. In each figure, the left y-axis shows the SMAPE (%) and the right y-axis shows the RMSE and MAE (DKK/MWh)

5.2.2 XGBoost

The combination of hyperparameters used for the final evaluation of DK1 and DK2 were described in section 4.2.2. For DK1, this combination of hyperparameters was:

- Max depth: 5
- Learning rate: 0.05
- Num estimators: 400
- Subsample: 1.0
- Colsample_bytree: 1.0

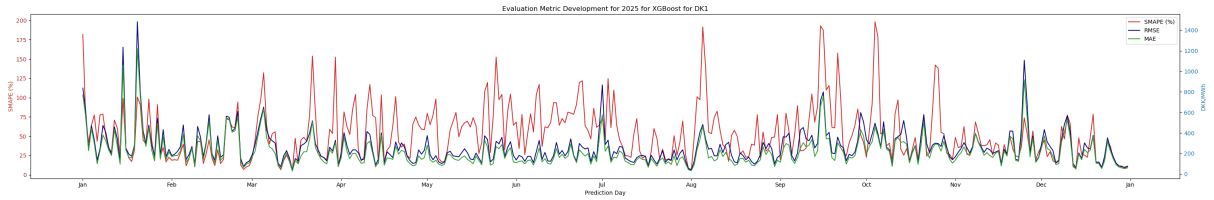
And the hyperparameters used for the final evaluation of DK2 were:

- Max depth: 7
- Learning rate: 0.05
- Num estimators: 400
- Subsample: 1.0
- Colsample_bytree: 0.7

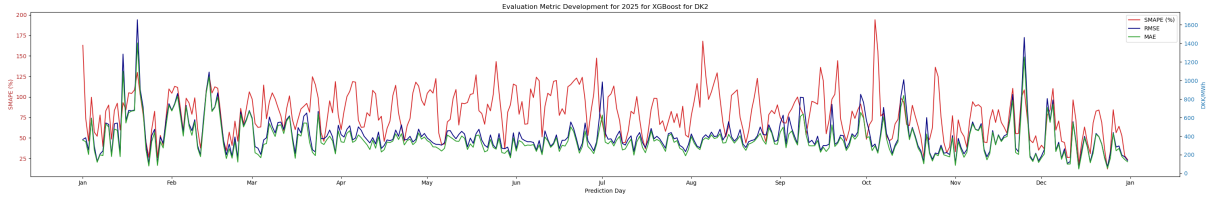
The results of the final evaluation can be seen in Table 5.5. DK1 yielded an overall average SMAPE of 52.25 %, RMSE of 286.44 DKK/MWh, and MAE of 217.77 DKK/MWh. The results for DK2 were somewhat similar, where it achieved an overall average SMAPE of 78.99 %, RMSE of 428.39 DKK/MWh, and MAE of 362.99 DKK/MWh. Figure 5.9 shows the development of these metrics during 2025, where Figure 5.9a shows the development of these metrics for DK1 and Figure 5.9b shows the development for D22.

Table 5.5: Final evaluation results for XGBoost for DK1 and DK2 for the evaluation metrics SMAPE, RMSE, and MAE using the hyperparameters found during the respective hyperparameter tuning.

Bidding Zone	Test SMAPE	Test RMSE	Test MAE
DK1	52.25 %	286.44 DKK/MWh	217.77 DKK/MWh
DK2	78.99 %	428.39 DKK/MWh	362.99 DKK/MWh



(a) Development of SMAPE, RMSE, and MAE for XGBoost for DK1 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).



(b) Development of SMAPE, RMSE, and MAE for XGBoost for DK2 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).

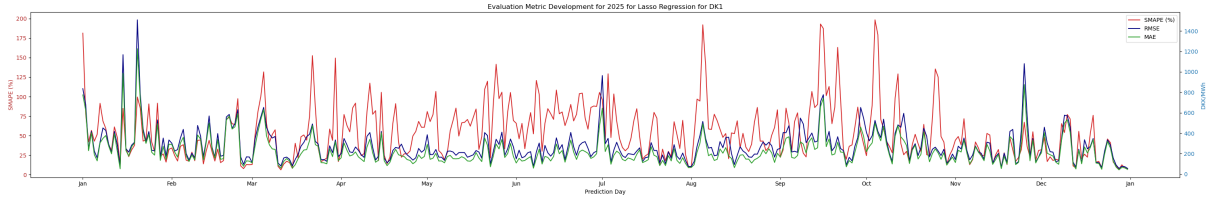
Figure 5.9: Development of SMAPE, RMSE, and MAE for XGBoost for the entire test period. The top figure shows this development for DK1, while the bottom figure shows it for DK2. In each figure, the left y-axis shows the SMAPE (%) and the right y-axis shows the RMSE and MAE (DKK/MWh)

5.2.3 Linear Regression

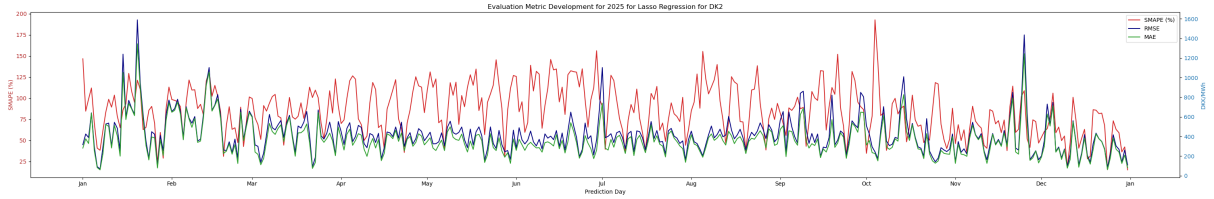
For both DK1 and DK2, Linear Regression with Lasso regularization was the best performing type of Linear Regression based on the hyperparameter tuning results described in section 4.2.3. DK1 performed best with an alpha value of 0.0001, and DK2 performed best with an alpha value of 50. The results of the final evaluation can be seen in Table 5.6. DK1 had an overall average SMAPE of 53.45 %, while DK2 reached 84.22 %. DK1 also ended with an average RMSE of 298.95 DKK/MWh and an average MAE of 226.65 DKK/MWh. DK2 achieved an average RMSE of 437.18 DKK/MWh and an average MAE of 365.04 DKK/MWh. The development of the metrics for both bidding zones can be seen in Figure 5.10. The development for DK1 is shown in Figure 5.10a and the development for DK2 is shown in Figure 5.10b.

Table 5.6: Final evaluation results for Lasso Linear Regression for DK1 and DK2 for the evaluation metrics SMAPE, RMSE, and MAE using the hyperparameters found during the respective hyperparameter tuning.

Bidding Zone	Test SMAPE	Test RMSE	Test MAE
DK1	53.45 %	298.95 DKK/MWh	226.65 DKK/MWh
DK2	84.22 %	437.18 DKK/MWh	365.04 DKK/MWh



(a) Development of SMAPE, RMSE, and MAE for Lasso Regression for DK1 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).



(b) Development of SMAPE, RMSE, and MAE for Lasso Regression for DK2 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).

Figure 5.10: Development of SMAPE, RMSE, and MAE for Lasso Regression for the entire test period. The top figure shows this development for DK1, while the bottom figure shows it for DK2. In each figure, the left y-axis shows the SMAPE (%) and the right y-axis shows the RMSE and MAE (DKK/MWh)

5.2.4 Random Forest

The final evaluation of Random Forest used the hyperparameter combinations determined in section 4.2.4 for DK1 and DK2, respectively. The optimal combination found for DK1 was:

- n_estimators: 200
- max_depth: None
- min_samples_split: 2
- min_samples_leaf: 1

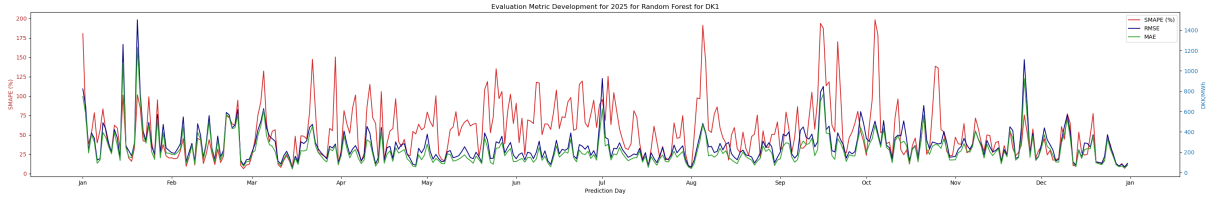
And the hyperparameters used for the final evaluation of DK2 were:

- n_estimators: 50
- max_depth: None
- min_samples_split: 2
- min_samples_leaf: 10

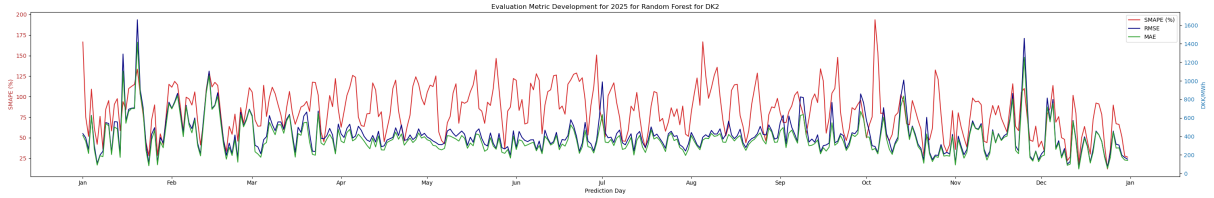
The final evaluation results can be seen in Table 5.7. These results show that DK1 achieved an overall average SMAPE of 53.15 %, RMSE of 293.34 DKK/MWh, and MAE of 221.93 DKK/MWh. For the DK2, the results were an overall average SMAPE of 82.17 %, RMSE of 440.09, and MAE of 373.61 DKK/MWh. The development of these metrics for the entire testing period can be seen in Figure 5.12. The development for DK1 and DK2 can be seen in Figure 5.12a and Figure 5.12b, respectively.

Table 5.7: Final evaluation results for Random Forest for DK1 and DK2 for the evaluation metrics SMAPE, RMSE, and MAE using the hyperparameters found during the respective hyperparameter tuning.

Bidding Zone	Test SMAPE	Test RMSE	Test MAE
DK1	53.15 %	293.34 DKK/MWh	221.93 DKK/MWh
DK2	82.17 %	440.09 DKK/MWh	373.61 DKK/MWh



(a) Development of SMAPE, RMSE, and MAE for Random Forest for DK1 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).



(b) Development of SMAPE, RMSE, and MAE for Random Forest for DK2 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).

Figure 5.11: Development of SMAPE, RMSE, and MAE for Random Forest for the entire test period. The top figure shows this development for DK1, while the bottom figure shows it for DK2. In each figure, the left y-axis shows the SMAPE (%) and the right y-axis shows the RMSE and MAE (DKK/MWh)

5.2.5 LightGBM

Section 4.2.5 described the combination of hyperparameters used for the final evaluation of DK1 and DK2 based on the results of the hyperparameter tuning for each bidding zone. The best performing combination of hyperparameters for DK1 was:

- n_estimators: 200
- learning_rate: 0.01
- num_leaves: 15
- min_child_samples: 1

- subsample: 0.8

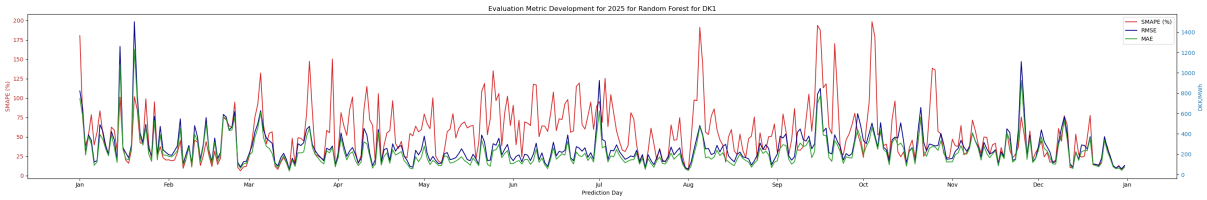
And for DK2, this combination was:

- n_estimators: 200
- learning_rate: 0.01
- num_leaves: 127
- min_child_samples: 50
- subsample: 0.8

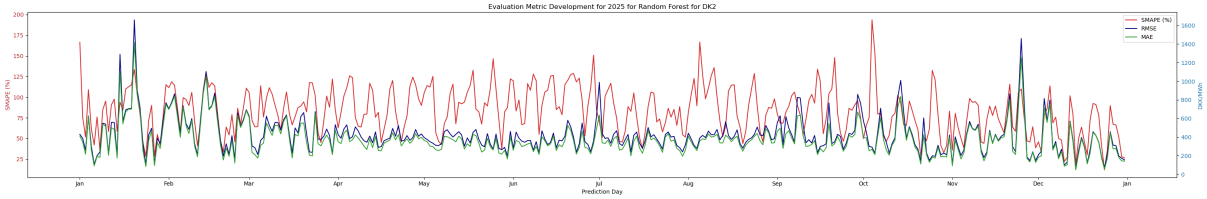
Using this combination of hyperparameters, meant that the final evaluation for DK1 yielded an overall average SMAPE of 50.99 %, RMSE of 314.48 DKK/MWh, and MAE of 237.93 DKK/MWh. The results of the final evaluation for DK2 showed an overall average SMAPE of 69.,43 %, RMSE of 397.04 DKK/MWh, and MAE of 332.16 DKK/MWh. These results are also shown in Table 5.8.

Table 5.8: Final evaluation results for LightGBM for DK1 and DK2 for the evaluation metrics SMAPE, RMSE, and MAE using the hyperparameters found during the respective hyperparameter tuning.

Bidding Zone	Test SMAPE	Test RMSE	Test MAE
DK1	50.99 %	314.48 DKK/MWh	237.93 DKK/MWh
DK2	69.43 %	397.04 DKK/MWh	332.16 DKK/MWh



(a) Development of SMAPE, RMSE, and MAE for Random Forest for DK1 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).



(b) Development of SMAPE, RMSE, and MAE for Random Forest for DK2 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).

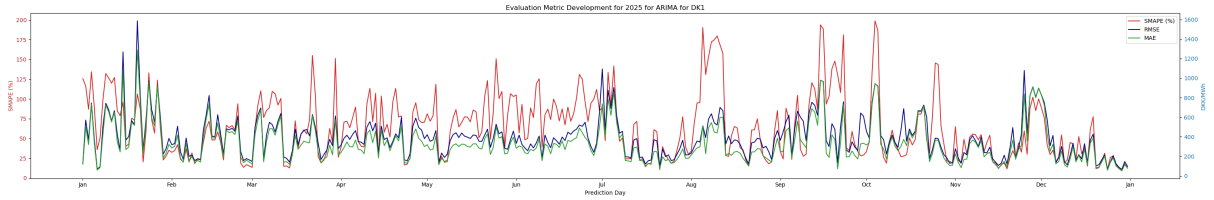
Figure 5.12: Development of SMAPE, RMSE, and MAE for Random Forest for the entire test period. The top figure shows this development for DK1, while the bottom figure shows it for DK2. In each figure, the left y-axis shows the SMAPE (%) and the right y-axis shows the RMSE and MAE (DKK/MWh)

5.2.6 ARIMA

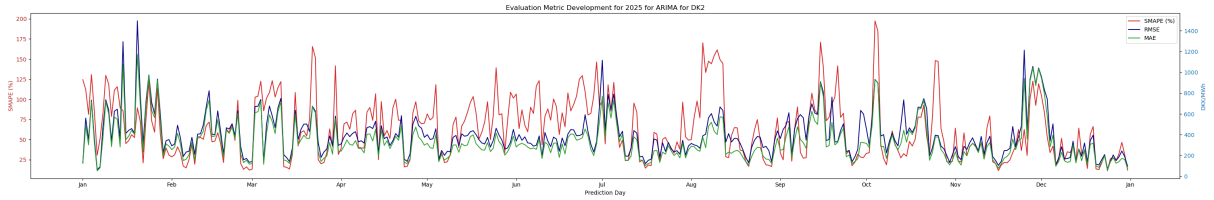
The final evaluation for DK1 and DK2 both used the order (2, 1, 1). The results of this final evaluation can be seen in Table 5.9. For DK1, this meant that the overall average SMAPE reached 63.29 % along with an average RMSE of 414.26 DKK/MWh and an average MAE of 330.33 DKK/MWh. DK2 achieved an overall average SMAPE of 62.56 %, RMSE of 418.11 DKK/MWh, and MAE of 334.48 DKK/MWh. The development of these metrics can be seen in Figure 5.13, where Figure 5.13a shows the development for DK1 and Figure 5.13b shows the development for DK2.

Table 5.9: Final evaluation results for ARIMA for DK1 and DK2 for the evaluation metrics SMAPE, RMSE, and MAE using the hyperparameters found during the respective hyperparameter tuning.

Bidding Zone	Test SMAPE	Test RMSE	Test MAE
DK1	63.29 %	414.26 DKK/MWh	330.33 DKK/MWh
DK2	62.56 %	418.11 DKK/MWh	334.48 DKK/MWh



(a) Development of SMAPE, RMSE, and MAE for ARIMA for DK1 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).



(b) Development of SMAPE, RMSE, and MAE for ARIMA for DK2 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).

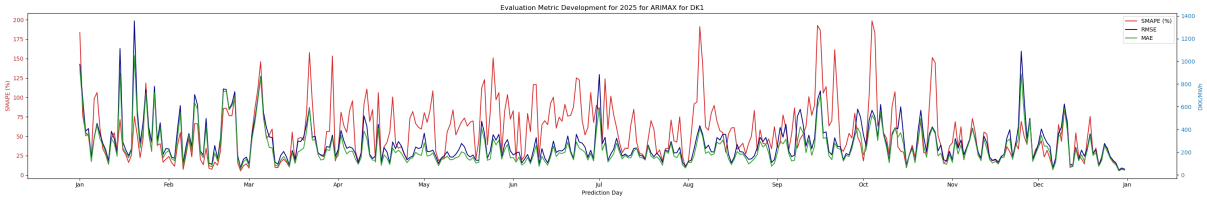
Figure 5.13: Development of SMAPE, RMSE, and MAE for ARIMA for the entire test period. The top figure shows this development for DK1, while the bottom figure shows it for DK2. In each figure, the left y-axis shows the SMAPE (%) and the right y-axis shows the RMSE and MAE (DKK/MWh)

5.2.7 ARIMAX

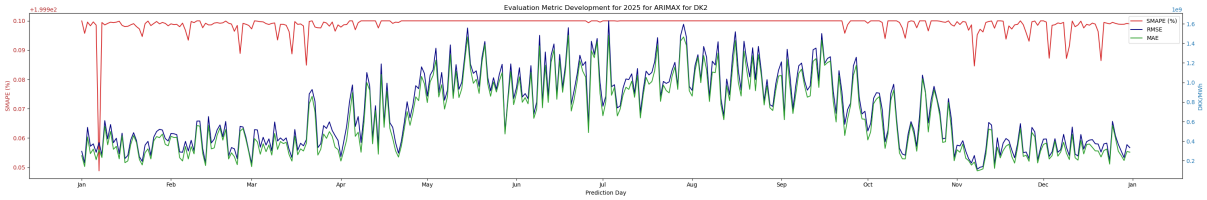
The final evaluation for DK1 and DK2 used the order (2, 1, 0) and (1, 1, 0), respectively. The results of this final evaluation can be seen in Table 5.10. For DK1, this meant that the overall average SMAPE reached 54.34 % along with an average RMSE of 305.72 DKK/MWh and an average MAE of 237.34 DKK/MWh. DK2 achieved an overall average SMAPE of 200.00 %, RMSE of 7.34e8 DKK/MWh, and MAE of 6.48e8 DKK/MWh. The development of these metrics can be seen in Figure 5.13, where Figure 5.13a shows the development for DK1 and Figure 5.13b shows the development for DK2.

Table 5.10: Final evaluation results for ARIMAX for DK1 and DK2 for the evaluation metrics SMAPE, RMSE, and MAE using the hyperparameters found during the respective hyperparameter tuning.

Bidding Zone	Test SMAPE	Test RMSE	Test MAE
DK1	54.34 %	305.72 DKK/MWh	237.34 DKK/MWh
DK2	200.00 %	7.34e8 DKK/MWh	6.48e8 DKK/MWh



(a) Development of SMAPE, RMSE, and MAE for ARIMAX for DK1 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).



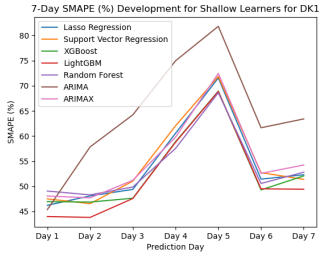
(b) Development of SMAPE, RMSE, and MAE for ARIMAX for DK2 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).

Figure 5.14: Development of SMAPE, RMSE, and MAE for ARIMAX for the entire test period. The top figure shows this development for DK1, while the bottom figure shows it for DK2. In each figure, the left y-axis shows the SMAPE (%) and the right y-axis shows the RMSE and MAE (DKK/MWh)

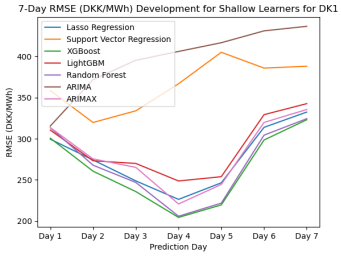
5.2.8 Consolidated Visualizations

Just as for the baseline models, the development of error across the 7-day prediction horizon has been visualized for each of the shallow learners. The average SMAPE for each day in the prediction horizon is calculated as part of the final evaluation, which is not the case for RMSE and MAE, which had to be calculated for these visualizations. The prediction values for each model were saved, which made it possible to calculate the daily RMSE and MAE, respectively. These daily errors were then averaged for each day of the prediction horizon using the same method

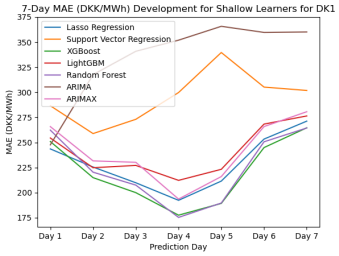
as for the baseline models described in section 5.1.4. This 7-day prediction horizon visualization can be seen for DK1 in Figure 5.15. Figure 5.15a shows the daily prediction SMAPE (%), Figure 5.15b shows it for RMSE (DKK/MWh), and Figure 5.17c shows it for MAE (DKK/MWh). The same visualizations can be seen for DK2 in Figure 5.16, where Figure 5.16a shows the SMAPE (%) averaged for each day in the 7-day prediction horizon, and Figure 5.16b and Figure 5.16c shows the averaged RMSE (DKK/MWh) and MAE (DKK/MWh), respectively, averaged across each day of the 7-day prediction horizon. ARIMAX has been left out of the RMSE and MAE figures for DK2 due to the size of its errors, which would skew the size of the figure and not make it possible to see the development of the other models clearly.



(a) Development of SMAPE (%) for each of the shallow learners for DK1 for the 7-day prediction horizon. These values are averages of each 7-day prediction horizon evaluated for the entire test set.

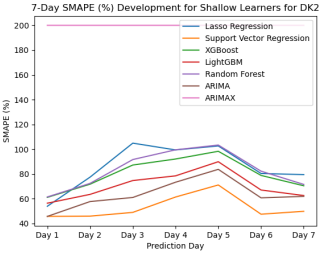


(b) Development of RMSE (DKK/MWh) for each of the shallow learners for DK1 for the 7-day prediction horizon. These values are averages of each 7-day prediction horizon evaluated for the entire test set.

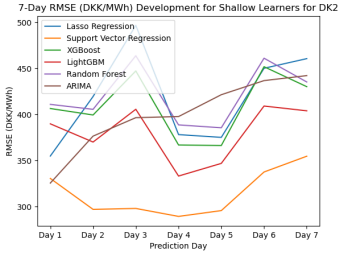


(c) Development of MAE (DKK/MWh) for each of the shallow learners for DK1 for the 7-day prediction horizon. These values are averages of each 7-day prediction horizon evaluated for the entire test set.

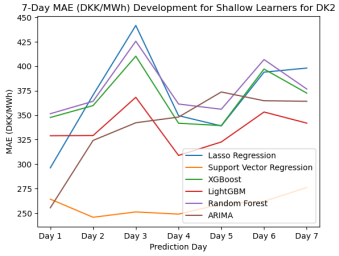
Figure 5.15: Development of the evaluation metrics for each of the shallow learners for DK1 for the 7-day prediction horizon. These values are averages of each 7-day prediction horizon evaluated for the entire test set. The top figure shows the development of SMAPE (%), the middle figure shows the development of RMSE (DKK/MWh), and the bottom figure shows the development of MAE (DKK/MWh).



(a) Development of SMAPE (%) for each of the shallow learners for DK2 for the 7-day prediction horizon. These values are averages of each 7-day prediction horizon evaluated for the entire test set.



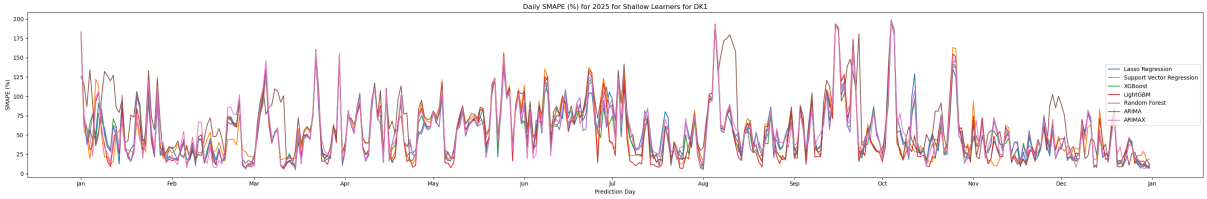
(b) Development of RMSE (DKK/MWh) for each of the shallow learners apart from ARIMAX for DK2 for the 7-day prediction horizon. These values are averages of each 7-day prediction horizon evaluated for the entire test set.



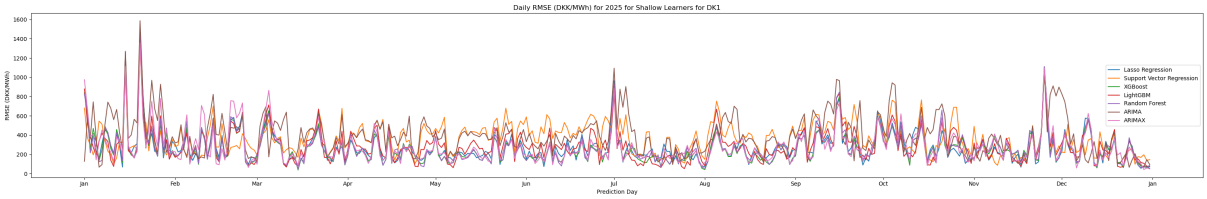
(c) Development of MAE (DKK/MWh) for each of the shallow learners apart from ARIMAX for DK2 for the 7-day prediction horizon. These values are averages of each 7-day prediction horizon evaluated for the entire test set.

Figure 5.16: Development of the evaluation metrics for the shallow learners for DK2 for the 7-day prediction horizon. These values are averages of each 7-day prediction horizon evaluated for the entire test set. The top figure shows the development of SMAPE (%), the middle figure shows the development of RMSE (DKK/MWh), and the bottom figure shows the development of MAE (DKK/MWh). ARIMAX has not been included in the RMSE and MAE figures.

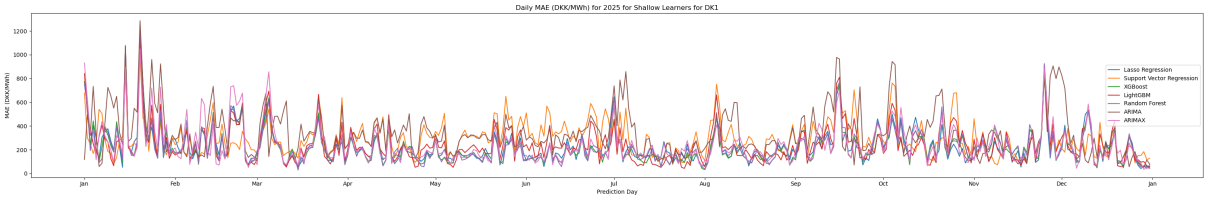
Consolidated visualizations across the entire test dataset have also been made for the same 5 shallow learners to compare their individual development. Figure 5.17 shows the consolidated visualizations for DK1, where Figure 5.17a shows the development of SMAPE (%), Figure 5.17b shows the development of RMSE (DKK/MWh), and Figure 5.17c shows the development of MAE (DKK/MWh). The same figures were also made for DK2, which can be seen in Figure 5.18. The development of SMAPE (%) for DK2 can be seen in Figure 5.18a, while the development of RMSE (DKK/MWh) and MAE (DKK/MWh) can be seen in Figure 5.18b and Figure 5.18c, respectively. Just as with the 7-day developments, ARIMAX has been left out of the RMSE and MAE figures due to the magnitude of its errors, which would skew the figures and make it impossible to determine the development of the other figures.



(a) Development of SMAPE (%) for 5 shallow learners for DK1 for the entire test period.

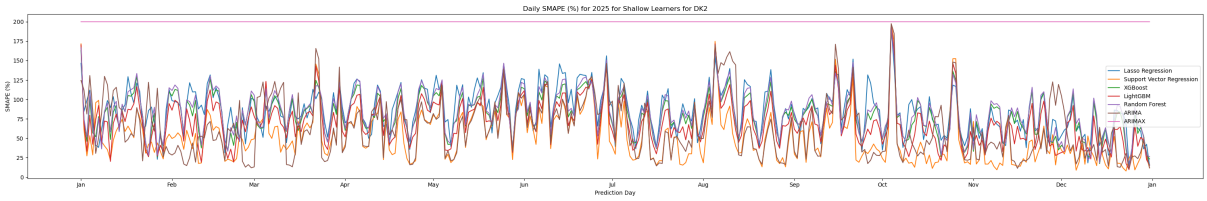


(b) Development of RMSE (DKK/MWh) for 5 shallow learners for DK1 for the entire test period.

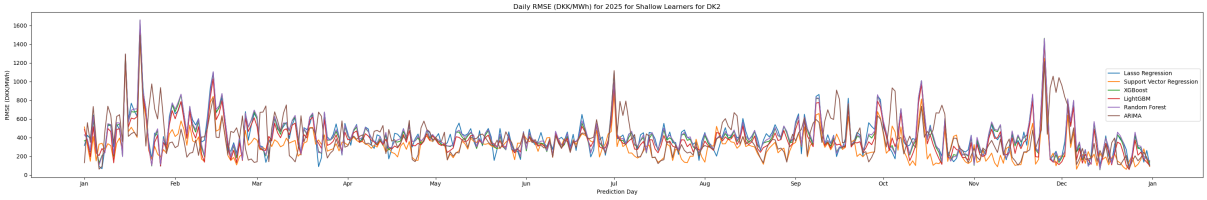


(c) Development of MAE (DKK/MWh) for 5 shallow learners for DK1 for the entire test period.

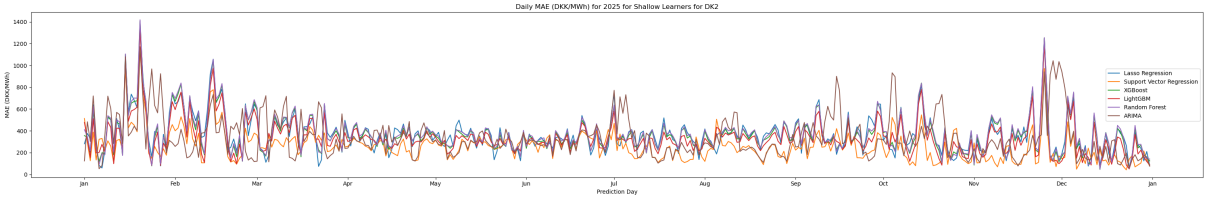
Figure 5.17: Development of SMAPE, RMSE, and MAE for 5 shallow learners for the entire test period for DK1. The top figure shows this development of SMAPE (%), the middle model shows the development of RMSE (DKK/MWh), and the bottom figure shows the development of MAE (DKK/MWh).



(a) Development of SMAPE (%) for each of the shallow learners for DK2 for the entire test period.



(b) Development of RMSE (DKK/MWh) for 6 shallow learners for DK2 for the entire test period.



(c) Development of MAE (DKK/MWh) for 6 shallow learners for DK2 for the entire test period.

Figure 5.18: Development of SMAPE, RMSE, and MAE for the shallow learners for the entire test period for DK2. The top figure shows this development of SMAPE (%), the middle model shows the development of RMSE (DKK/MWh), and the bottom figure shows the development of MAE (DKK/MWh). ARIMAX has been left out of the RMSE and MAE figures.

5.3 Deep Learners

As mentioned in section 3.6 we decided to only train deep learning models for the DK1 price zone. Hence, this section will only show results for DK1.

5.3.1 Simple Recurrent Neural Network

The final evaluation for Simple RNN used the hyperparameter combination determined in section 4.3.1. This combination used the following hyperparameters:

- Hidden size: [96]
- Layers: [2]
- Learning rate: [0.0003]
- Batch size: [64]
- Sequence length: [24]
- Dropout: [0.2]

The results of the final evaluation can be seen in Table 5.11, which shows an overall average SMAPE of 52.15 %, RMSE of 306.12 DKK/MWh and MAE of 248.29 DKK/MWh.

Table 5.11: Final evaluation results for Simple RNN for DK1 for the evaluation metrics SMAPE, RMSE, and MAE using the hyperparameters found during the respective hyperparameter tuning.

Train SMAPE	Val SMAPE	Test SMAPE	Test RMSE	Test MAE
31.64 %	50.17 %	52.15 %	306.12 DKK/MWh	248.29 DKK/MWh

The curves of the training and validation SMAPEs can be seen in Figure 5.19 and the development of the evaluation metrics across the entire testing period can be seen in Figure 5.20.

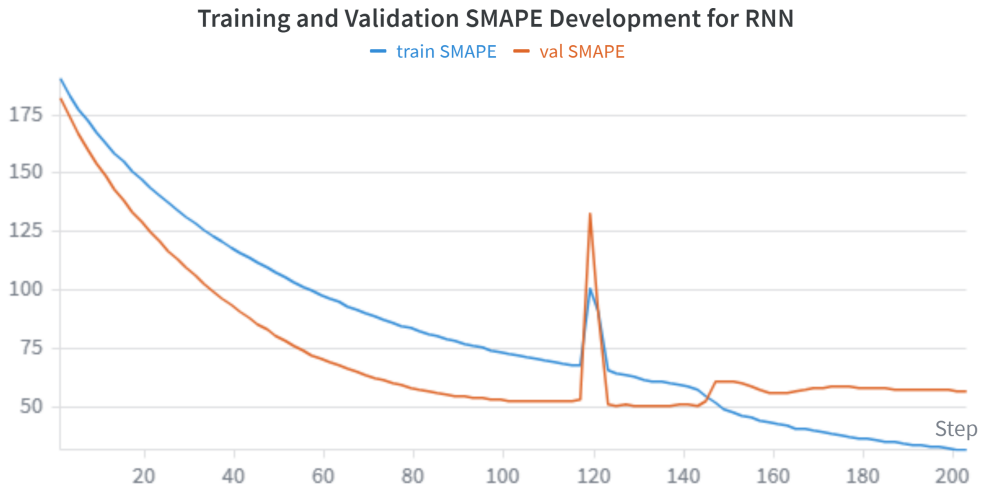


Figure 5.19: The curves of the training and validation SMAPEs for the final RNN model. The Blue curve is the training SMAPE and the orange curve is the validation SMAPE. The final model was saved at epoch 72.

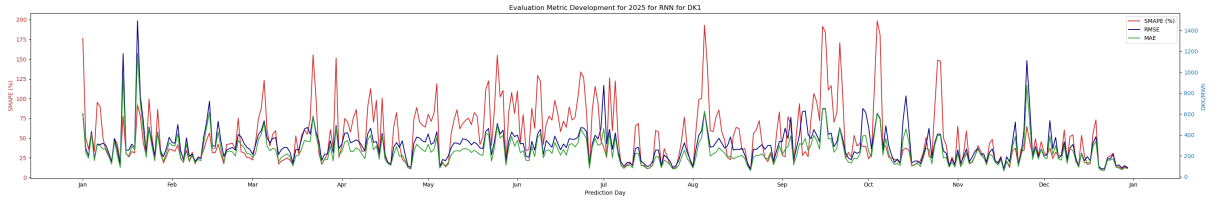


Figure 5.20: Development of SMAPE, RMSE, and MAE for RNN for DK1 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).

5.3.2 Gated Recurrent Units

The final evaluation of the GRU model was performed using the hyperparameters determined during the hyperparameter tuning described in section 4.3.2. These hyperparameters can also be seen below:

- Hidden size: [256]
- Layers: [2]
- Learning rate: [0.0008]

- Batch size: [64]
- Sequence length: [168]
- Dropout: [0.4]

The results of the final evaluation can be seen in Table 5.12. These results show that GRU achieved an overall average SMAPE of 56.24 %, RMSE of 340.01 DKK/MWh, and MAE of 279.65 DKK/MWh.

Table 5.12: Final evaluation results for GRU for DK1 for the evaluation metrics SMAPE, RMSE, and MAE using the hyperparameters found during the respective hyperparameter tuning.

Train SMAPE	Val SMAPE	Test SMAPE	Test RMSE	Test MAE
12.83 %	52.33 %	56.24 %	340.01 DKK/MWh	279.65 DKK/MWh

The curves of the training and validation SMAPEs can be seen in Figure 5.21, while the development of the evaluation metrics for the test dataset can be seen in Figure 5.22.

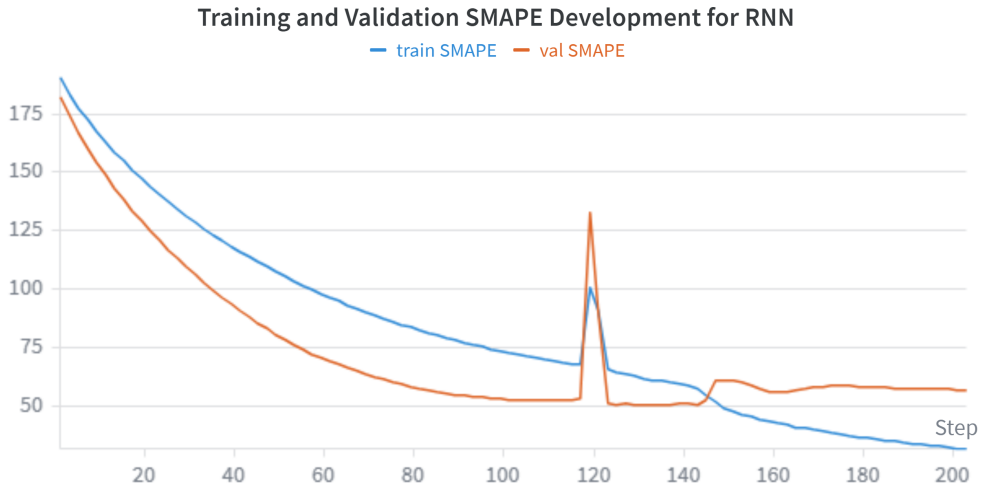


Figure 5.21: The curves of the training and validation SMAPEs for the final GRU model. The Blue curve is the training SMAPE and the orange curve is the validation SMAPE. The final model was saved at epoch 9.

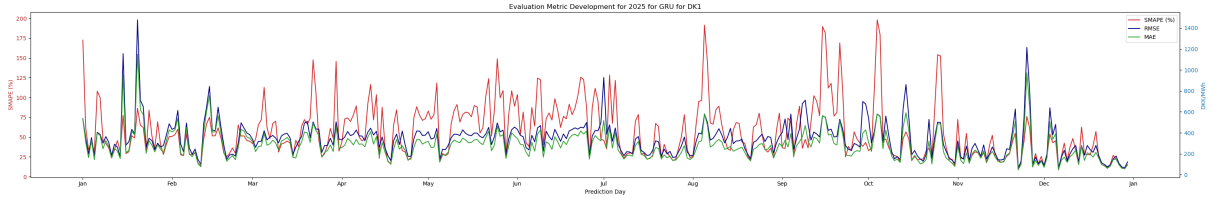


Figure 5.22: Development of SMAPE, RMSE, and MAE for GRU for DK1 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).

5.3.3 Long Short-Term Memory

The final evaluation of the LSTM model was performed using the hyperparameters determined during the hyperparameter tuning described in section 4.3.3. The best combination of hyperparameters was this one:

- Hidden size: [64]
- Layers: [3]
- Learning rate: [0.001]
- Batch size: [64]
- Sequence length: [24]
- Dropout: [0.2]

The results of the final evaluation can be seen in Table 5.13. These results show that LSTM achieved an overall average SMAPE of 57.09 %, RMSE of 379.43 DKK/MWh and MAE of 282.69 DKK/MWh.

Table 5.13: Final evaluation results for LSTM for DK1 for the evaluation metrics SMAPE, RMSE, and MAE using the hyperparameters found during the respective hyperparameter tuning.

Train SMAPE	Val SMAPE	Test SMAPE	Test RMSE	Test MAE
69.48 %	52.29 %	57.09 %	379.43 DKK/MWh	282.69 DKK/MWh

The curves of the training and validation SMAPEs can be seen in Figure 5.23.

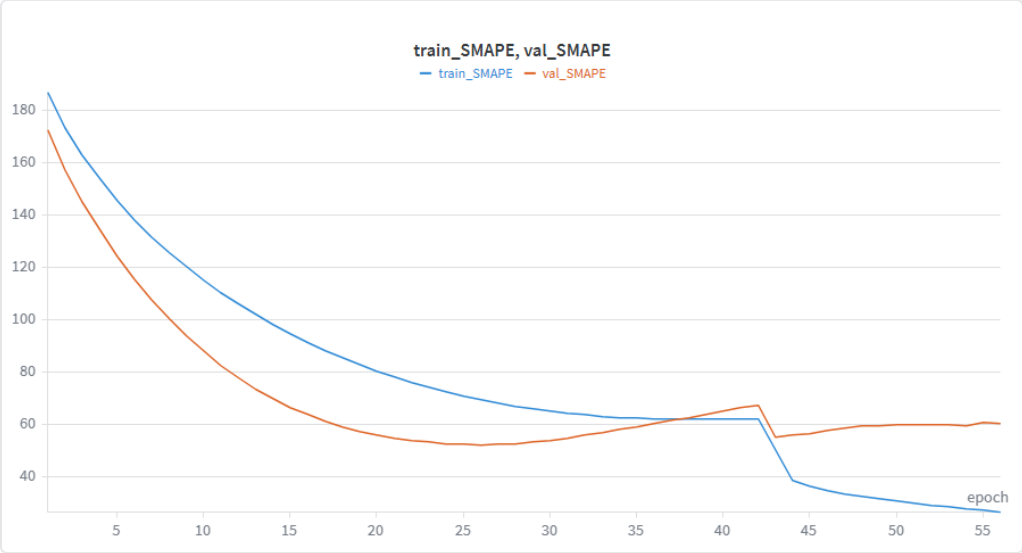


Figure 5.23: The curves of the training and validation SMAPEs for the final LSTM model. The blue curve is the training SMAPE and the orange curve is the validation SMAPE. The final model was saved at epoch 26.

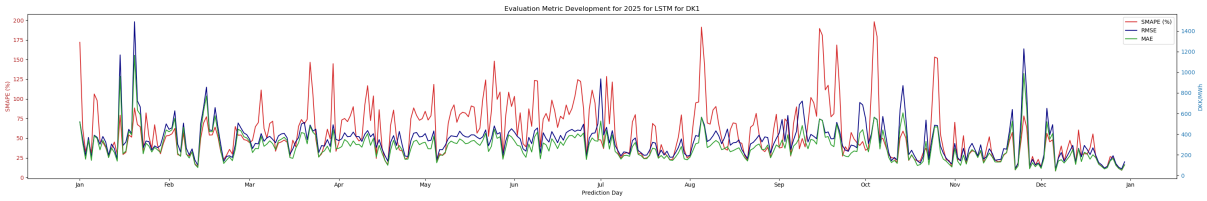


Figure 5.24: Development of SMAPE, RMSE, and MAE for LSTM for DK1 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).

5.3.4 LSTM Autoencoder

The final evaluation of the LSTM model was performed using the hyperparameters determined during the hyperparameter tuning described in section 4.3.4. The best combination of hyperparameters was this one:

Feature encoder parameters:

- Dense layers: 1
- Latent dimensions: 33

Encoder parameters:

- Encoder hidden size: 28
- Encoder layers: 2
- Encoder dropout: 0.0

Decoder parameters:

- Decoder hidden size: 48
- Decoder layers: 2
- Decoder dropout: 0.2

Shared parameters:

- Learning rate: 0.0005
- Batch size: 64
- Sequence length: 24

The results of the final evaluation can be seen in Table 5.14. These results show that LSTM Autoencoder achieved a test SMAPE of 75.43 %, RMSE of 413.70 DKK/MWh and MAE of 320.05 DKK/MWh.

Table 5.14: Final evaluation results for LSTM Autoencoder for DK1 for the evaluation metrics SMAPE, RMSE, and MAE using the hyperparameters found during the respective hyperparameter tuning.

Train SMAPE	Val SMAPE	Test SMAPE	Test RMSE	Test MAE
66.22 %	66.94 %	75.43 %	413.70 DKK/MWh	320.05 DKK/MWh

The curves of the training and validation SMAPEs can be seen in Figure 5.25. Figure 5.26 shows the development of the three evaluation metrics during the final evaluation of the test dataset.

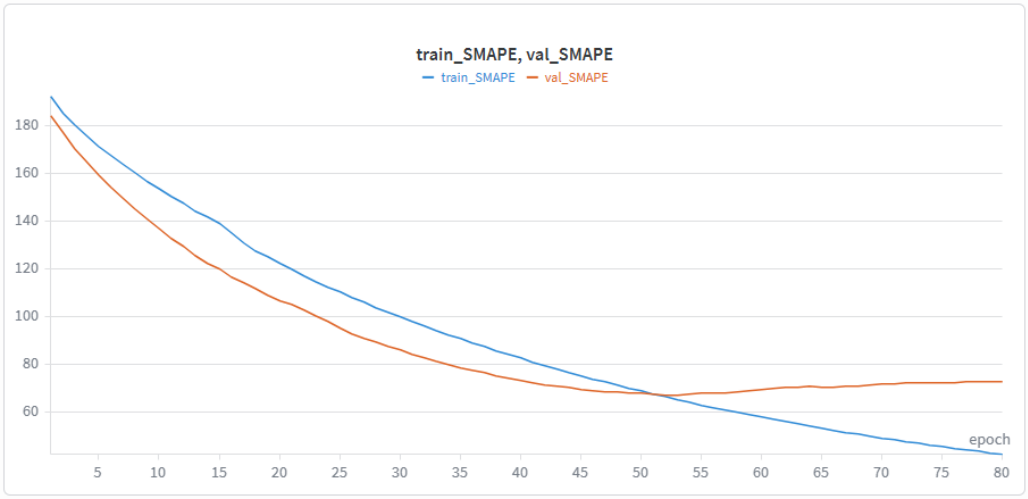


Figure 5.25: The curves of the training and validation SMAPEs for the final LSTM Autoencoder model. The blue curve is the training SMAPE and the orange curve is the validation SMAPE. The final model was saved at epoch 26.

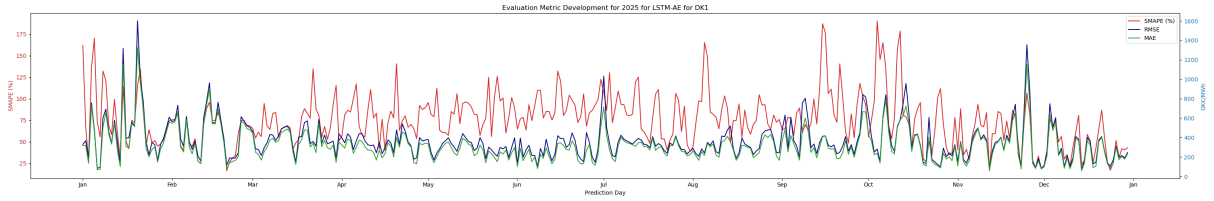
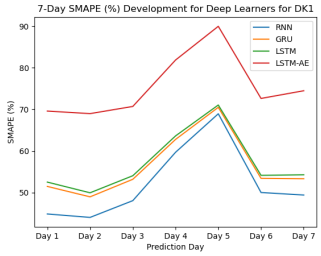


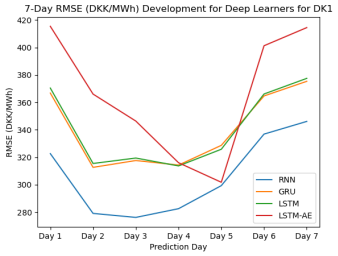
Figure 5.26: Development of SMAPE, RMSE, and MAE for LSTM AUtoencoder for DK1 for the entire test period. The left y-axis shows the SMAPE (%), while the right y-axis shows the RMSE and MAE (DKK/MWh).

5.3.5 Consolidated Visualizations

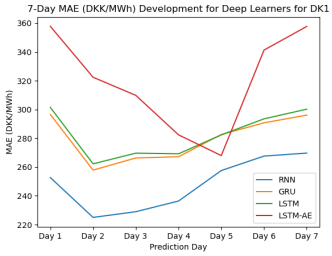
The consolidated visualizations were also made for the deep learners. The method for generating the evaluation metric data for the deep learners followed the same procedure as what was explained for the shallow learners in Section 5.2.8. The 7-day prediction horizon evaluation metric developments for each of the deep learners for DK1 can be seen in Figure 5.27. The development of SMAPE (%) is shown in Figure 5.27a, the development of RMSE (DKK/MWh) is shown in Figure 5.27b, and the development of MAE (DKK/MWh) is shown in Figure 5.27c.



(a) Development of SMAPE (%) for each of the deep learners for DK1 for the 7-day prediction horizon. These values are averages of each 7-day prediction horizon evaluated for the entire test set.



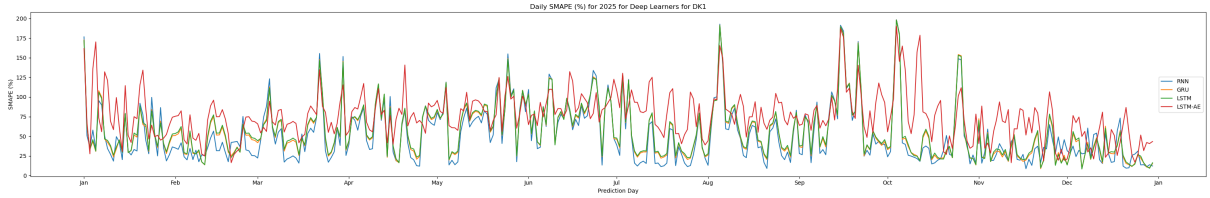
(b) Development of RMSE (DKK/MWh) for each of the deep learners for DK1 for the 7-day prediction horizon. These values are averages of each 7-day prediction horizon evaluated for the entire test set.



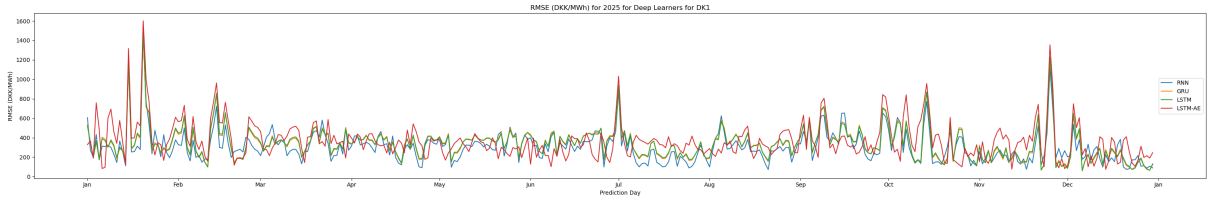
(c) Development of MAE (DKK/MWh) for each of the deep learners for DK1 for the 7-day prediction horizon. These values are averages of each 7-day prediction horizon evaluated for the entire test set.

Figure 5.27: Development of the evaluation metrics for each of the deep learners for DK1 for the 7-day prediction horizon. These values are averages of each 7-day prediction horizon evaluated for the entire test set. The top figure shows the development of SMAPE (%), the middle figure shows the development of RMSE (DKK/MWh), and the bottom figure shows the development of MAE (DKK/MWh).

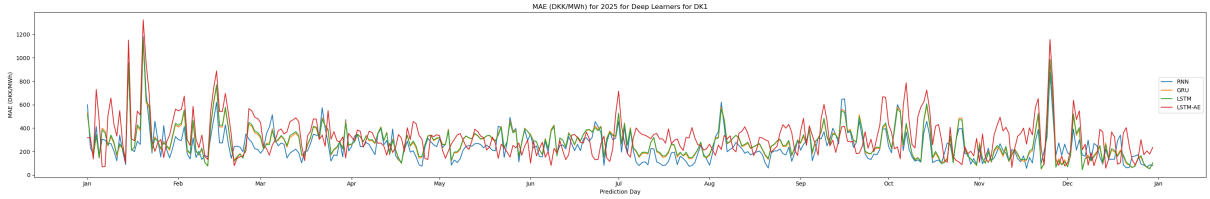
Just as for the baseline models and the shallow learners, consolidated visualizations of the full test period have also been made to further compare their performance. Figure 5.28 shows the development of the three evaluation metrics across the entire test dataset. Figure 5.28a shows the development of SMAPE (%), Figure 5.28b shows the development of RMSE (DKK/MWh), and Figure 5.28c shows the development of MAE (DKK/MWh).



(a) Development of SMAPE (%) for each of the deep learners for DK1 for the entire test period.



(b) Development of RMSE (DKK/MWh) for each of the deep learners for DK1 for the entire test period.



(c) Development of MAE (DKK/MWh) for each of the deep learners for DK1 for the entire test period.

Figure 5.28: Development of SMAPE, RMSE, and MAE for each of the deep learners for the entire test period for DK1. The top figure shows this development of SMAPE (%), the middle model shows the development of RMSE (DKK/MWh), and the bottom figure shows the development of MAE (DKK/MWh).

6 Discussion

Many components were tested or analyzed during the project. Over the course of the work, it was also necessary to make several assumptions and decisions. The consequences of these assumptions and decisions are also discussed in the following sections, along with the results, which were presented in section 5.

6.1 Results

This section will focus on the results of the final evaluation of the different models, along with some additional analyses of e.g., the importance of explanatory features and influence of weather forecast noise on model performance.

6.1.1 Best Performing Models

In this section, we will analyze the results of the model tests with the goal of answering the main objective of this thesis; "Which models are best suited for week-ahead forecasts in a context with high volatility?".

Because the deep learners were not examined for DK2, it is not possible to make a final conclusion as to which model is the most well-suited in this context. But it does still make sense to take a look at the results achieved for the baseline models and the shallow learners. These results can be seen in Table 6.1. ARIMAX has had a horrible performance, where it has reached the upper bound of SMAPE and the values of RMSE and MAE are significantly larger than the other models. Only one of the shallow learners has achieved a test SMAPE that is lower than the baseline models, which is SVR. However, its RMSE and MAE values are still higher than for Seasonal Naïve, which is the best performing baseline model. ARIMA has achieved a SMAPE that is lower than that of Naïve Persistence, but its RMSE and MAE are still higher. The remaining shallow learners have performed worse than the baseline models, which is surprising, as they are more advanced, so one would expect them to handle the patterns of the data better. Based on the final evaluation metrics alone, it is not possible to conclude, why the performance is poor. It could be due to something within the pipeline, it could be due to having too many features in the dataset, etc. But it is something that should be investigated further.

Table 6.1: All final models' performance results for DK2 sorted by test SMAPE.

Model	Test SMAPE	Test RMSE	Test MAE
Seasonal Naive	55.17 %	324.59 DKK/MWh	240.42 DKK/MWh
Moving Average	59.46 %	387.59 DKK/MWh	308.62 DKK/MWh
Naïve Persistence	66.36 %	405.85 DKK/MWh	316.25 DKK/MWh
SVR	52.24 %	330.48 DKK/MWh	255.29 DKK/MWh
ARIMA	62.56 %	418.11 DKK/MWh	334.48 DKK/MWh
LightGBM	69.43 %	397.04 DKK/MWh	332.16 DKK/MWh
XGBoost	78.99 %	428.39 DKK/MWh	362.99 DKK/MWh
Random Forest	82.17 %	440.09 DKK/MWh	373.61 DKK/MWh
Lasso Regression	84.22 %	437.18 DKK/MWh	365.04 DKK/MWh
ARIMAX	200.00 %	7.34e8 DKK/MWh	6.48e8 DKK/MWh

Table 6.2 shows the performance results of all the final models for DK1, divided into three categories: baseline models, shallow learners and deep learners, each group being sorted by test SMAPE.

Table 6.2: All final models' performance results for DK1 sorted by test SMAPE.

Model	Test SMAPE	Test RMSE	Test MAE
Seasonal Naive	56.59 %	341.39 DKK/MWh	243.73 DKK/MWh
Moving Average	60.31 %	397.96 DKK/MWh	303.34 DKK/MWh
Naïve Persistence	63.50 %	413.82 DKK/MWh	302.81 DKK/MWh
LightGBM	50.99 %	314.48 DKK/MWh	237.93 DKK/MWh
XGBoost	52.25 %	286.44 DKK/MWh	217.77 DKK/MWh
Random Forest	53.15 %	293.34 DKK/MWh	221.93 DKK/MWh
Lasso Regression	53.45 %	298.95 DKK/MWh	226.65 DKK/MWh
SVR	54.12 %	383.90 DKK/MWh	292.18 DKK/MWh
ARIMAX	54.34 %	305.72 DKK/MWh	237.34 DKK/MWh
ARIMA	63.29 %	414.26 DKK/MWh	330.33 DKK/MWh
RNN	52.15 %	306.12 DKK/MWh	248.29 DKK/MWh
GRU	56.24 %	340.01 DKK/MWh	279.65 DKK/MWh
LSTM	57.09 %	379.43 DKK/MWh	282.69 DKK/MWh
LSTM AE	75.43 %	413.70 DKK/MWh	320.05 DKK/MWh

First we take a closer look at the baseline models and their performances, before moving on to compare them to the shallow learners and deep learners.

Table 6.3: All final baseline models performance results sorted by test SMAPE.

Model	Test SMAPE	Test RMSE	Test MAE
Seasonal Naive	56.59 %	341.39 DKK/MWh	243.73 DKK/MWh
Moving Average	60.31 %	397.96 DKK/MWh	303.34 DKK/MWh
Naïve Persistence	63.50 %	413.82 DKK/MWh	302.81 DKK/MWh

When looking at the baseline models in table 6.3, we can see that the Naïve Persistence model has the worst performance of the baseline models based on both SMAPE and RMSE, while being almost tied with Moving Average in terms of MAE. The Naïve Persistence model predicts the next hours price to be exactly the same as the current actual price. As the model predicts 168 hours at a time it can only base the first prediction on the actual price value and for the rest of the prediction horizon it bases the predictions on its own previous predictions. This means it will predict a constant price during the 168-hour prediction horizon, which matches the last known price. When this model performs worse than the Moving Average model, it may be partly because the 168-hour prediction horizon always begins at midnight, which means the price at 11 PM the previous day determines the price for the next 168 hours and the price at that time of the day may not be very representative for the entire daily price cycle. Figure 6.1 and 6.2 show the daily price cycle for one week in January and one week in June 2025, respectively. Here we can see that there is a clear daily pattern in the summer week, while it is more difficult to identify in the winter week, which is more volatile with prices spanning a wider range of values. This means the Naïve Persistence model will perform worse on the winter data than on the summer data as one price value is less representative for price values for the next 168 hours. For the summer prices we see that each new day begins in the high valley of the curve, which means that the first high valley of the prediction horizon will determine the price for the next 168 hours, which may be too high to be representative for the rest of the days' prices.

Figure 6.1: One week of hourly electricity prices for price zone DK1 in January 2025.

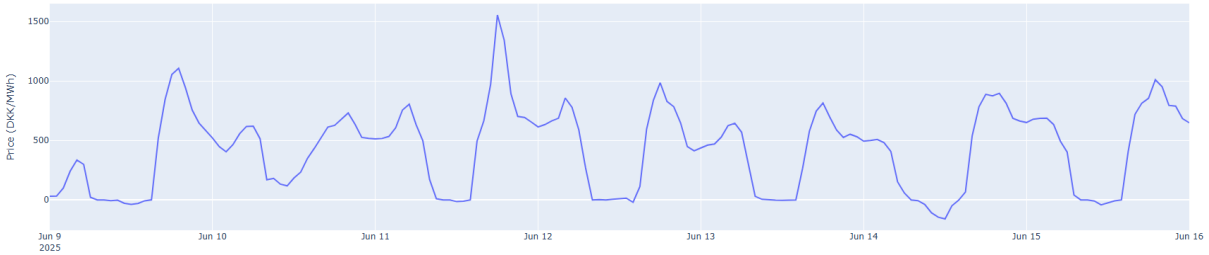


Figure 6.2: One week of hourly electricity prices for price zone DK1 in June 2025.

When we look at the Moving Average and Seasonal Naïve models performances in table 6.3, we see that the Seasonal Naïve model performs better on all three metrics. This is expected as the Seasonal Naïve model is designed to identify both a daily and a weekly seasonality in the data, while the Moving Average model only takes the overall trend of the previous 168 hours into account. This result may indicate that the daily and weekly seasonalities have a stronger effect on the price than the weekly trends do. We can examine the weekly seasonality visually by examining figure 6.3, which shows the hourly prices for price zone DK1 in December 2024 and January and February 2025 and figure 6.4, that shows the same in June, July and August 2025. The weekends (Saturdays and Sundays) are marked with a red line to help identify weekly patterns. For December 2024 to February 2025, figure 6.3 shows a more volatile price behavior with periods of one to two weeks of high prices between 1000 and 2000 DKK/MWh followed by shorter periods of lower prices between 0 and 1000 DKK/MWh and a few tall peaks with very high prices of up to 4000 and 7000 DKK/MWh. A weekly pattern is hard to identify as for a few of the weeks, the weekend seems to have slightly lower prices that the weekdays, while in most weeks, this is not the case. It does seem to have a fairly consistent daily seasonality though, with the lowest price of the day occurring around 5-6 AM, a peak around 7-8 AM, a drop around midday, the tallest peak of the day around 5-7 PM and then a drop again during the rest of the night. For June to August 2025, figure 6.4 shows a more stable price behavior with prices moving up and down within the same range between 0 and 1000 DKK/MWh most days in most weeks and only a few peaks between 1500 and 2000 DKK/MWh and a single peak at 3500 DKK/MWh. A weekly pattern is also hard to identify in this period, but some weeks seem to have lower price drops in the weekends than during the weekdays. Like for the winter months the daily seasonality is still very consistent, but with a slightly different pattern in the summer months. The lowest price drop occurs at midday, rising to the tallest peak around 5-7 PM, a small drop during the night and then a small peak around 7-8 AM.

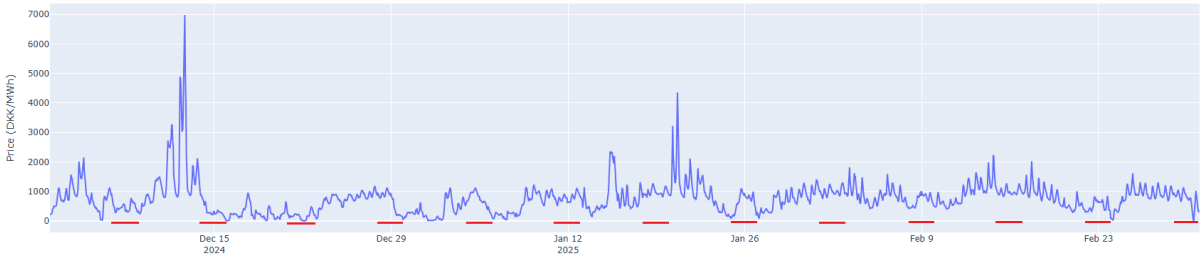


Figure 6.3: Hourly electricity prices for price zone DK1 in December 2024, January and February 2025.

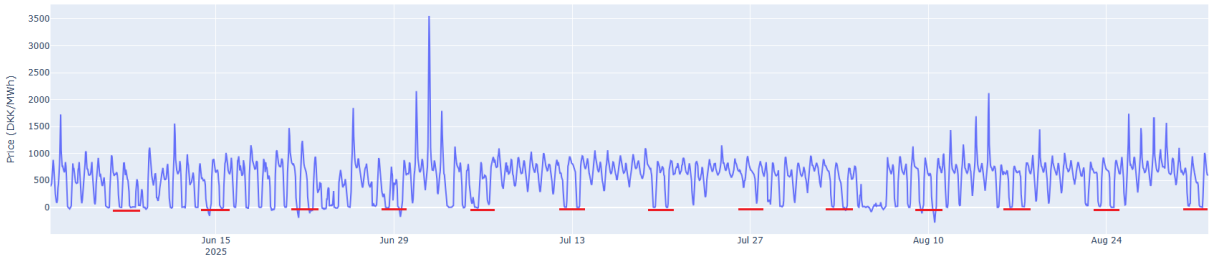


Figure 6.4: Hourly electricity prices for price zone DK1 in June, July and August 2025.

When studying these figures, it seems reasonable that during the summer months the Seasonal Naïve model might do well and possibly better than the Moving Average model, but during the winter months, there is no stable weekly price level, which means the daily pattern is less useful for the model to identify as the daily range is smaller than the range of periods of higher and lower prices and the Moving Average model might do better here. To examine this we can look at the predictions of the two models compared to the actual prices for January 2025 and July 2025, which are shown in figure 6.5 and figure 6.6, respectively. In July 2025, the Seasonal Naïve model seems to be very good at following the daily seasonality and as we do not see week specific patterns, following the daily seasonality is sufficient for the model to perform well. However, figure 6.6 also shows that the tall price peak on July 1st affects the predictions on both July 8th, July 15th and July 22nd. This is because the Seasonal Naïve looks 3 weeks back to make predictions and that means that every time there is an unusually high peak or low drop, it will affect 3 predictions in the future. This does not happen often during the summer months, but it happens more often during the winter where we also see the Seasonal Naïve struggle a lot more to follow the actual price trends. As the price in the winter months is more volatile than it is in the summer months, week old information is not very useful for the predictions. Even though the Seasonal Naïve model is having some challenges winter data, the Moving Average models also seem to struggle a lot during the winter months. As price levels change a lot from week to week, the Moving Average model can also be greatly mislead by week old information. These figures provide a good understanding of why the Seasonal Naïve model performs better than the two others.

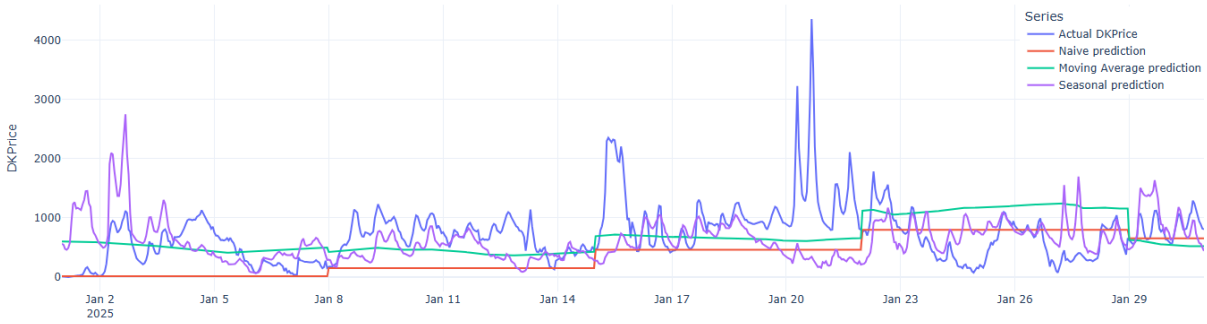


Figure 6.5: Hourly electricity prices for price zone DK1 in January 2025 and the predicted prices of Naïve Persistence, Moving Average, and Seasonal Naïve. The blue line is the actual price, the red line is the Naïve Persistence prediction, the green line is the Moving Average prediction, and the purple line is the Seasonal Naïve prediction.

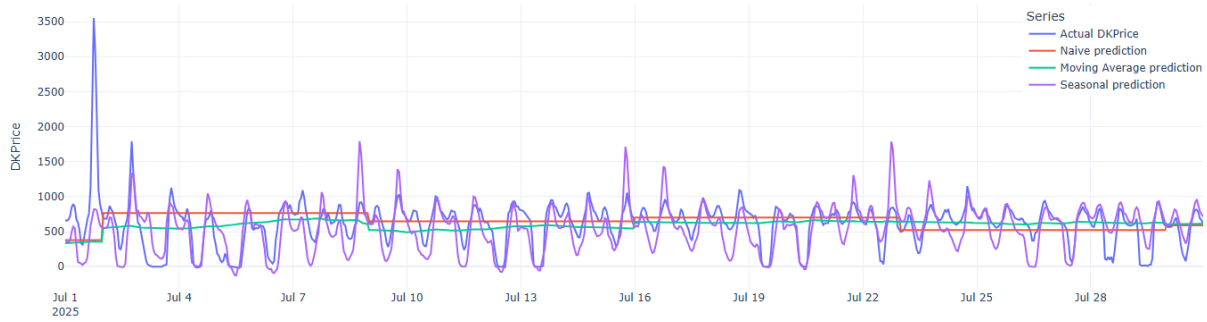


Figure 6.6: Hourly electricity prices for price zone DK1 in July 2025 and the predicted prices of Naïve Persistence, Moving Average, and Seasonal Naïve. The blue line is the actual price, the red line is the Naïve Persistence prediction, the green line is the Moving Average prediction, and the purple line is the Seasonal Naïve prediction.

Table 6.4 shows the performance of all the shallow learners on data from price zone DK1 for the year 2025. There is not a large variation in the performance, but ARIMA has the worst performance based on all three metrics with a SMAPE of 63.29 %. The rest of the models have test SMAPEs between 50.99 % for the LightGBM and 54.34 % for the ARIMAX. The LightGBM is the best performing model measured on all three metrics. The shallow learners, not including ARIMA, have a test SMAPE between 50.99 % and 54.34 %, an RMSE between 286.44 and 314.48 DKK/MWh, and an MAE between 217.77 and 292.18 DKK/MWh. These results show that ARIMA performs about the same as the Naïve Persistence baseline model, which was the worst performer out of the three baseline models.

Table 6.4: All final shallow learner models performance results for DK1 sorted by test SMAPE.

Model	Test SMAPE	Test RMSE	Test MAE
LightGBM	50.99 %	314.48 DKK/MWh	237.93 DKK/MWh
XGBoost	52.25 %	286.44 DKK/MWh	217.77 DKK/MWh
Random Forest	53.15 %	293.34 DKK/MWh	221.93 DKK/MWh
Lasso Regression	53.45 %	298.95 DKK/MWh	226.65 DKK/MWh
SVR	54.12 %	383.90 DKK/MWh	292.18 DKK/MWh
ARIMAX	54.34 %	305.72 DKK/MWh	237.34 DKK/MWh
ARIMA	63.29 %	414.26 DKK/MWh	330.33 DKK/MWh

Figure 6.7 and figure 6.8 show the actual prices and the predictions of the three tree based models LightGBM, Random Forest, and XGBoost, for each hour in January 2025 and July 2025, respectively. If we examine figure 6.8 we can see that during July, LightGBM is the model that follows the actual prices most closely. It generally seems to have figured out how to predict the price fairly accurately most of the time. Random Forest is also good but has larger and more frequent errors than LightGBM. XGBoost seems to be struggling significantly more than the other two models. If we look at figure 6.7, to get an idea of how the models perform during winter months, we see that all three models have difficulties. None of them seem to be able to predict, when prices will go up and when they will go down. LightGBM and Random Forest clearly have some expectations of when price levels change, but they do not follow the actual price changes. They do seem to have learned the right daily patterns, though. XGBoost is at a more consistent low price level with a daily seasonality, and it doesn't really seem to have any idea of when the price levels change and when it does change its price level, its only very small changes.

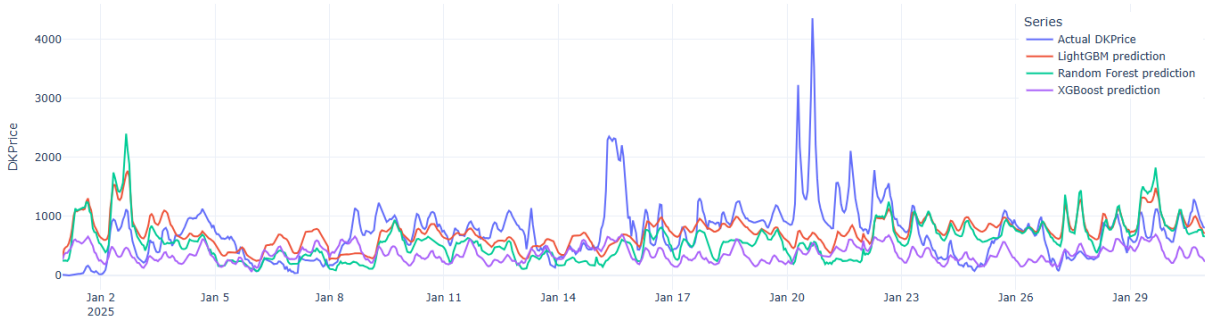


Figure 6.7: Hourly electricity prices for price zone DK1 in January 2025 and the predicted prices of LightGBM, Random Forest and XGBoost. The blue line is the actual price, the red line is the LightGBM prediction, the green line is the Random Forest prediction and the purple line is the XGBoost prediction.

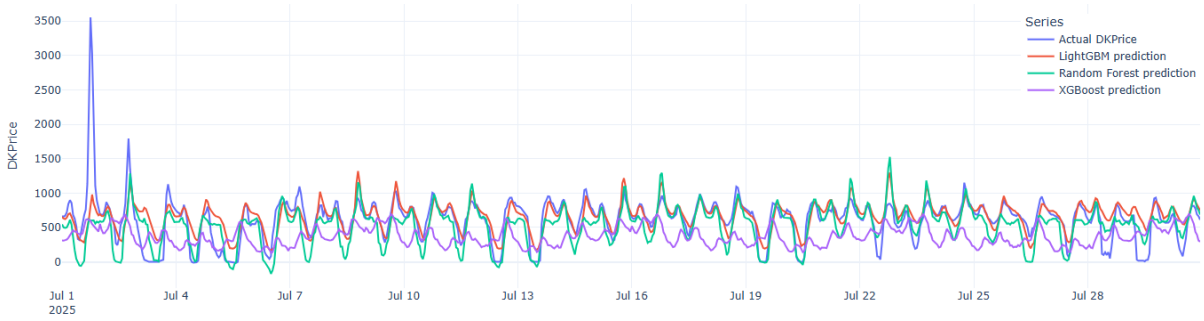


Figure 6.8: Hourly electricity prices for price zone DK1 in July 2025 and the predicted prices of LightGBM, Random Forest and XGBoost. The blue line is the actual price, the red line is the LightGBM prediction, the green line is the Random Forest prediction and the purple line is the XGBoost prediction.

To get a better understanding of how the models predict the prices, we can examine which of the explanatory features they base their predictions on. This can be done with a Shapley Additive Explanations (SHAP) analysis. The SHAP analysis starts by looking at numerous predictions for a given model to make an estimate of the expected value of the models predictions. Then it creates many random subsets of the explanatory features and a large random sample of data points. For each sampled data point it lets the model use the different subsets of features one at a time and compares the predicted values, to estimate how each feature influences the prediction. Each feature is given a SHAP value, where a positive SHAP value means the feature has a positive influence on the prediction and a negative SHAP value means the feature has a negative influence on the prediction. When all the sampled data points have been analyzed, the final SHAP values for each feature are the average of the absolute SHAP values across all the data points. It is important to note that because the resulting SHAP values are averages they do not tell if there is a large variation in the ranking across data points and if the ranking is different for different seasons of the year or under different circumstances. Figure 6.9 shows bar charts of the SHAP analysis for the three tree models, revealing which of the explanatory features have the largest influence on the predictions of the models. There is a remarkable resemblance between the three models that almost have the same ranking of features. They all agree that Price_lag1 is the feature with the largest influence followed by DEPrice. These two are by far the highest ranked features followed by SE4Price, NO2Price, NLPrice, and OnshoreWindPower in different orders for the three models.

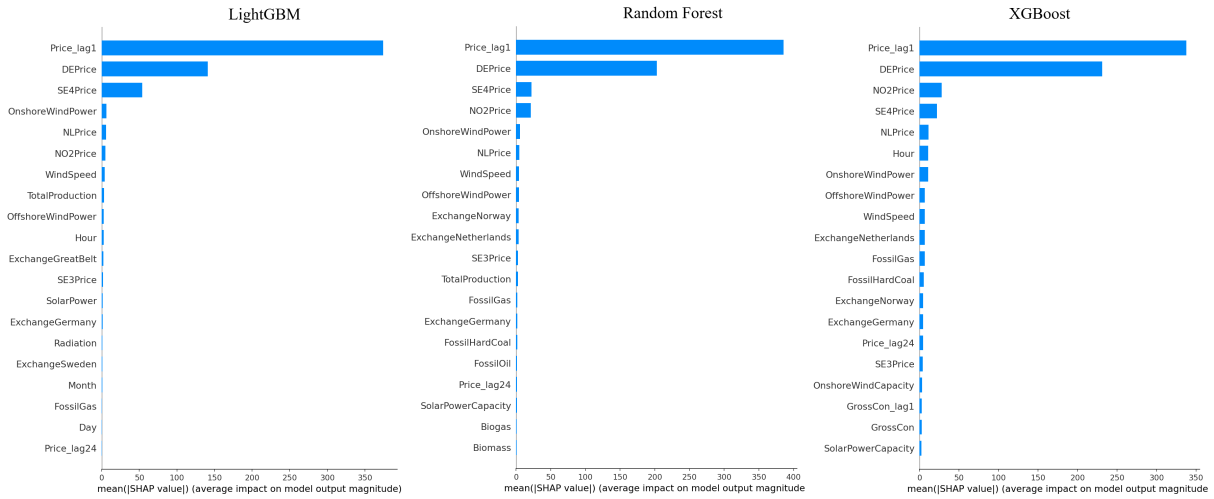


Figure 6.9: SHAP analysis for tree models in price zone DK1.

Figure 6.10 and figure 6.11 show the actual prices and the predictions of the Linear Regression and SVR models for January 2025 and July 2025, respectively. It shows that the SVR models' predictions lie on a very static price level around 700 to 800 DKK/MWh in both January and July. It completely fails to follow daily seasonality and price trends. The Linear Regression model seems to have identified a daily pattern, that it follows most of the time, while also catching some of the price trends and shifts in price levels, even though it greatly underestimates the scale of the changes. This is the case for both January and July.

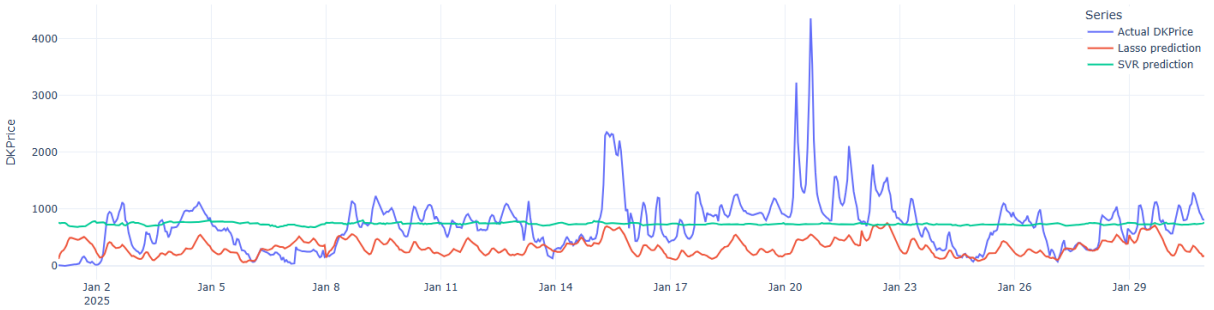


Figure 6.10: Hourly electricity prices for price zone DK1 in January 2025 and the predicted prices of Linear Regression and SVR. The blue line is the actual price, the red line is the Linear Regression prediction and the green line is the SVR prediction.

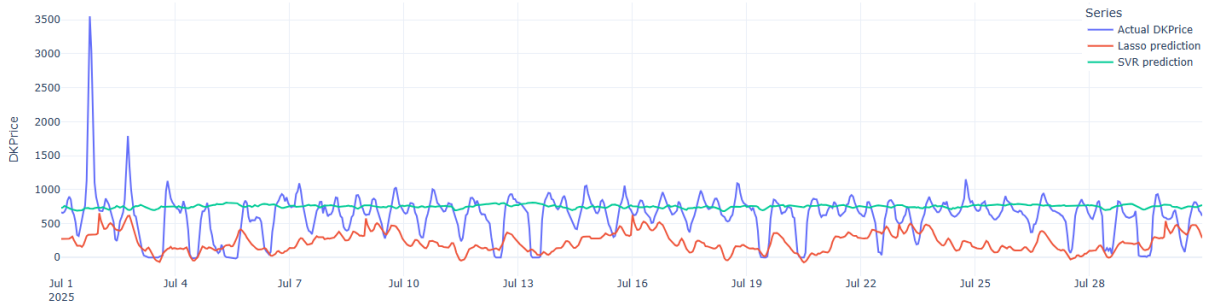


Figure 6.11: Hourly electricity prices for price zone DK1 in July 2025 and the predicted prices of Linear Regression and SVR. The blue line is the actual price, the red line is the Linear Regression prediction and the green line is the SVR prediction.

Figure 6.12 and figure 6.13 show the actual prices and the predictions the ARIMA and ARIMAX models for January and July 2025. The ARIMA model is an autoregressive model that uses its own previous predictions to make future predictions when forecasting a 168-hour horizon. Every time it starts predicting a new block of 168 hours its state is reset with actual past values, which simulates a real use-case situation. That is why we see the ARIMA models' predictions start at actual price level and then quickly finding a stable level that it follows for the rest of the 168-hour block and then restart at the actual price level, when starting a new block. This is the case through the entire year, which results in smaller errors during summer with more stable prices and smaller changes in price levels and results in larger errors during winter with more volatile prices and larger changes in price levels. The ARIMAX model is also an autoregressive model, but it also uses the explanatory features to make predictions, which is why we see it behaving very differently from the ARIMA model. The ARIMAX seems to have learned the daily seasonality in both January and July and follows the price level fairly accurately most of the time in July. In January, it is significantly more difficult for the ARIMAX to follow the changes in price levels, as many of the large changes are completely ignored through the entire 168-hour prediction horizon including the tall spikes.

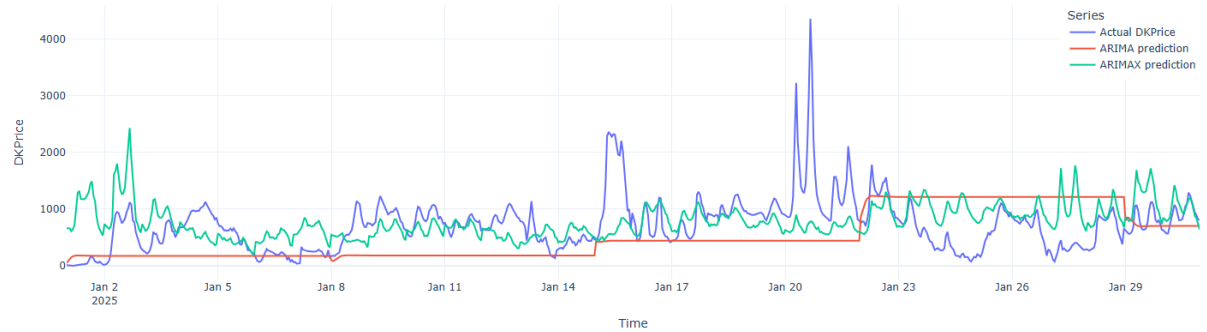


Figure 6.12: Hourly electricity prices for price zone DK1 in January 2025 and the predicted prices of ARIMA and ARIMAX. The blue line is the actual price, the red line is the ARIMA prediction, and the green line is the ARIMAX prediction.

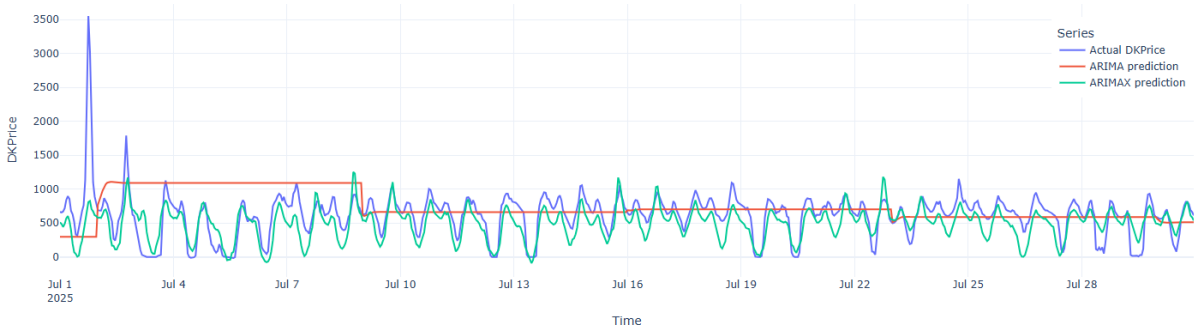


Figure 6.13: Hourly electricity prices for price zone DK1 in July 2025 and the predicted prices of ARIMA and ARIMAX. The blue line is the actual price, the red line is the ARIMA prediction, and the green line is the ARIMAX prediction.

Figure 6.14 shows the ranking of features for the Linear Regression, SVR and ARIMAX models, based on their SHAP values. The Linear Regression model is the one that agrees the most with the tree based models in terms of feature ranking. It values Price_lag1 far higher than the other features and then comes DEPrice. NLPrice, NO2Price and SE4Price are also high on the list within the top 7, but GrossCon_lag1 and GrossCon are also in the top 7, while for the tree based models only the XGBoost had GrossCon inside the top 20 and GrossCon_lag1 was placed outside by all three models. The SVR model has very small SHAP values for all features with a maximum value at around 8, compared with a maximum SHAP value at around 350 to 450 for all the other shallow learners. This means that none of the features can cause large changes to the expected value of the SVR predictions, which is why we see the SVR predictions lying on such a static price level in figures 6.10 and 6.11. The ARIMAX model is an autoregressive model, meaning that it uses most recent past values of the DKPrice when predicting future values, so the model does not need the Price_lag1 feature as input. That is why we do not see Price_lag1 on the top 20 SHAP values of the ARIMAX features. The ARIMAX places the DEPrice at the top of the list with the far greatest SHAP value of just above 400 and N02Price having the second-highest value at around 75. This is very much in line with the tree based models, if we ignore the Price_lag1 feature. The rest of the features have much smaller values, with SE4Price, NLPrice and OnshoreWindPower within the top 7, which is also in line with the tree based models.

As ARIMA is a univariate model, it does not use explanatory features, thus this model was not included in the SHAP analysis.

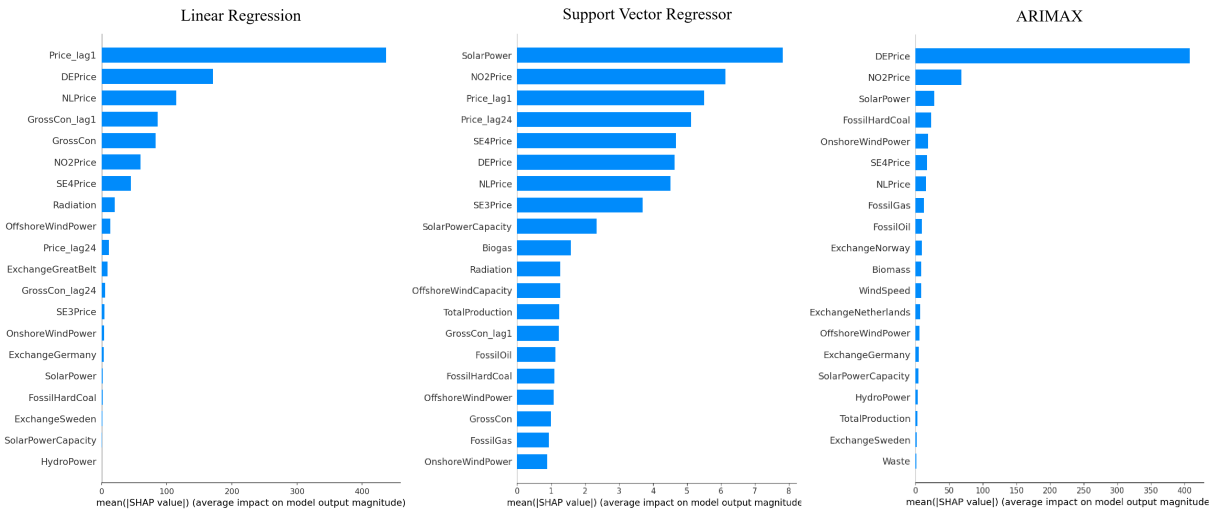


Figure 6.14: SHAP analysis for Linear Regression, SVR and ARIMAX in price zone DK1.

The fact that the Price_lag1 has such a dominating influence on the predictions, shows that the models rely more on past values of the target variable than future values of the explanatory features, when predicting future values. This indicates that either the explanatory features do not provide the necessary information to the models to make accurate predictions or that the models were not able to learn how to use this information. Instead, they need to rely on the assumption that the price level for the next 168 hours will be similar to the price level of the last known price. This is not a reliable assumption to make in a volatile domain and can be part of reason why the models perform significantly worse in January, where price levels are more unstable than in July. As mentioned before though, we don't know if the SHAP value rankings of the features are the same across all seasons of the year. The ones shown in the SHAP plots are the average SHAP values across a sample of data point taken across the entire year of 2025.

In figure 6.15 the daily SMAPEs of the shallow learners Linear Regression, SVR, XGBoost, LightGBM, and Random Forest for the year 2025 are compared with the average daily prices of 2025. It shows that all the spikes in SMAPE happens when the price level deviates a lot from the red line, which indicates that the models expect the price level to be at around 700 to 800 DKK/MWh and if it deviates a lot from that they fail to predict that deviation. That fits with what we saw in the SHAP plots, where all other features than Price_lag1 and DEPrice have small SHAP values, meaning that no features are able to cause large changes to the expected value of the predictions.

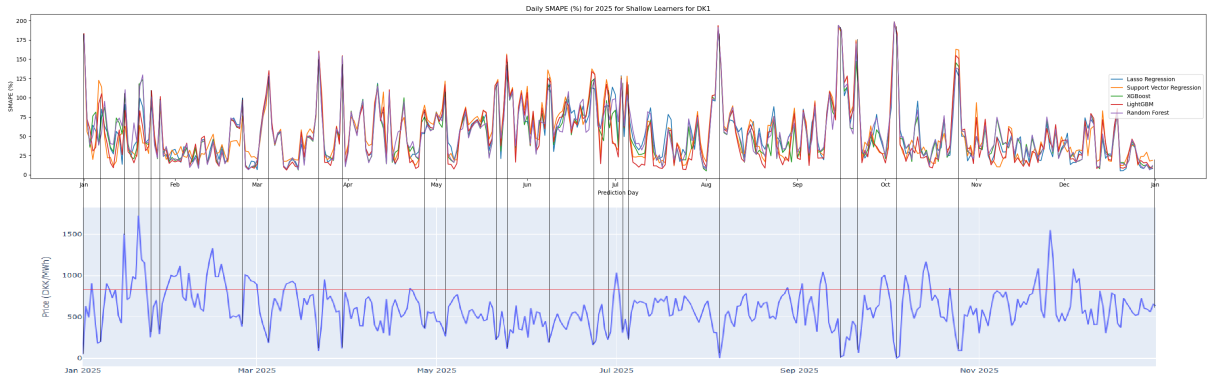


Figure 6.15: Daily SMAPEs for some of the shallow learners in price zone DK1 for 2025 compared with the daily prices. The upper chart shows the daily SMAPEs for shallow learners and the lower chart shows the daily prices as the blue curve. The red line is the price level at which the models have the lowest SMAPE. The vertical lines connect the spikes in SMAPE with the corresponding prices.

Table 6.5 shows the performance metrics for all final deep learner models. The RNN is the best performing deep learner in all three metrics and is also better than all the baseline models in SMAPE and RMSE and is only slightly worse than the Seasonal Naïve model in terms of MAE. It outperforms all the shallow leaners except for LightGBM in terms of SMAPE and is only beaten by LightGBM and Random Forest when looking at MAE and by Random Forest when looking at RMSE. So far we have seen that the models than can only use information about the past price like the Moving Average, Naïve Persistence and the ARIMA model, do not perform well, while models that utilize the explanatory features and Price_lag1 perform better. The RNN has the ability to use both the price values and the explanatory features from the past and the explanatory features for the future prediction, which was expected to be an advantage compared to the baseline models and the shallow learners. The disadvantage of the RNN is its exploding and vanishing gradient problem, which makes it difficult for the model to remember information from a long input sequence like 168 hours. The rest of the deep learners are designed to solve this problem, while still having the advantages of the RNN, which is why it is surprising that the RNN is the one with the best performance.

Model	**Test SMAPE**	**Test RMSE**	**Test MAE**
RNN	52.15 %	306.12 DKK/MWh	248.29 DKK/MWh
GRU	56.24 %	340.01 DKK/MWh	279.65 DKK/MWh
LSTM	57.09 %	379.43 DKK/MWh	282.69 DKK/MWh
LSTM AE	75.43 %	413.70 DKK/MWh	320.05 DKK/MWh

Table 6.5: All final deep learner models performance results sorted by test SMAPE.

To get a better understanding of how the RNN can be the best performing deep leaner, we take a look at the actual predictions of the models. First we look at the predictions of the RNN and GRU models for price zone DK1 for January 2025 and July 2025 in figure 6.16 and figure 6.17. They show that the GRU model predicts one constant value of 578.58 DKK/MWh for every hour through both January and July and in fact, further inspections proved that the same value is repeated through the entire year. The RNN has a bit more variation in its predictions even though it uses a narrow range of values from 477 to 714 DKK/MWh. In January, the predictions mostly jump between these two values and stays at the same value for a whole day to 1 a week at a time. In July the predictions also jump between the two boundaries in a daily pattern.

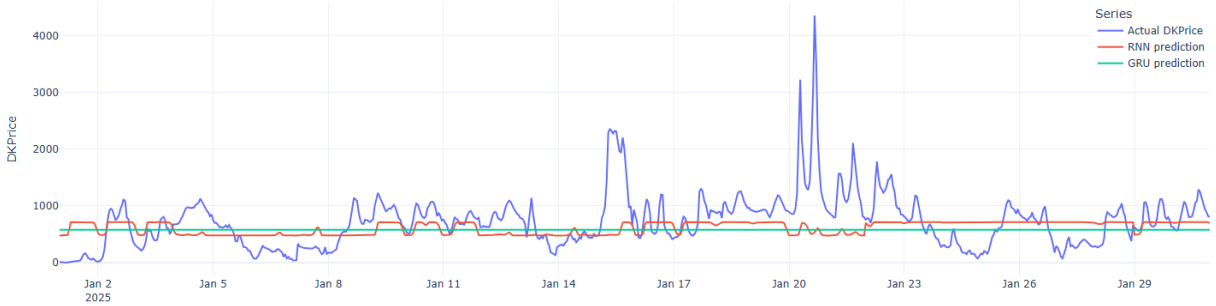


Figure 6.16: Hourly electricity prices for price zone DK1 in January 2025 and the predicted prices of RNN and GRU. The blue line is the actual price, the red line is the RNN prediction and the green line is the GRU prediction.

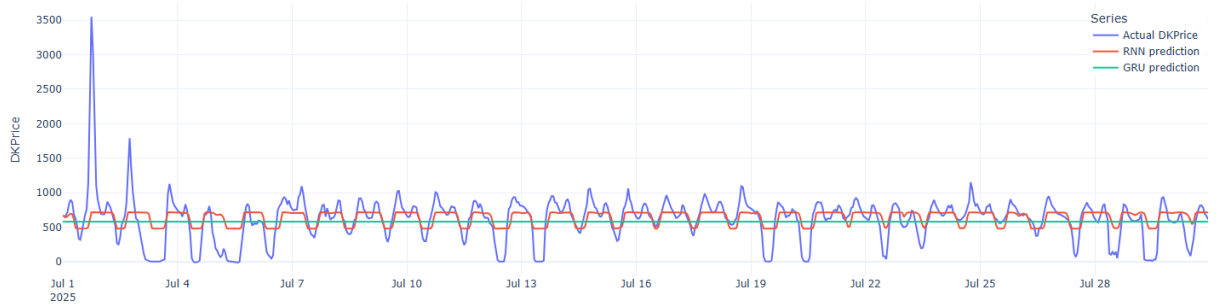


Figure 6.17: Hourly electricity prices for price zone DK1 in July 2025 and the predicted prices of RNN and GRU. The blue line is the actual price, the red line is the RNN prediction and the green line is the GRU prediction.

To get a better understanding of how the RNN model make its predictions, we can examine the SHAP values for the model, which are shown in figure 6.18. The RNN has very small SHAP values with top 20 values between 1 and 14, which means that no features are able to cause large changes to the expected value. This explains why the RNN predictions are in a narrow range and why it struggles to follow large changes in price levels. The feature with the largest SHAP value for RNN is SolarPowerCapacity, which is a very stable feature because its value only changes a couple of times during the year as new power plants are built. During 2025 the solar power capacity in DK1 changes 11 times starting at 3100 MW in January and ending at

3700 MW in December, as shown in figure 6.19. When prices have large changes every hour, it is not very useful to use a constant value for the predictions. It would make more sense to rely more on radiation or forecasted numbers for solar power production, but these are given small SHAP values at around 2. During winter the ranking of SolarPowerCapacity makes even less sense as the solar power production is very low. This might explain why we see the RNN predictions being more static during in January than in July, as its top ranked feature brings no useful information here. An interesting detail is that Price_lag1 does not appear on the top 20, while most of the shallow learners agreed that Price_lag1 was the most important feature. However, it does have some similarities with the shallow leaners' rankings as DEPrice has the second highest SHAP value and OnshoreWindPower, NLPrice and SE4Price are all in the top 7. The SHAP results for the GRU model are not shown in the figure as the model only predicts one constant value, which means that all features have a SHAP value of 0, as they do not affect the output.

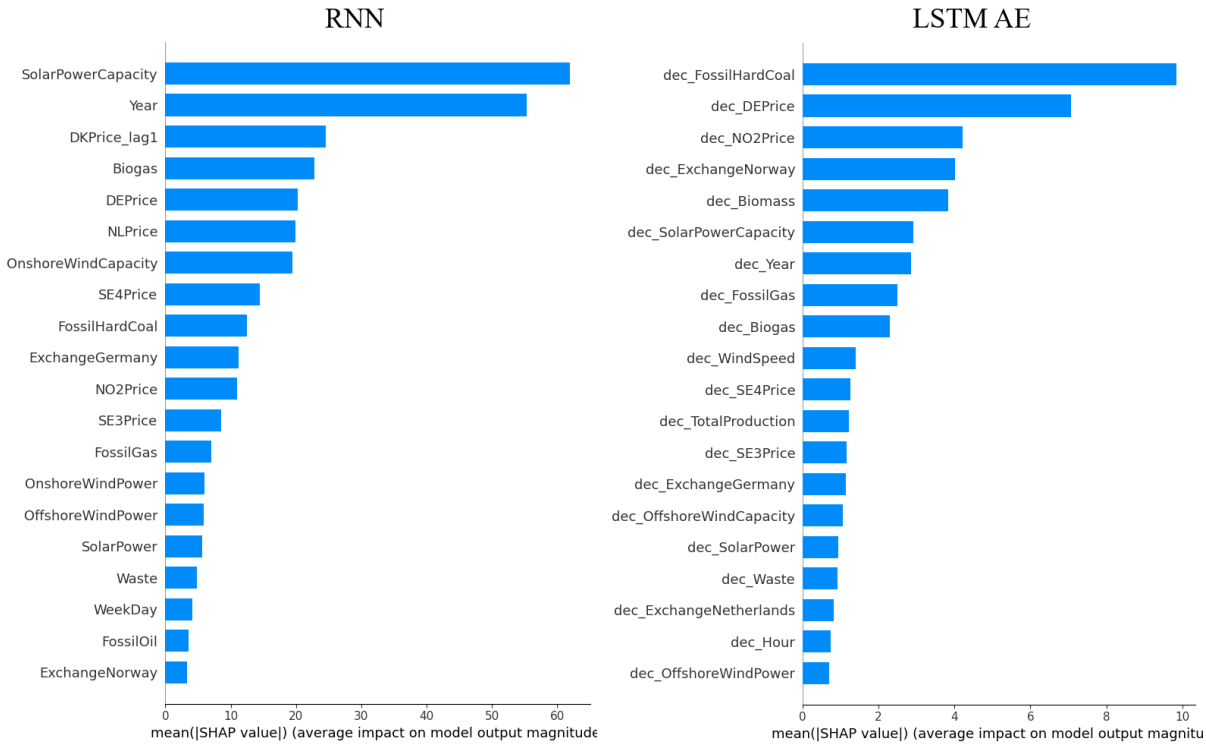


Figure 6.18: SHAP analysis for the RNN and LSTM Autoencoder models in price zone DK1.

Figure 6.19: The development in solar power capacity in price zone DK1 for 2025.

Figure 6.20 and figure 6.21 show the predictions of the LSTM model and the LSTM Autoencoder model for January and July of 2025. They show that the LSTM, just like the GRU model, predicts one constant value for both months and further examination shows that the value is the same for the entire year. The constant price is 563.39 DKK/MWh. For the same reason its SHAP results are not shown. The LSTM AE model is that of the deep learners with the widest prediction range spanning from -347.53 DKK/MWh to 446.01 DKK/MWh. It is the only deep learner capable of predicting prices at the same level as the lowest actual prices, but not capable of predicting higher than 446.01 DKK/MWh, which is lower than the mean price value of 2025 at 605.35 DKK/MWh. This upper limit makes the model unable to reach the actual prices most of the time both for January and July and when prices drop into its range, it is not even good at predicting the exact time and scale of the drops. However, it does seem to identify a daily seasonality that it follows most days in July, but is not visible in January. If we look at the SHAP values for the LSTM AE in figure 6.18, we see that all the features in top 20 are decoder features, which means features from the input sequence. This means that the 20 most influencing features for the predictions only provide information about the past. The top 20 features all have very low SHAP values between 0.5 and 10, which means that all future explanatory features have SHAP values below 0.5 and thus have very little influence on the predictions. The top ranked feature is FossilHardCoal, which has its values plotted in figure 6.22. As the figure shows FossilHardCoal provides very little information during the summer months and in the winter months it can only provide some information about the production from the past 168 hours. Like most of the other models it has DEPrice ranked second and NO2Price is within the top 7.

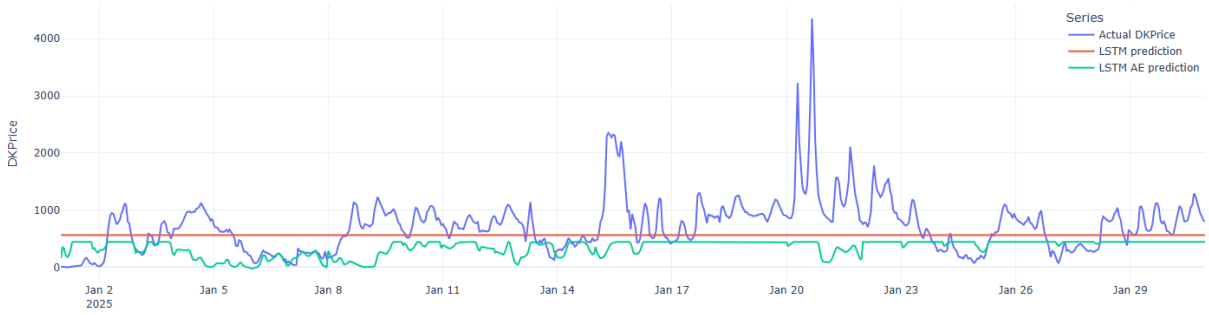


Figure 6.20: Hourly electricity prices for price zone DK1 in January 2025 and the predicted prices of LSTM and LSTM AE. The blue line is the actual price, the red line is the LSTM prediction, and the green line is the LSTM AE prediction.

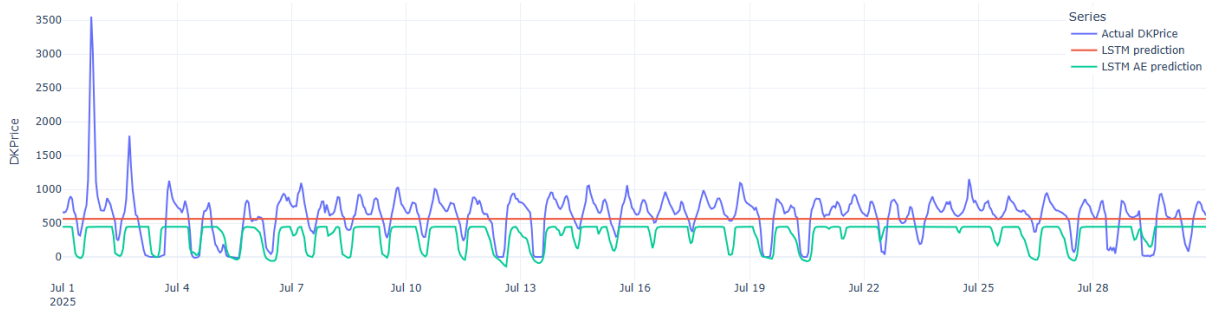


Figure 6.21: Hourly electricity prices for price zone DK1 in July 2025 and the predicted prices of LSTM and LSTM AE. The blue line is the actual price, the red line is the LSTM prediction, and the green line is the LSTM AE prediction.

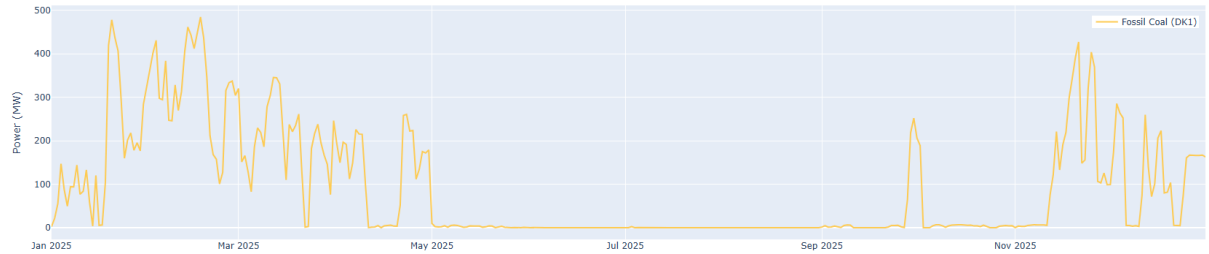


Figure 6.22: Hourly values of the FossilHardCoal feature showing the power production from fossil hard coal measured in MW for price zone DK1 in 2025.

Overall we can see that some of the models perform well in July, but all models perform poorly in January. This might indicate that either it is very difficult for the models to both predict well in the summer period and the winter period, or that the data does not provide the necessary information for the models to make accurate predictions during the winter.

Another way to compare the models is to look at their average daily error across a 7-day prediction horizon. This comparison is done in section 6.1.3, and will therefore not be discussed in this section. In section 6.1.5, we compare the models with statistical descriptors of the models' predictions. The final conclusion on which model is the best performing will be made in section 6.1.6.

6.1.2 Impact of Explanatory Features

In this section, we examine how the inclusion of explanatory features impacts the performance of the models, which is the first of our secondary research questions. Among the models tested in this project we both had some univariate models: Seasonal Naïve, Moving Average, Naïve Persistence and ARIMA, which only take past values of the target variable as input, thus these models do not use any explanatory features to make predictions. We also tested some multivariate models: LightGBM, XGBoost, Random Forest, Linear Regression, SVR, ARIMAX, RNN, GRU, LSTM and LSTM AE, which all use explanatory features as input. For some of the best multivariate shallow and deep learners, LightGBM, ARIMAX and RNN, we have tested a univariate version of those models to compare their performances with and without explanatory features in order to get an understanding of their contribution to the models performances. Table 6.6 shows these results.

Table 6.6: Univariate and multivariate models' performance results sorted by test SMAPE.

Model	Test SMAPE	Test RMSE	Test MAE
LightGBM univariate	54.88 %	368.73 DKK/MWh	282.45 DKK/MWh
LightGBM multivariate	50.58 %	314.41 DKK/MWh	237.86 DKK/MWh
ARIMA	63.29 %	414.26 DKK/MWh	330.33 DKK/MWh
ARIMAX	61.47 %	332.30 DKK/MWh	263.68 DKK/MWh
RNN univariate	69.05 %	497.17 DKK/MWh	350.72 DKK/MWh
RNN multivariate	52.44 %	292.00 DKK/MWh	240.07 DKK/MWh

The results show that for all three types of models, the multivariate version that uses explanatory features performs better across all three metrics than the univariate version. The SHAP analyses of these models in section 6.1.1 showed that many of the explanatory features have a significant influence on the predictions, which indicate that the explanatory features provide valuable information and that the models are able to learn how to use the information to improve their predictions. Table 6.7 shows the performance of all the univariate models and their average and table 6.8 shows the performance of all the multivariate models and their average.

Table 6.7: Univariate and multivariate models' performance results sorted by test SMAPE.

Model	Test SMAPE	Test RMSE	Test MAE
Seasonal Naive	56.59 %	341.39 DKK/MWh	243.73 DKK/MWh
Moving Average	60.31 %	397.96 DKK/MWh	303.34 DKK/MWh
Naïve Persistence	63.50 %	413.82 DKK/MWh	302.81 DKK/MWh
ARIMA	63.29 %	414.26 DKK/MWh	330.33 DKK/MWh
LightGBM univariate	54.88 %	368.73 DKK/MWh	282.45 DKK/MWh
RNN univariate	69.05 %	497.17 DKK/MWh	350.72 DKK/MWh
Average	61.27 %	405.56 DKK/MWh	302.23 DKK/MWh

Table 6.8: Univariate and multivariate models' performance results sorted by test SMAPE.

Model	Test SMAPE	Test RMSE	Test MAE
LightGBM	50.99 %	314.48 DKK/MWh	237.93 DKK/MWh
XGBoost	52.25 %	286.44 DKK/MWh	217.77 DKK/MWh
Random Forest	53.15 %	293.34 DKK/MWh	221.93 DKK/MWh
Lasso Regression	53.45 %	298.95 DKK/MWh	226.65 DKK/MWh
SVR	54.12 %	383.90 DKK/MWh	292.18 DKK/MWh
ARIMAX	54.34 %	305.72 DKK/MWh	237.34 DKK/MWh
RNN	52.15 %	306.12 DKK/MWh	248.29 DKK/MWh
GRU	56.24 %	340.01 DKK/MWh	279.65 DKK/MWh
LSTM	57.09 %	379.43 DKK/MWh	282.69 DKK/MWh
LSTM AE	75.43 %	413.70 DKK/MWh	320.05 DKK/MWh
Average	55.92 %	332.21 DKK/MWh	256.45 DKK/MWh

Most of the multivariate models perform better than the best performing univariate model, and the average performance of the multivariate models is better in all three metrics than the average performance of the univariate models. Based on these results we can conclude that the explanatory features have a positive impact on the performance of the models. The three types of models in table 6.6 are the most comparable, as they are compared within the same model type, so we can use their reults to get an idea of how much the explanatory features improve the performance of the models. Table 6.9 shows how much each model is improved in percentage in all three metrics by the inclusion of explanatory features.

Table 6.9: Univariate and multivariate models' performance results sorted by test SMAPE.

Model	Test SMAPE	Test RMSE	Test MAE
LightGBM	7.84 %	14.73 %	15.79 %
ARIMA(X)	2.88 %	19.78 %	20.18 %
RNN	24.06 %	41.27 %	31.55 %
Average	11.59 %	25.26 %	22.50 %

6.1.3 Prediction Error Across 7 days

Section 5.1.4 present figures showing the development of each evaluation metric averaged across all of 2025 for the 7 prediction days for the three baseline models. Looking at Figures 5.4a and 5.5a showing the development in SMAPE, the graphs do not follow a linear trend, but they do share a similar overall trend. All three models increase in SMAPE from Day 1 up to and including Day 5 for both DK1 and DK2, although the actual SMAPE values are larger for DK2 than DK1. From Day 5 to Day 6, the SMAPE decreases substantially, before having a slight increase from Day 6 to Day 7. Both RMSE and MAE do not show any clear trend. Figure 5.4b shows the development of RMSE for DK1. Naïve Persistence appears to steadily increase across the 7 days, which makes sense, since it predicts the same value across the entire prediction horizon. Moving

Average has RMSE values similar to Naïve Persistence, but there is more variation across the days. Seasonal Naïve has a decreasing RMSE from Day 1 to Day 4, from Day 4 to Day 7, it then increases. This is nearly the opposite pattern of the SMAPE graph. Figure 5.5b shows the development of RMSE for DK2, which shows the same patterns for the three models as DK1. These patterns also appear for MAE, which is shown in Figure 5.4c for DK1 and in Figure 5.5c for DK2.

The same figures were made for the shallow learners, which can be found in section 5.2.8. Figure 5.15a shows the 7-day development in SMAPE for DK1. ARIMA has a larger SMAPE than the other 6 shallow learners, but it does follow a trend similar to the other s hallow learners, which is also the same trend as the baseline models, though there is a slight difference. Their SMAPE stays at plateau from Day 1 to Day 3, it then increases from Day 3 to Day 5, and finally it decreases at Day 6 and remains at the same level on Day 7. The patterns are not as obvious for DK2, which can be seen in Figure 5.16a. Here, ARIMAX remains at the maximum value during all 7 days. Lasso Regression increases in SMAPE from Day to Day 3 and then slowly decreases from there. The five remaining shallow learners generally follow the same pattern; they increase from Day 1 to Day 5, then decrease from Day 5 to Day 6, and stay at that level for Day 7. This is the same as for DK1.

The 7-day development in RMSE for DK1 can be seen in Figure 5.15b. Just as with SMAPE, ARIMA has a larger value than the other models, which steadily increases from Day 1 to Day 7. SVR does not follow the same pattern as the 5 remaining shallow learners. It has a slight decrease from Day 1 to Day 2, and then an increase until Day 5, before it decreases once more. The remaining models all decrease in value from Day 1 to Day 4, and then increase again until Day 7. For DK2, it was not possible to include ARIMAX in the visualization, as it would have made it impossible to differentiate between the other models. The other 6 shallow learners' 7-day development in RMSE can be seen in Figure 5.16b. There are no clear patterns or trends for them, apart from SVR which is quite consistent. Instead, they increase and decrease across the 7 days, apart from ARIMA which has a pattern similar to what it did for DK1, where it steadily increases from Day 1 to Day 7. Figure 5.15c shows the 7-day development in MAE for DK1, which shows a very similar pattern for each model as with RMSE. The same is the case for DK2, where the 7-day development in MAE can be seen in Figure 5.16c. Once again ARIMAX has been excluded, so it is possible to discern between the other models. SVR remains at a steady level, while the others fluctuate more across the 7 days.

Figure 5.27a shows the 7-day development in SMAPE for the four deep learners for DK1. The shape of each graph follows the same patterns, where there is a very slight overall increase from Day 1 to Day 3, it then increases a large amount from Day 3 to Day 5, then a decrease at Day 6, before it plateaus for Day 7. This is the same overall patterns as with the baseline models and the shallow learners, which suggest that there are some patterns in the data that they have not learnt properly. Figure 5.27b shows the 7-day development in RMSE, and Figure 5.27c shows the development in MAE. The two figures shows very similar patterns for the four models. RNN has the lowest values and decreases in value from Day 1 to Day 2, after which it steadily increases. GRU and LSTM follow each other quite closely and have a similar pattern as RNN. The graphs for LSTM AE are noticeably different from the others. It decreases substantially in value from Day 1 to Day 5 and then increases again until it hits a similar level as it was at for Day 1. Once again, there are some patterns that have similarities with the baseline models and shallow learners.

Since the prediction horizon is 168 hours, or 7 days, the first day of the prediction horizon will always fall on the same weekday, which happens to be a Wednesday. This means that the increase that happens for Days 4 and 5 is due to it being the weekend. Most people work a regular full-time job or attend school, university, etc., which occurs 5 days of the week. These are days, where the electricity demand is higher, due to workplaces uses larger amounts of electricity. If the models get used to this 5 day trend, it makes sense that the error generally increases from Day 3 to Day 5, as people are home, which means that there is a sudden change in the demand and consumption of electricity. A model that does capture this pattern well is the Seasonal Naïve, but this has more to do with the fact that it averages the same hour for the 3 previous weeks, so the pattern is already included in the data.

It is also important to note once again that these 7-day developments do not reflect the influence that a weather forecast may have on the predictions, since these errors are based on using actual weather data for the predictions. A small scenario analysis has been done to determine the influence that this might have on the models, which is described in the following section, section 6.1.4.

6.1.4 Addition of Noise to Weather Data

The dataset used for the all training, validation, and testing of the models examined used actual values for the weather features; wind speed and radiation. This means that the predictions made also used the actual values rather than forecasts. The implications and consequences of this will be described in further detail in section 6.2.2. It was not possible to find actual, historic forecasts of the weather features, which would have been the optimal solution. However, there are different methods of simulating the inaccuracy that is expected with weather forecasts.

Weather forecasts have steadily become more accurate over time as more data and better methods of forecasting have become available. This also means that data on the accuracy of weather forecasts exist. It is estimated that a 3-day weather forecast has an accuracy of around 97 %, a 5-day weather forecast of around 92 %, and a 7-day forecast of around 77 % [73]. This means that the predicted hours from 1 up to and including hour 72 should have 3 % of noise added to the weather features to simulate a 3-day weather forecast. The hours from hour 73 up to and including hour 120 should have 8 % of noise added to the weather features to simulate a 5-day weather forecast. And finally, the hours from hour 121 up to and including hour 168 should have 23 % noise added to the weather features to simulate a 5-day weather forecast. These noise values were added to the predictions function of the code, so that all weather features fed to the testing of the model used the "forecasted" weather features.

Using the "forecasted" weather features was not relevant for the baseline models or ARIMA, as those models only used DKPrice for the testing. The analysis was done for all other models using the same hyperparameter combinations that were described in section 5. This means that the models were not hyperparameter tuned again with the weather feature noise added during the validation portion of the tuning. It is possible that using weather features with noise during validation (and testing) may mean that a different set of hyperparameters would be the best performer.

The results of adding noise to the weather features for the shallow learners for DK1 can be seen in Table 6.10. The models have been sorted according to their test SMAPE when tested on actual values. There is not a stark contrast between using the actual values or adding noise for prediction purposes. The models show nearly the same overall performance in the two scenarios, and if there is a difference, it is a slight worsening of their performance, when the noise has been added.

Table 6.10: Performance of shallow learners with weather noise for DK1 sorted by test SMAPE based on their performance using actual weather values.

Using Actual Values

Model	Test SMAPE	Test RMSE	Test MAE
LightGBM	50.99 %	314.48 DKK/MWh	237.93 DKK/MWh
XGBoost	52.25 %	286.44 DKK/MWh	217.77 DKK/MWh
Random Forest	53.15 %	293.34 DKK/MWh	221.93 DKK/MWh
Lasso Regression	53.45 %	298.95 DKK/MWh	226.65 DKK/MWh
SVR	54.12 %	383.90 DKK/MWh	292.18 DKK/MWh
ARIMAX	54.34 %	305.72 DKK/MWh	237.34 DKK/MWh

Using Noise for Prediction

Model	Test SMAPE	Test RMSE	Test MAE
LightGBM	50.99 %	314.48 DKK/MWh	237.93 DKK/MWh
XGBoost	52.12 %	289.21 DKK/MWh	219.14 DKK/MWh
Random Forest	53.16 %	293.42 DKK/MWh	222.08 DKK/MWh
Lasso Regression	53.49 %	299.18 DKK/MWh	226.82 DKK/MWh
SVR	54.13 %	383.89 DKK/MWh	292.17 DKK/MWh
ARIMAX	54.35 %	305.81 DKK/MWh	237.42 DKK/MWh

The same trends are present, when this analysis is performed for DK2, which can be seen in Table 6.11. The performance of the models has nearly stayed the same, when the noise has been added. However, the performance of XGBoost is worth looking at, as it has actually improved across the three evaluation metrics. It is not possible to conclude based on these results that XGBoost actually performs better with noise added, but it is something that would be worth analyzing more in-depth.

Table 6.11: Performance of shallow learners with weather noise for DK2 sorted by test SMAPE based on their performance using actual weather values.

Using Actual Values

Model	Test SMAPE	Test RMSE	Test MAE
SVR	52.24 %	330.48 DKK/MWh	255.29 DKK/MWh
LightGBM	69.43 %	397.04 DKK/MWh	332.16 DKK/MWh
XGBoost	78.99 %	428.39 DKK/MWh	362.99 DKK/MWh
Random Forest	82.17 %	440.09 DKK/MWh	373.61 DKK/MWh
Lasso Regression	84.22 %	437.18 DKK/MWh	365.04 DKK/MWh
ARIMAX	200.00 %	7.34e8 DKK/MWh	6.48e8 DKK/MWh

Using Noise for Prediction

Model	Test SMAPE	Test RMSE	Test MAE
SVR	52.25 %	330.63 DKK/MWh	255.40 DKK/MWh
LightGBM	69.43 %	397.05 DKK/MWh	332.16 DKK/MWh
XGBoost	77.57 %	425.14 DKK/MWh	359.96 DKK/MWh
Random Forest	82.17 %	440.08 DKK/MWh	373.59 DKK/MWh
Lasso Regression	84.21 %	437.20 DKK/MWh	365.08 DKK/MWh
ARIMAX	200.00 %	7.34e8 DKK/MWh	6.48e8 DKK/MWh

The analysis of the two scenarios was also repeated for the deep learners, which can be seen in Table 6.12. For the deep learners, there is less variation than with the shallow learners, only RNN and LSTM AE show a slight variation between the two scenarios. They both show a slight decrease in performance, when noise is added to the weather features.

Table 6.12: Performance of deep learners with weather noise for DK1 sorted by test SMAPE based on their performance using actual weather values.

Using Actual Values

Model	**Test SMAPE**	**Test RMSE**	**Test MAE**
RNN	52.15 %	306.12 DKK/MWh	248.29 DKK/MWh
GRU	56.24 %	340.01 DKK/MWh	279.65 DKK/MWh
LSTM	57.09 %	341.21 DKK/MWh	282.69 DKK/MWh
LSTM AE	75.43 %	366.05 DKK/MWh	320.05 DKK/MWh

Using Noise for Prediction

Model	**Test SMAPE**	**Test RMSE**	**Test MAE**
RNN	52.15 %	306.19 DKK/MWh	248.35 DKK/MWh
GRU	56.24 %	340.01 DKK/MWh	279.65 DKK/MWh
LSTM	57.09 %	341.21 DKK/MWh	282.69 DKK/MWh
LSTM AE	74.88 %	366.93 DKK/MWh	320.91 DKK/MWh

The results of the two scenarios are also supported by the findings from the SHAP analysis, which was discussed in section 6.1.1. The models do not appear to base their predictions on the values of wind speed and radiation. It is important to note that this is based on models that have been hyperparameter tuned, where the validation was done using actual values. If the entire process had been repeated with noise added during the predictions, the models may not necessarily have reached the same set of hyperparameters. This also means that it is not possible to conclude based on this data, whether the weather data plays a large role or not.

6.1.5 Predicted Values

While preparing data for the various results and analyses, we noticed that the predicted electricity prices looked strange for some of the models. To better determine this, a number of statistical metrics were calculated for each model and for the actual values to allow for comparison. These statistical metrics can be seen for the baseline models for DK1 in Table 6.13. By just taking a quick glance at these metrics for the three baseline models and the actual data, the Seasonal Naïve model appears to most closely resemble the characteristics of the actual data, although it does not capture the same range and variance. Naïve Persistence and Moving Average show a much more narrow range and spread. This does make sense, as they do not capture any

real trends within the data. The results are very similar for DK2, which can be seen in Table 6.14. It once again appears that Seasonal Naïve best encompasses the characteristics and trends of the data, although it is not able to capture the large fluctuations likely caused by the high volatility.

Table 6.13: Statistics metrics for baseline models for DK1 calculated using the predictions value from the final evaluation of the given model. The final column shows the statistical metrics based on the actual prices. All metrics have the unit DKK/MWh, apart from the sample variance, which has the unit $(DKK/MWh)^2$.

	Naïve Persistance	**Moving Average**	**Seasonal Naïve**	**Actuals**
Mean	562.32	596.66	604.93	604.46
Median	648.01	581.33	616.73	636.91
Bias	-42.13	-7.79	0.47	0.00
Sample Variance	42,147.73	31,893.72	92,534.21	144,681.03
Sample STD	205.30	178.59	304.19	380.37
Maximum	917.31	1,247.83	2,747.23	4,352.69
Minimum	16.11	189.54	-124.17	-284.65
Range	901.20	1,058.29	2,871.41	4,637.34
Q1 Percentile	450.06	491.51	430.88	399.81
Q3 Percentile	680.95	668.60	772.99	804.91
IQR	230.89	177.09	342.11	405.10

Table 6.14: Statistics metrics for baseline models for DK2 calculated using the predictions value from the final evaluation of the given model. The final column shows the statistical metrics based on the actual prices. All metrics have the unit DKK/MWh, apart from the sample variance, which has the unit $(DKK/MWh)^2$.

	Naïve Persistance	**Moving Average**	**Seasonal Naïve**	**Actuals**
Mean	548.96	606.49	616.19	615.74
Median	614.20	589.30	614.01	639.78
Bias	-55.50	2.04	11.73	0.00
Sample Variance	49,527.81	33,808.70	98,194.32	149,351.96
Sample STD	222.55	183.87	313.36	386.46
Maximum	955.09	1,298.86	2,747.31	4,352.69
Minimum	18.94	291.34	-120.12	-189.10
Range	936.15	1,007.52	2,867.43	4,541.79
Q1 Percentile	446.96	495.91	431.14	385.85
Q3 Percentile	678.38	669.84	787.17	815.78
IQR	231.42	173.93	356.03	429.92

Table 6.15 shows the calculated statistical metrics for the shallow learners along with the actuals. SVR has a sample variance and IQR that are substantially lower than that of the dataset. This suggests that it is not able to capture the many sudden peaks that may occur and instead remains at a very steady level. The remaining 6 models show a greater variance, range, and IQR, though LightGBM only captures around half of the range and variance of the other models. Random Forest appears to best capture the characteristics of the data, at least for DK1. But it is

followed closely by ARIMAX, XGBoost, and ARIMA, which all appear to capture part of the characteristics of the dataset. The models generally struggle to capture the sudden peaks that occur during the test period, but ARIMAX, Lasso Regression, Random Forest, and XGBoost have all learnt that it is possible for the price to be quite negative.

The same statistical metrics but for DK2 can be seen in Table 6.16. It is immediately obvious that ARIMAX is an enormous outlier compared to the other models, which was also clear from the evaluation metric results. Just as with DK1, ARIMA appears to be the model that best captures the characteristics of the dataset due to its large sample variance, range, and IQR. Compared to the statistical metrics calculated for DK1, the DK2 models generally perform worse at understanding the characteristics of the dataset. They all show a smaller variance, range, and IQR. This is also confirmed by the increase in SMAPE, RMSE, and MAE during the final evaluation compared to their DK1 counterpart.

Table 6.15: Statistics metrics for shallow learners for DK1 calculated using the predictions value from the final evaluation of the given model. The final column shows the statistical metrics based on the actual prices. All metrics have the unit DKK/MWh, apart from the sample variance, which has the unit $(DKK/MWh)^2$.

	ARIMA	ARIMAX	Lasso	LightGBM
Mean	648.67	633.57	605.50	708.80
Median	650.53	624.38	635.55	707.40
Bias	44.21	29.11	1.05	104.34
Sample variance	71,916.65	85,966.24	78,221.71	42,081.53
Sample standard deviation	268.17	293.20	279.68	205.14
Maximum	1,375.30	2,429.66	2,311.21	1,772.68
Minimum	27.32	−417.63	−434.49	167.01
Range	1,347.98	2,847.28	2,745.70	1,605.67
Q1 percentile	483.06	466.03	456.39	601.92
Q3 percentile	754.43	796.93	762.32	819.72
IQR	271.37	330.91	305.93	217.81

	Random Forest	**SVR**	**XGBoost**	**Actuals**
Mean	615.06	743.85	617.47	604.46
Median	621.15	743.40	633.26	636.91
Bias	10.61	139.40	13.02	0.00
Sample variance	87,733.85	343.32	79,737.64	144,681.03
Sample standard deviation	296.20	18.53	282.38	380.37
Maximum	2,357.09	816.84	2,279.66	4,352.69
Minimum	−325.70	680.30	−416.25	−284.65
Range	2,682.79	136.54	2,695.91	4,637.34
Q1 percentile	445.02	731.56	444.16	399.81
Q3 percentile	778.69	755.75	776.86	804.91
IQR	333.67	24.19	332.70	405.10

Table 6.16: Statistics metrics for shallow learners for DK2 calculated using the predictions value from the final evaluation of the given model. The final column shows the statistical metrics based on the actual prices. All metrics have the unit DKK/MWh, apart from the sample variance, which has the unit $(DKK/MWh)^2$.

	ARIMA	**ARIMAX**	**Lasso**	**LightGBM**
Mean	656.33	−5.11E+08	312.12	420.43
Median	647.25	−5.00E+08	301.47	394.03
Bias	40.59	-5.11E+08	-292.34	-184.03
Sample variance	72,514.44	3.93E+17	19,449.13	9,392.31
Sample standard deviation	269.29	6.27E+08	139.46	96.91
Maximum	1,403.59	9.97E+08	914.11	674.87
Minimum	12.32	−2.59E+09	−62.09	271.15
Range	1,391.27	3.58E+09	976.20	403.73
Q1 percentile	479.35	−9.50E+08	214.60	346.89
Q3 percentile	779.00	−3.84E+07	399.48	478.29
IQR	299.65	9.11E+08	184.89	131.40

	Random Forest	**SVR**	**XGBoost**	**Actuals**
Mean	343.51	576.51	354.59	615.74
Median	320.44	576.55	331.21	639.78
Bias	-260.94	-27.95	-249.86	0.00
Sample variance	14,655.60	7,339.40	12,224.05	149,351.96
Sample standard deviation	121.06	85.67	110.56	386.46
Maximum	648.81	919.52	668.61	4,352.69
Minimum	104.06	338.19	115.14	−189.10
Range	544.75	581.33	553.46	4,541.79
Q1 percentile	257.62	515.20	277.86	385.85
Q3 percentile	425.60	635.78	407.62	815.78
IQR	167.98	120.58	129.75	429.92

This process was also repeated for the deep learners, which can be seen in Table 6.17. Looking at the table, one thing is immediately obvious; both GRU and LSTM predict the same value during the entire prediction window. This is clear from the variance, range, and IQR of 0 for both. GRU only predicts electricity prices of 578.58 DKK/MWh, and LSTM only predicts electricity prices of 563.39 DKK/MWh. Based on these metrics, it is not possible to conclude why that is. It could be issues with the structure and definition of the model, which does not allow it to make continuous predictions. Maybe there is an underlying issue within the pipeline that the models have been integrated in. In order to utilize these models to their fullest potential, it would be crucial to investigate further why this is. LSTM AE is better at predicting the variance and range that the prices may have than RNN. RNN only predicts prices that are in the range from 476.34 DKK/MWh to 714.86 DKK/MWh, while the actual prices range from

-284.65 DKK/MWh to 4,352.69 DKK/MWh. This means that the range of the actual prices is around 20 times larger than what RNN predicts. LSTM AE is much better at predicting very low prices, but the maximum price predicted is only 446.01 DKK/MWh, which is just under 10 times less than the actual maximum price.

Table 6.17: Statistics metrics for deep learners for DK1 calculated using the predictions value from the final evaluation of the given model. The final column shows the statistical metrics based on the actual prices. All metrics have the unit DKK/MWh, apart from the sample variance, which has the unit $(DKK/MWh)^2$.

	RNN	**GRU**	**LSTM**	**LSTM AE**	**Actuals**
Mean	596.11	578.58	563.39	281.04	604.46
Median	605.63	578.58	563.39	443.57	636.91
Bias	-8.34	-25.87	-41.07	-323.42	0.00
Sample Variance	8,688.01	0.00	0.00	48,862.94	144,681.03
Sample STD	93.21	0.00	0.00	221.05	380.37
Maximum	714.86	578.58	563.39	446.01	4,352.69
Minimum	476.34	578.58	563.39	-347.53	-284.65
Range	238.52	0.00	0.00	793.54	4,637.34
Q1 Percentile	488.50	578.58	563.39	-3.99	399.81
Q3 Percentile	694.19	578.58	563.39	446.01	804.91
IQR	205.69	0.00	0.00	450.00	405.10

6.1.6 Final Model Evaluation

The final evaluation of the models used all three evaluation metrics, which can fluctuate in size. This means that it is difficult to determine the overall best performing model by focusing one of the evaluation metrics. This is also discussed further in section 6.3.4. Section 6.1.5 also showed the importance of considering other statistical metrics than the evaluation metrics, as they do not show the complete picture. Therefore, to evaluate the overall performance of each model, a weighted sum model was defined. The weighted sum model considers 6 metrics, where 3 of them are the evaluation metrics, and the remaining three are quartile metrics for quartile 1 (25 %), quartile 2 (50 %), and quartile 3 (75 %) based on the average error for each prediction. The SMAPE, RMSE, and MAE were also calculated slightly differently. During hyperparameter tuning and final evaluation, they have been calculated as weekly averages. In this case, they are hourly averages, which means that the values will likely differ slightly from those presented previously. The values of each metric are normalized to allow for easier comparison across models, which are summarized to give the composite score for the model.

The results of this evaluation for DK1 can be seen in Table 6.18, where the lowest value of each metric has been marked bold. LightGBM has one bold value, XGBoost has four bold values, and Random Forest has one bold value. This means that based on the 6 metrics examined, XGBoost is the best overall performing model for DK1. The best baseline model is Seasonal Naïve, and the best deep learner is RNN.

Table 6.18: Final evaluation metrics for all DK1 models, which have been sorted based on their hourly average SMAPE. The metric values made bold show the lowest value within that metric.

Part 1: Error metrics

Category	Model	SMAPE (%)	RMSE (DKK/MWh)	MAE (DKK/MWh)
Shallow	LightGBM	**51.69**	336.26	240.93
Deep	RNN	52.15	345.96	248.29
Shallow	XGBoost	52.93	**311.48**	**220.35**
Shallow	RandomForest	53.81	318.04	224.38
Shallow	Lasso	54.20	323.11	229.62
Shallow	SVR	54.71	402.96	294.89
Shallow	ARIMAX	55.13	332.77	240.55
Deep	GRU	56.24	378.05	279.65
Deep	LSTM	57.09	379.43	282.69
Baseline	SeasonalNaive	57.31	344.39	246.53
Baseline	MovingAverage	61.13	401.04	307.20
Shallow	ARIMA	64.12	448.49	334.47
Baseline	Naive	64.35	417.05	306.58
Deep	LSTM AE	75.43	413.70	320.05

Part 2: Quantile metrics

Category	Model	Q1 (DKK/MWh)	Q2 (DKK/MWh)	Q3 (DKK/MWh)
Shallow	LightGBM	70.19	171.41	352.25
Deep	RNN	75.85	181.65	379.10
Shallow	XGBoost	70.21	**163.10**	**300.81**
Shallow	RandomForest	**69.53**	169.71	312.36
Shallow	Lasso	74.49	166.13	319.06
Shallow	SVR	90.45	201.88	432.65
Shallow	ARIMAX	85.08	180.07	325.58
Deep	GRU	93.46	207.65	429.41
Deep	LSTM	99.81	216.10	428.74
Baseline	SeasonalNaive	81.55	188.71	340.78
Baseline	MovingAverage	116.13	251.56	446.27
Shallow	ARIMA	103.18	249.14	505.47
Baseline	Naive	87.55	229.15	470.11
Deep	LSTM AE	146.38	283.98	425.72

Even though the deep learners weren't examined for DK2, it is still possible to determine the best performing model based on the results from the baseline models and the shallow learners. The results of their evaluation can be seen in Table 6.19, where the lowest value for each metric has been made bold. ARIMAX has not been included due to the magnitude of its errors. SVR has 2 bold values, while Seasonal Naïve has the remaing 4. This means that the best DK2 model is, surprisingly, a baseline model. The best shallow learner is SVR.

Table 6.19: Final evaluation metrics for all DK2 models, which have been sorted based on their hourly average SMAPE. The metric values made bold show the lowest value within that metric.

Part 1: Error metrics

Category	Model	SMAPE (%)	RMSE (DKK/MWh)	MAE (DKK/MWh)
Shallow	SVR	**52.90**	353.89	258.22
Baseline	SeasonalNaive	55.85	**350.66**	**253.26**
Baseline	MovingAverage	60.16	408.88	312.04
Shallow	ARIMA	63.39	450.63	338.74
Baseline	Naive	67.26	430.04	320.25
Shallow	LightGBM	70.32	421.57	336.17
Shallow	XGBoost	79.90	453.30	366.92
Shallow	RandomForest	83.09	464.26	377.43
Shallow	Lasso	85.35	461.53	369.70

Part 2: Quantile metrics

Category	Model	Q1 (DKK/MWh)	Q2 (DKK/MWh)	Q3 (DKK/MWh)
Shallow	SVR	**82.75**	197.77	392.11
Baseline	SeasonalNaive	86.67	**196.45**	**350.75**
Baseline	MovingAverage	119.04	258.53	445.01
Shallow	ARIMA	114.12	261.80	495.48
Baseline	Naive	100.52	250.10	482.18
Shallow	LightGBM	166.95	313.90	432.08
Shallow	XGBoost	194.28	329.81	482.86
Shallow	RandomForest	200.55	339.45	501.32
Shallow	Lasso	183.50	321.85	497.48

6.2 Data and Features

This section will focus on discussing some of the assumptions and choices made during the project regarding data and features. This will also include consequences of these choices.

6.2.1 Impact of Forecasted Features

To avoid having frozen explanatory features during prediction periods for training, validation, and testing, several explanatory features were forecasted using the different methods described in section 3.4.1. Due to being forecasted, they also have errors associated with them. Figure 3.12 shows the SMAPE associated with each feature based on the forecast method used. The average forecast SMAPE for all forecasted features for DK1 was 43.34 % and for DK2 it was 38.73 %.

These errors are far from negligible and will likely have significant contribution to error of the final prediction models that they are used in. The baseline models only used the price for their simple predictions, where the Seasonal Naïve model achieved results that are comparable to the more advanced models, which can be seen in section 5.1.2. These results yield an average SMAPE of 56.59 % and 55.17 %, respectively, for DK1 and DK2. In comparison the best shallow learner in terms of SMAPE, which was LightGBM, achieved an average SMAPE of 50.99 % for DK1 and the best deep learner in terms of SMAPE, which was RNN, achieved an average SMAPE of 52.15 %. This is not a significant improvement and in section 6.1.2 we concluded that the inclusion of the explanatory features only added an average of 11.59 % improvement. However, to understand the true impact of forecasting the explanatory features, we can compare the performance of a model that recieves the true values of the explanatory features with the performance of a model that receives the partly forecasted features as used for the models tested in this project. Table 6.20 shows that the LightGBM using the true values of the explanatory features is improved by 17.45 % in SMAPE, 53.21 % in RMSE and 14.26 % in MAE. Thats a significant improvement and it shows the fill potential of the LightGBM model with optimal input quality. Off course it would never receive 100 % accurate input data in a real world scenario, but it shows the upper limit of this models potential.

Table 6.20: The performance of LightGBM with true explanatory feature values compared with the performance of LightGBM with forecasted explanatory feature values for DK1 in 2025.

Model	**SMAPE (%)**	**RMSE (DKK/MWh)**	**MAE (DKK/MWh)**
LightGBM True	42.09 %	147.16 DKK/MWh	204.01 DKK/MWh
LightGBM Forecasted	50.99 %	314.48 DKK/MWh	237.93 DKK/MWh
Improvement	8.90 %	167.32 DKK/MWh	33.92 DKK/MWh
Improvement (%)	17.45 %	53.21 %	14.26 %

To examine the strengths and weaknesses of the LighGBM model with the true explanatory feature values, we can look at the predictions of the model. Figure 6.23 shows the predictions of both versions of the LightGBM model for January 2025 and figure 6.24 shows the same for July 2025. In both months, we see that, most of the time, the LightGBM with true values is remarkably better at predicting both the time and the scale of the peaks, drops, and longer changes in price levels, than LightGBM with forecasted values, even though the scale is still not fully captured for the tallest peaks.

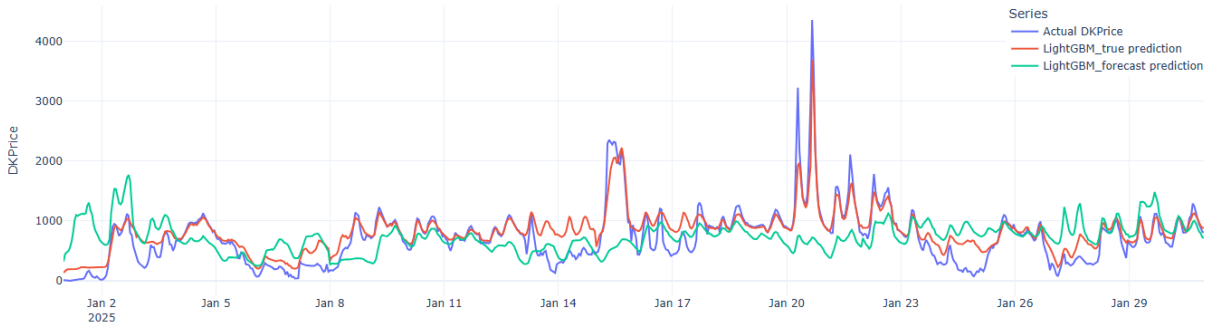


Figure 6.23: Hourly electricity prices for price zone DK1 in January 2025 and the predicted prices of LightGBM with true input values for the explanatory features. The blue line is the actual price, the red line is the LightGBM prediction with true input and the green line is the LightGBM prediction with forecasted input.

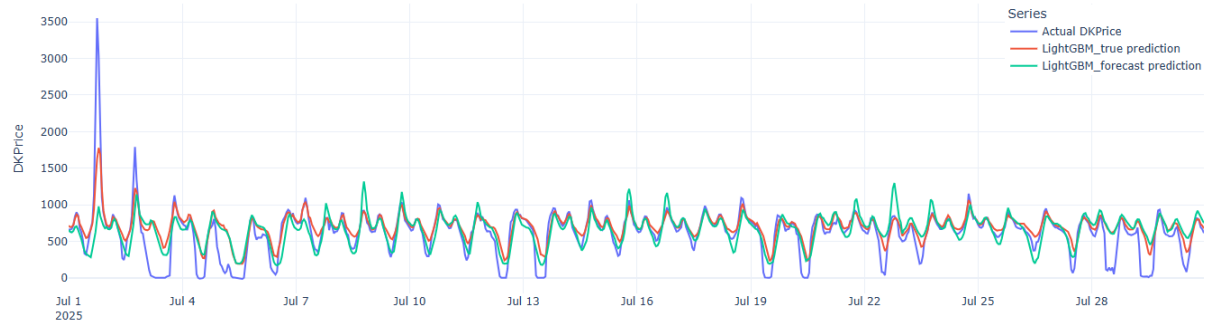


Figure 6.24: Hourly electricity prices for price zone DK1 in July 2025 and the predicted prices of LightGBM with true input values for the explanatory features. The blue line is the actual price, the red line is the LightGBM prediction with true input and the green line is the LightGBM prediction with forecasted input.

When looking at the results of table 6.20 and figures 6.23 and 6.24, it is clear that the explanatory features lose a lot of information when they are forecasted. It is likely that if more work is put into developing more accurate and precise forecasting models that the results of the actual prediction models will also improve. Due to time constraints, it was not possible to spend more time on the forecast models than the analysis described in section 3.4.1. This analysis was not comprehensive, and it is possible that a more thorough hyperparameter tuning of the Random Forest models could have improved performance. Or that other machine learning models might yield better results. In general, any improvements that can be made to the data and forecasting features will in turn also improve the models as they will have more accurate data to work with. Another insight that can be learned from prediction graphs in figures 6.23 and 6.24 is that even the true values of the explanatory features do not provide the model with enough information to accurately predict the prices at all times, which is likely because two features are still not 100 % accurate in this test and thats the WindSpeed and Radiation features. These are still engineered with weighted averages, which means they be misleading at times. This is discussed in section 6.2.2.

6.2.2 Weather data

Section 3.2 described how the two weather features, radiation and wind speed, were averaged for DK1 and DK2 using a weighted average based on the installed capacity in each municipality in each bidding zones. As previously mentioned, this means that the weather features become a kind of statistical proxy for the actual values. It is also based on an inherent assumption that there exists a linear relationship between power production from solar and wind, respectively, and their installed capacities. Sections 6.1.1 and 6.1.4 showed that these features aren't necessarily important for the models, though this could also be different with a different set of hyperparameters.

To get a clearer picture of whether the wind speed and radiation weighted averaged follow the same general trends as the power production from solar and wind, respectively, a couple of visualizations were made. Figure 6.25 shows a comparison between the weighted average wind speed (m/s) and the power production (MW) from onshore and offshore turbines, respectively, for DK1 for January 2025. The dark blue dotted line shows the wind speed, which generally follows the shape of the onshore power production. It is important to note that due to difference in units, it is not possible to fully compare the wind speed and the production, but we can compare the overall trends of the curves to get an idea of how good an indicator the trend of the weighted average of wind speed is of the trend in power production.

Figure 6.26 shows this development for July instead. Here, the wind speed generally follows the onshore wind production. There are some periods, where the wind speed shows a different shape than the onshore production. The similarity in shape between wind speed and onshore power production in both figures could suggest that the wind speed is not a terrible proxy to use.

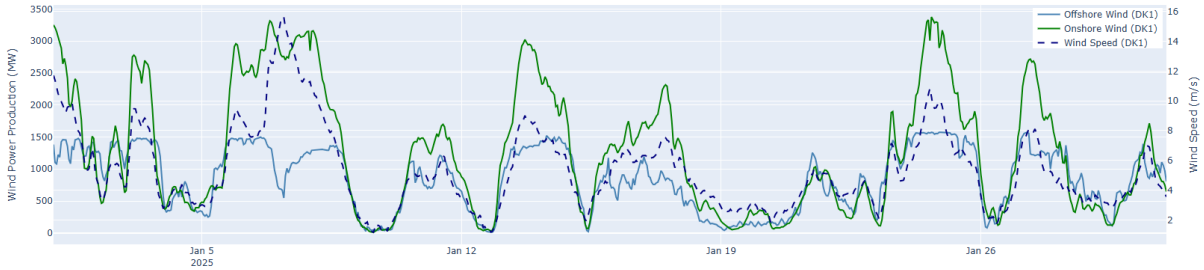


Figure 6.25: Comparison of the average wind speed (m/s) in DK1 with the wind power production of onshore and offshore wind turbines (MW), respectively. This comparison is shown for the month of January 2025.

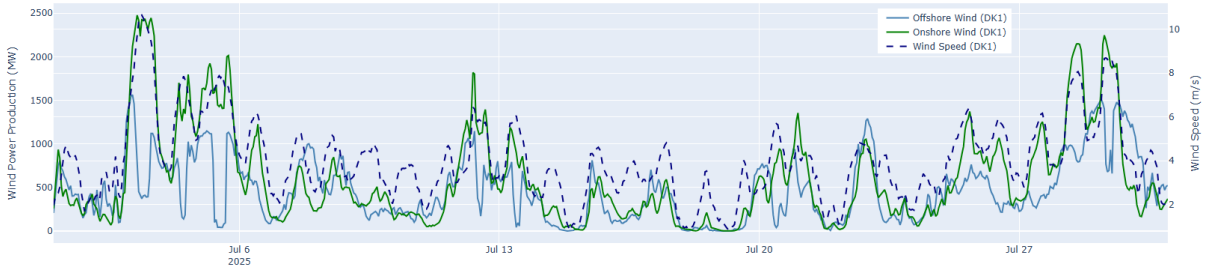


Figure 6.26: Comparison of the average wind speed (m/s) in DK1 with the wind power production of onshore and offshore wind turbines (MW), respectively. This comparison is shown for the month of July 2025.

Figure 6.27 shows the development of radiation (W/m^2) and solar power production (MW) for DK1 in January 2025. The two graphs follow each other quite closely, although the peaks for radiation are generally taller than that of solar power production. Figure 6.28 shows this development for July 2025 instead. The graphs still follow each other quite closely, although solar power production does occasionally reach a higher peak than radiation. But just as with the win speed and wind power production, solar radiation and solar power production do seem to follow similar trends. So, it is not a terrible proxy to use for the purpose of this project.

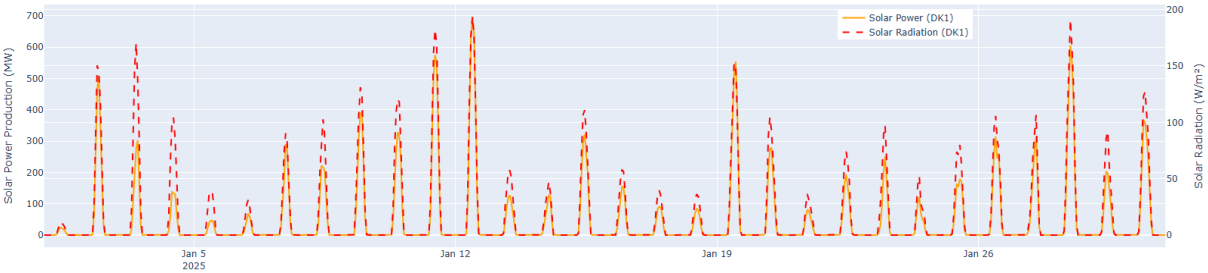


Figure 6.27: Comparison of the average solar radiation (W/m^2) in DK1 with the solar power production (MW). This comparison is shown for the month of January 2025.

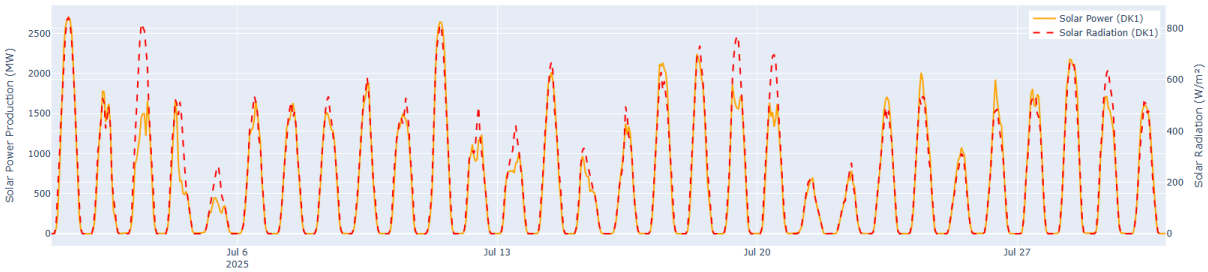


Figure 6.28: Comparison of the average solar radiation (W/m^2) in DK1 with the solar power production (MW). This comparison is shown for the month of July 2025.

Judging from the graphs the weighted averages of the wind speed and radiation seem to be fairly accurate indicators of the changes in wind and solar power production. However we see that sometimes the wind speed follows the trend of the Onshore power production and sometimes it follows the trend of the offshore power production and sometimes its something in between. This

might make it difficult for the models to learn exactly how to use the wind speed feature and no matter how it does it, the wind speed might be misleading at times. The radiation seems to be more represenative and as it only points to one production type it may be easier to utilize. So overall the weigthed averages provide useful information most of the time, but especially the wind speed might cause some errors at times.

6.2.3 Capacity Features

Section 3.2 described how the data was processed to be used for training the various models. The weather data collected from DMI was spread out across the 98 municipalities of Denmark, and we needed one value for DK1 and one value for DK2 for each hour. In order to aggregate the values, a weighted average was calculated by using the installed capacity of each municipality. This installed capacity was the total installed capacity and not the utilized capacity of each municipality, which is not always the same. The installed capacity might give to optimistic expectations to the power production. The installed capacity of solar panels, onshore wind turbines, and offshore wind turbines were also used as features themselves, which has some of the same issues as with the weather features. The installed capacity of solar panels and wind turbines only described the upper boundary of power production, which may be vastly different from the utilized capacity. This also means that the approach we used does not necessarily reflect reality. Though it is appropriate to assume that municipalities with large installed capacities also have a greater production, which means that the windspeed and/or radiation recorded in those areas is more important than others. This assumption can be made, because city planners are not likely to build renewable energy parks in areas that do not have the windspeed or radiation to make it profitable. However, this is not a guarantee.

Therefore, a more accurate approach would have been to collect data that had an hourly resolution of the capacity used for each municipality. This also means that it would have to be calculated hourly, as the production has the potential to change greatly from one hour to the next. This method would increase the overall complexity, which would likely slow down training and require more computational power. However, simply including the actual utilized capacity instead of installed capacity as features would be an easy fix, although during our analysis of results, we attempted to find data for this, but we were unsuccessful. A next step could be to contact the companies that run these different parks or Energinet to see, if they would be willing to share this type of data.

6.2.4 Additional Data

While this project already uses a large number of features to predict the electricity prices, there may be other features that would also have an impact on the Danish prices. There exists a connection between electricity prices and the price of natural gas. This connection is due to the use of natural gas to produce electricity in periods, where there is not enough production from renewable sources [74]. This means that it might be useful to also include features such as gas prices, since there is an existing connection between the two.

Experiments were also made in terms of using cyclic time features, though the results of these showed a decrease in model performance. This is also why they were not included in the dataset used for the hyperparameter tuning, training, validating, and testing described in this report. However, it is possible that they were not utilized correctly, or that the correct combination of cyclic time features and actual time features was not found. It is definitely something that could help the model understand the cyclic nature of hours, months, and weekdays, as it is not clear that weekday 0 follows weekday 6. But when it is converted to sin and cosine values, the context becomes clearer.

6.2.5 Redundant Features

The training of most shallow and deep learners used at least 32 features, which is quite a large number. It is likely that some of these features may have been redundant. There are a number of different feature selection methods that can be used to decrease the number of features used and focus on features that provide the most information for the model. One such method is to use a correlation matrix to determine which features may be correlated. Correlated features are likely to provide the same information and essentially create an overlap. There is a risk that correlated features can negatively impact the model's performance [75]. A correlation matrix is a matrix, or table, that shows the correlation between all features of a dataset. This means that a feature X will also have a correlation calculated with itself, which will always be equal to 1. All correlation values fall between -1 and 1, where a value close to -1 indicates a negative correlation, and a value close to 1 indicates a positive correlation, provided that it is a correlation between two different features. A correlation value equal to or close to 0 indicates no linear relationship or correlation between the two features [76]. This means that if two features of our dataset had a correlation value of around -1 or 1, it would be worth examining, whether removing one of the features would improve the performance of the model. Taking into account that our dataset

includes several features related to price and production, it is very likely that several features would be highly correlated. However, without testing it, it is not possible to conclude, whether this would actually improve the performance, since the features may also be providing context for the models.

Other methods of determining the importance of features exist. There are different statistical tests such as scikit-learn's feature_selection module, which features several different methods. One of these uses a chi-square test to evaluate how important each feature is in relation to the target feature. It can then determine the features with the highest scores, which are the "most relevant" features. Another tool from scikit-learn is recursive feature elimination (RFE), which recursively removes the least important features and then builds the model using the remaining. It continues this process until it reaches the number of features defined by the user [77].

Many more feature selection tools exist, which also means that each tool or method might reach a slightly different result. This implies that it would be useful to examine and test these different methods to determine, which subset of features actually yields the overall best performing model. It is also likely that this will vary between the different models, which has already been shown with SHAP values in section 6.1.1 and the analysis of explanatory features in section 6.1.2. This would be especially important, if more features were added to the dataset, which was discussed in section 6.2.4. The dataset already includes 32 features with the possible inclusion of lag features, and if more features were added, then the time spent training and hyperparameter tuning would increase along with the overall complexity of each model. Though this would be a balance between time spent examining features subsets and time spent on the actual training.

6.3 Method

This section focuses on choices made regarding the method and how that may have impacted the results along with possible solutions to be used for any future work.

6.3.1 Reduced Validation Period

Due to the complexity of the dataset and the models, the validation period during hyperparameter tuning was reduced to save time and computational power. While the validation periods selected include prediction horizons from all seasons, they do not include a full year. This means that the results of the hyperparameter tuning don't necessarily provide a full picture of the model's actual performance. Here it becomes a balancing act between having a more trust worthy hyperparameter search result, while also having the time to investigate several models and their associated hyperparameters.

It could possibly have been useful to set up several scenarios of different lengths of validation periods and perhaps also the selection of these periods across the year. This would require knowledge of, which weeks are most representative of their respective seasons to accurately gauge the performance of the models tested. The optimal solution would be to validate across the entire validation dataset, since it would be the most honest assessment of the model's performance. But this will be dependent on the computational resources available.

6.3.2 Hyperparameter Tuning

Hyperparameter tuning is a very important aspect of machine learning, since the hyperparameters selected form the foundation for the training, validation, and testing of the model, and therefore directly impacts the performance. It is also a very time-consuming step as it requires the training and validation of many models in order to reach the optimum for the given model. Due to it being time-consuming, it has not been possible to perform a fully comprehensive hyperparameter search for all models examined. This means that the hyperparameters found and used for each model may not be the optimal combination. If any changes are made to the method, including the forecasted features, this will also directly impact the hyperparameters.

The shallow learners were hyperparameter tuned once for each model, which means that the search space was not narrowed down across multiple searches. They were, however, tested for a larger number of combinations. Due to the added complexity of the deep learners, they were tuned across multiple search grids, but the search was still not all encompassing and fully comprehensive, as this would have taken too much time. It is possible that if more time had been spent on the hyperparameter tuning that the results might have improved.

The deep learners were also all optimized using Adam, meaning that the choice of optimizer was not included in the hyperparameter search grid. Many of the studies described in section 2 also used Adam as the optimizer. One study even compared the performance of Adam to RMSprop and SGD, and found that Adam outperformed the others [11]. But several other studies used other optimization methods such as particle swarm optimization and sparrow search algorithm, and more studies exist that use optimization methods not included in the PyTorch library. Optimizers are used to find the best combination of weights and biases [78], which means that they have a direct influence on the performance of the model. Therefore, spending more time on searching for the best performing optimizer might also lead to an overall better performing model. This is something that would be useful to examine further in any future work.

6.3.3 Training Period

An analysis of the optimal training period length was only done for two models; Random Forest and LightGBM, and only for one set of hyperparameters for each model. The analysis was performed for both bidding zones, which found that training on the most recent 2-3 years yielded the best results. This was somewhat surprising, since 2022 was a year with unusually high prices and volatility due to an energy crisis. The models that trained on the full training set, all 8 years, were the worst performers. The result of this analysis became the foundation for the hyperparameter tuning of all subsequent models.

Since the analysis was only done for two models and for a limited set of hyperparameters, it is not possible to conclude that this is actually the optimal training period for all other models regardless of hyperparameters. But it did save time on the hyperparameter tuning. To have a greater justification, it would be necessary to repeat the analysis for more models and ideally for more hyperparameter combinations. An entirely comprehensive analysis is probably not realistic due to the time that it will require.

6.3.4 Evaluation Metrics

During the hyperparameter tuning, SMAPE was used as the decision metric. This means that when determining the best performing hyperparameter combination, the SMAPE values was the deciding factor above RMSE and MAE. It is possible, if RMSE or MAE had been used as the decision metric instead of SMAPE that the combination of hyperparameters may have been different. The three evaluation metrics each calculate an error, but their equation is different, which means that they will reach different results. This also means that relying on a single metric as the deciding factor will cause the model to be tuned to yield the lowest value of that metric, which does not necessarily ensure that the optimal combination, and thereby model performance, is achieved.

A method of handling this is called multi-objective optimization (MOO), where it is possible to optimize several objective functions, such as evaluation metrics, simultaneously. MOO attempts to find a balance between the objective functions used, which also means that it is not always possible to find a single optimal solution. MOO is not an actual method in itself, but rather a category of different methods such as scalarization and Pareto. The scalarization approach creates multi-objective functions, which are made into a single solution by utilizing weights before optimization [79].

The Pareto approach is more complicated. It uses a continuously updated algorithm to find a dominated solution and a non-dominated solution. A dominated solution is a solution that is not optimal, so the solution is dominated by a better or more optimal solution. A non- dominated solution is a solution that is not dominated by any other solution. These are also referred to as Pareto Efficient, and the non-dominated solution set is called the Pareto Optimal Set. This also means that Pareto optimality is a state that is reached, when the resources of a given system are optimized in a way, where it is not possible to improve one dimension without worsening another [79].

Using a method of MOO, it may have been possible to balance the optimization of the three evaluation metrics, and this could have led to a different set of hyperparameters being selected for the final evaluation. Which might have yielded a different result and conclusion. In this case, using such a method would have been particularly useful, as SMAPE, the main decision metric, has an upper bound built into its definition. It is possible that this features may make its value more optimistic than that of RMSE and MAE, which do not have an upper bound and focus on the "absolute" error. Therefore, any conclusion of the best performing model and set of parameters is contingent on the use of SMAPE as the decision metric during hyperparameter tuning, which does not necessarily make it the definite optimal model.

It also becomes more apparent that there are limitations with using the evaluation metrics, when the statistical metrics from section 6.1.5 are considered as well. Both GRU and LSTM only predicted a single value, yet they still managed to have better evaluation metric values than LSTM AE. SVR also appeared to be a good model for both DK1 and DK2 based on its evaluation metric values, but when including the other statistical metrics, it is clear that the model has not actually learnt any patterns within the data. At least not enough to generate statistical metrics similar to the characteristics of the dataset. It is also not clear based on the evaluation metrics and the statistical metrics, whether the models have truly grasped the seasonality of the dataset. This is also something that would be worth examining further as there may be some models that will work better for certain parts of the year, and others that will work better at other times.

7 Conclusion

This thesis has examined several different models and analyzed the results in different context, which means that it is not easy to determine one absolute conclusion of the work. There are many additional avenues to be explored, which are described further in section 8. A great amount of time was spent developing the overall methodology and logic that the actual training should be based upon. Several minor analyses were also executed to support the work done to develop the methodology and improve the performance of the models. Due to time constraints, DK2 was not evaluated for deep learners, but it was evaluated for the three baseline models and the seven shallow learners. Evaluation of the final evaluation of the models was carried out using 6 metrics: overall hourly SMAPE (%), overall hourly RMSE (DKK/MWh), overall hourly MAE (DKK/MWh), average error quartile 1 (DKK/MWh), average error quartile 2 (DKK/MWh), and average error quartile 3 (DKK/MWh). The overall best performing model for DK1 was XGBoost with a SMAPE of 52.93 %, RMSE of 311.48 DKK/MWh, MAE of 220.35 DKK/MWh, quartile 1 of 70.21 DKK/MWh, quartile 2 of 163.10 DKK/MWh, and quartile 3 of 300.81 DKK/MWh. The best baseline model was Seasonal Naïve, and the best deep learner was RNN. The overall best performing model for DK2 was Seasonal Naïve, which had a SMAPE of 55.85 %, RMSE of 350.66 DKK/MWh, MAE of 253.26 DKK/MWh, quartile 1 of 86.67 DKK/MWh, quartile 2 of 196.45 DKK/MWh, and quartile 3 of 350.75 DKK/MWh. The best performing shallow learner was SVR. We also determined that the addition of explanatory features improves the performance of the models, and that the error does change over the course of the 7-day prediction period. This change in error is not a steady increase across the prediction days, but it is likely due to some underlying patterns in the data that the models struggle to learn. The introduction of noise to weather data during the prediction periods in the final evaluation did not have a big impact on the performance of the models.

8 Future Perspectives

While this thesis did examine many different aspects related to predicting electricity prices, there was also a large constraint called time. All models, apart from the baseline models, were hyperparameter tuned, but this could have been more comprehensive. Hyperparameter tuning of the deep learners also did not include the optimizer, which would be worth looking into. The hyperparameter tuning should also be evaluated using all three of the metrics rather than being based on SMAPE alone. The deep learners were only examined on the DK1 zone, so the process of hyperparameter tuning and evaluating should be repeated for DK2. This would also make

it possible to come closer to a proper conclusion for DK2. Overall, the DK2 models performed noticeably worse than their DK1 counterparts. Since the goal was to develop a good model for each zone respectively, the cause behind the DK2 models' poor performance should absolutely be investigated further.

It would also be worth spending more time on determining which features should be included in the dataset. There may be features that should be added, such as the gas price, and there may be features that are redundant. These redundant features could be found by using a correlation matrix or a feature selection library. If the weather features remain in the dataset, then they should have noise added during the validation step of the hyperparameter tuning, if it is not possible to obtain actual weather forecasts. The forecast models used for the features during validation and final evaluation should also be examined further. It is likely that there exist better performing models, which will improve the performance of the prediction models as well. The validation step during the hyperparameter tuning did not use the full validation period, 2024, but a couple of weeks spread across the year. This is also something that would be useful to examine, especially for the deep learners, where it will be a balancing act between having a larger training set or a larger validation set and what the implications of this might be.

9 Author Contributions

This thesis has been created as a means of collaboration. Both parties have contributed to the work, the ideas, reflections, and conclusions. But each person has contributed more to certain aspects. Nikolaj worked on developing and preprocessing the dataset along setting up the logic behind all of the modules and developing the pipeline and architecture of the project. He also wrote the majority of the methodology section. Christine focused on producing a well-researched literature review and descriptions of the various models examined in the project. Nikolaj wrote the introduction of the thesis, and Christine wrote the results section and defined the logic for evaluating the final models. Both parties collaborated on writing the discussion and on determining the appropriate reflections and considerations to be included, along with deciding the best method for choosing the overall best performing model. The hyperparameter tuning, final evaluation, and determining SHAP values was also a collaboration, where each person was in charge of running different parts of the code for different models. A number of additional analyses were also performed to enhance the results and discussion, which was also a collaborative effort.

10 References

1. Energinet. *Roller på elmarkedet* https://energinet.dk/el/elmarkedet/roller-pa-elmarkedet/.
2. Pool, N. *Bidding areas* https://www.nordpoolgroup.com/en/the-power-market/Bidding-areas/.
3. Energinet. *Om Energinet* https://energinet.dk/Om-os/Om-Energinet.
4. Energinet. *Hvad påvirker elpriserne i Danmark?* Notat. 2016.
5. Houthakker, H. S. Some Calculations on Electricity Consumption in Great Britain. *Journal of the Royal Statistical Society. Series A (General)* **114,** 359–371 (1951).
6. Labandeira, X., Labeaga, J. M. & López-Otero, X. Estimations of elasticity price of electricity with incomplete information. *Energy Economics* **34,** 627–633 (2012).
7. Joutz, F. L., Maddala, G. S. & Trost, R. P. An Integrated Bayesian Vector Autoregression and Error Correction Model for Forecasting Electricity Consumption and Prices. *Journal of Forecasting* **14,** 287–310 (1995).
8. McMenamin, J. S. & Monforte, F. A. Short Term Energy Forecasting with Neural Networks. *The Energy Journal* **19,** 43–61 (1998).
9. Scholz, C., Lehna, M., Brauns, K. & Baier, A. Towards the Prediction of Electricity Prices at the Intraday Market Using Shallow and Deep-Learning Methods. *Mining Data for Financial Applications* **12591,** 101–118 (2021).
10. Zhang, Y. *et al.* Hourly Electricity Price Prediction for Electricity Market with High Proportion of Wind and Solar Power. *Energies* **15** (2022).
11. Chang, Z., Zhang, Y. & Chen, W. Electricity price prediction based on hybrid model of adam optimized LSTM neural network and wavelet transform. *Energy* **187** (2019).
12. Yang, Z., Ce, L. & Lian, L. Electricity price forecasting by a hybrid model, combining wavelet transform, ARMA and kernel-based extreme learning machine methods. *Applied Energy* **190,** 291–305 (2017).
13. Ugurlu, U., Oksuz, I. & Tas, O. Electricity Price Forecasting Using Recurrent Neural Networks. *Energies* **11** (2018).
14. Rezaei, N., Rajabi, R. & Estebsari, A. Electricity Price Forecasting Model based on Gated Recurrent Units. *2022 IEEE International Conference on Environmental and Electrical Engineering and 2022 IEEE Industrial and Commercial Power Systems Europe* (2022).

15. Catalão, J., Mariano, S., Mendes, V. & Ferreira, L. Short-term electricity prices forecasting in a competitive market: A neural network approach. *Electric Power Systems Research* **77,** 1297–1304 (2007).

16. Peter, S. E. & Raglend, I. J. Sequential wavelet-ANN with embedded ANN-PSO hybrid electricity price forecasting model for Indian energy exchange. *Neural Computing & Applications* **28,** 2277–2292 (2017).

17. Wang, Z. *et al.* Enhanced Day-Ahead Electricity Price Forecasting Using a Convolutional Neural Network–Long Short-Term Memory Ensemble Learning Approach with Multimodal Data Integration. *Energies* **17** (2024).

18. MEdina, A. M. & Álvaro, J. A. H. Using Generative Pre-Trained Transformers (GPT) for Electricity Price Trend Forecasting in the Spanish Market. *Energies* **17** (2024).

19. Wang, B., Huang, B., Zhang, Q., Shi, Y. & Men, H. Temporal feature mixed inverted transformer: An inverted transformer for effective real-time electricity price forecasting. *Engineering Applications of Artificial Intelligence* **167** (2026).

20. Zhao, L., Lu, L. & Yu, X. MILET: multimodal integration and linear enhanced transformer for electricity price forecasting. *Systems Science & Control Engineering* **12** (2024).

21. Karabiber, O. A. & Xydis, G. Electricity Price Forecasting in the Danish Day-Ahead Market Using the TBATS, ANN and ARIMA Methods. *Energies* **12** (2019).

22. Kiliç, D. K., Nielsen, P. & Thibbotuwawa, A. Intraday Electricity Price Forecasting via LSTM and Trading Strategy for the Power Market: A Case Study of the West Denmark DK1 Grid Region. *Energies* **17** (2024).

23. For Preventive Action, C. *War in Ukraine* https://www.cfr.org/global-conflict-tracker/conflict/conflict-ukraine.

24. Of Transmission System Operators for Electricity, T. E. N. *Transparency platform* https://transparency.entsoe.eu/?appState=%7B%22sa%22%3A%5B%5D%2C%22st%22%3A%22BZN%22%2C%22mm%22%3Atrue%2C%22ma%22%3Afalse%2C%22sp%22%3A%22CLOSED%22%2C%22dt%22%3Anull%2C%22df%22%3Anull%2C%22tz%22%3A%22CET%22%7D.

25. Investing.com. *EUR/DKK - Euro Danish Krone* https://investing.com/currencies/eur-dkk-historical-data.

26. Zhi-Hua Zhou and Shaowu Liu. *Machine Learning* ISBN: 978-981-15-1966-6 (Springer, 2021).

27. Frost, J. *Root Mean Square Error (RMSE)* https://statisticsbyjim.com/regression/root-mean-square-error-rmse/.

28. Frost, J. *Mean Absolute Error (MAE)* https://statisticsbyjim.com/glossary/mean-absolute-error/#google_vignette.

29. Frost, J. *Mean Absolute Percentage Error (MAPE)* https://statisticsbyjim.com/glossary/mean-absolute-percentage-error/.

30. Mind, E. *sMAPE: Symmetric Mean Absolute Percentage Error* https://www.emergentmind.com/topics/symmetric-mean-absolute-percentage-error-smape.

31. Hewamalage, H., Ackermann, K. & Bergmeir, C. Forecast Evaluation for Data Scientists: Common Pitfalls and Best Practices. *Data Mining and Knowledge Discovery* (2022).

32. Docs, S. *Baseline forecaster* https://skforecast.org/0.11.0/user_guides/forecasting-baseline.

33. Elledge, A. *Naive Persistence Model: A Baseline Forecasting Technique in Quantitative Finance* https://medium.com/coinmonks/naive-persistence-model-a-baseline-forecasting-technique-in-quantitative-finance-3b1bf9b2e197.

34. QuestDB. *Simple Moving Average* https://questdb.com/glossary/simple-moving-average/.

35. OpenForecast. *Simple forecasting methods* https://openforecast.org/adam/simpleForecastingMethods.html.

36. Ömer Faruk Ertuğrul and Josep M. Guerrero and Musa Yilmaz. *Shallow Learning vs. Deep Learning: A Practical Guide for Machine Learning Solutions* ISBN: 978-3-031-69498-1 (Springer, 2024).

37. For Geeks, G. *Support Vector Machine (SVM) Algorithm* https://www.geeksforgeeks.org/machine-learning/support-vector-machine-algorithm/.

38. For Geeks, G. *Major Kernel Functions in Support Vector Machine (SVM)* https://www.geeksforgeeks.org/machine-learning/major-kernel-functions-in-support-vector-machine-svm/.

39. For Geeks, G. *Gamma Parameter in SVM* https://www.geeksforgeeks.org/machine-learning/gamma-parameter-in-svm/.

40. Omarzai, F. *Support Vector Machine (SVM) In Depth* https://medium.com/@fraidoonomarzai99/support-vector-machine-svm-in-depth-f582d8a9d1c1.

41. For Geeks, G. *XGBoost* https://www.geeksforgeeks.org/machine-learning/xgboost/.

42. IBM. *XGBoost* https://www.ibm.com/think/topics/xgboost.

43. Brownlee, J. *XGBoost for Regression* https://machinelearningmastery.com/xgboost-for-regression/.

44. For Geeks, G. *Linear Regression in Machine Learning* https://www.geeksforgeeks.org/machine-learning/ml-linear-regression/.

45. scikit-learn. *LinearRegression* https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html.

46. For Geeks, G. *Implementation of Lasso Regression From Scratch using Python* https://www.geeksforgeeks.org/machine-learning/implementation-of-lasso-regression-from-scratch-using-python/.

47. scikit-learn. *Lasso* https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Lasso.html.

48. For Geeks, G. *Ridge Regression* https://www.geeksforgeeks.org/machine-learning/what-is-ridge-regression/.

49. scikit-learn. *Ridge* https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html.

50. For Geeks, G. *Random Forest Regression in Python* https://www.geeksforgeeks.org/machine-learning/random-forest-regression-in-python/.

51. scikit-learn. *RandomForestRegressor* https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html.

52. For Geeks, G. *Random Forest Hyperparameter Tuning in Python* https://www.geeksforgeeks.org/machine-learning/random-forest-hyperparameter-tuning-in-python/.

53. For Geeks, G. *LightGBM (Light Gradient Boosting Machine)* https://www.geeksforgeeks.org/machine-learning/lightgbm-light-gradient-boosting-machine/.

54. For Geeks, G. *Train a model using LightGBM* https://www.geeksforgeeks.org/machine-learning/train-a-model-using-lightgbm/.

55. For Geeks, G. *LightGBM Feature parameters* https://www.geeksforgeeks.org/machine-learning/lightgbm-feature-parameters/.

56. TutorialsPoint. *LightGBM - Parameter Tuning* https://www.tutorialspoint.com/lightgbm/lightgbm-parameter-tuning.htm.

57. For Geeks, G. *Python - ARIMA Model for Time Series Forecasting* https://www.geeksforgeeks.org/machine-learning/python-arima-model-for-time-series-forecasting/.

58. For Geeks, G. *ARIMA* https://www.geeksforgeeks.org/machine-learning/model-selection-for-arima/.

59. For Geeks, G. *What Is an ARIMAX Model?* https://www.geeksforgeeks.org/artificial-intelligence/what-is-an-arimax-model/.

60. For Geeks, G. *RNN vs LSTM vs GRU vs Transformers* https://www.geeksforgeeks.org/deep-learning/rnn-vs-lstm-vs-gru-vs-transformers/.

61. For Geeks, G. *Vanishing and Exploding Gradients Problems in Deep Learning* https://www.geeksforgeeks.org/deep-learning/vanishing-and-exploding-gradients-problems-in-deep-learning/.

62. PyTorch. *RNN* https://docs.pytorch.org/docs/2.11/generated/torch.nn.RNN.html.

63. Israwi, D. *RNN hyperparameters* https://gist.github.com/DavidIsrawi/6c45744c12a4f8fc08bd5b8f7f9e

64. Soni, P. *The Ultimate Guide to Deep Learning Hyperparameter Tuning* https://www.blog.trainindata.com/the-ultimate-guide-to-deep-learning-hyperparameter-tuning/.

65. For Geeks, G. *Gated Recurrent Unit Networks* https://www.geeksforgeeks.org/machine-learning/gated-recurrent-unit-networks/.

66. Anishnama. *Understanding Gated Recurrent Unit (GRU) in Deep Learning* https://medium.com/@anishnama20/understanding-gated-recurrent-unit-gru-in-deep-learning-2e54923f3e2.

67. For Geeks, G. *What is LSTM - Long Short Term Memory?* https://www.geeksforgeeks.org/deep-learning/deep-learning-introduction-to-long-short-term-memory/.

68. Brownlee, J. *A Gentle Introduction to LSTM Autoencoders* https://machinelearningmastery.com/lstm-autoencoders/.

69. For Geeks, G. *Autoencoders in Machine Learning* https://www.geeksforgeeks.org/machine-learning/auto-encoders/.

70. Brownlee, J. *Encoder-Decoder Long Short-Term Memory Networks* https://machinelearningmastery.com/encoder-decoder-long-short-term-memory-networks/.

71. For Geeks, G. *Multi-Layer Perceptron Learning in Tensorflow* https://www.geeksforgeeks.org/deep-learning/multi-layer-perceptron-learning-in-tensorflow/.

72. For Geeks, G. *Latent Space in Deep Learning* https://www.geeksforgeeks.org/deep-learning/latent-space-in-deep-learning/.

73. Ritchie, H. *Weather forecasts have become much more accurate; we now need to make them available to everyone* https://ourworldindata.org/weather-forecasts?ref=plaid.is.

74. SEF. *Få svar på, hvorfor priserne på el og naturgas er steget* https://sefenergi.dk/privat/kontakt-os/hvorfor-stiger-priserne/.

75. Signal, C. *Understanding and Handling Redundant or Correlated Features in Datasets* https://codesignal.com/learn/courses/intro-to-data-cleaning-and-preprocessing-with-titanic/lessons/understanding-and-handling-redundant-or-correlated-features-in-datasets.

76. Mastrandrea, G. *Correlation Matrix, Demystified* https://medium.com/data-science/correlation-matrix-demystified-3ae3405c86c1.

77. C, B. P. *Tips for Effective Feature Selection in Machine Learning* https://machinelearningmastery.com/tips-for-effective-feature-selection-in-machine-learning/.

78. For Geeks, G. *Optimization Rule in Deep Neural Networks* https://www.geeksforgeeks.org/deep-learning/optimization-rule-in-deep-neural-networks/.

79. Salian, A. C. *A Gentle Introduction to Multi-Objective Optimisation* https://codemonk.io/blog/a-gentle-introduction-to-multi-objective-optimization/.

A Appendix

A.1 Declaration of Use of AI

The following pages show our declaration of use of AI during our work on this thesis.

Deklaration for anvendt Generativ Kunstig Intelligens (GAI)
– gældende version fra 1. APRIL 2026

Brug af GAI i opgaver
Som udmøntning af de gældende regler og retningslinjer for anvendelsen af kunstig intelligens på SDU har vi på Naturvidenskab designet en deklarationsformular, som du kan vedlægge som bilag til opgavebesvarelser og bruge til at angive, om du har anvendt AI værktøjer. Det er frivilligt at anvende formularen, men hvis du vælger ikke at anvende den, skal du stadig sikre, at din brug af GAI-værktøjer deklareres i overensstemmelse med SDU's gældende regler.

Deklarationsformularen kan vedlægges dine besvarelser af alle opgaver, der indleveres til bedømmelse, dog naturligvis ikke ved skriftlige stedprøver og andre prøver, hvor det er angivet i kursusbeskrivelsen, at GAI-værktøjer ikke er tilladt.

Hvis du *har* brugt GAI-værktøjer, skal du i deklarationsformularen angive, *hvilke* værktøjer du har brugt, og *hvordan* du har brugt dem.

Ved at deklarere din brug af GAI-værktøjer sikrer du, at der ikke opstår udfordringer i forhold til reglerne om eksamenssnyd, idet SDUs regler foreskriver, at manglende oplysning eller markering af brug af GAI-værktøjer vil blive behandlet som plagiat.

Bedømmelsen af opgaven sker alene ud fra, hvordan du demonstrerer opfyldelse af læringsmålene i kursusbeskrivelsen. Korrekt og hensigtsmæssig anvendelse af GAI-værktøjer kan være med til en højere grad af målopfyldelse, mens uhensigtsmæssig brug kan mindske graden af målopfyldelse. Gå i dialog med din underviser eller vejleder om, hvordan GAI-værktøjer i din konkrete opgave kan anvendes hensigtsmæssigt og højne din demonstration af læringsmålene.

Opmærksomhedspunkter:

- Husk altid at undersøge gældende regler og retningslinjer for brug af GAI på mitsdu.dk
- Vær opmærksom på ikke at fodre GAI-værktøjer med personfølsomme eller andre fortrolige oplysninger.
- Hvis du har brugt GAI-værktøjer til at generere tekst, skal du deklarere, hvordan du har brugt værktøjerne. Husk at følge regler for korrekt parafrasering.
- Hvis du har citeret direkte fra tekst, der er genereret af GAI, skal du angive det som citat med en reference til det GAI-værktøj, der er benyttet – se vejledning på SDU Biblioteket.

Sådan deklarerer du brugen af GAI-værktøjer
I din deklaration af brugen af GAI-værktøjer skal du som minimum:

1. tilkendegive om du har brugt GAI eller ej,
2. hvis du har brugt GAI, hvilke teknologier du har brugt,
3. beskrive, hvordan information er blevet genereret, beskrive de anvendte input samt forklare, hvordan du brugte outputtet i opgavebesvarelsen.

Du skal bruge følgende skema, når du laver din deklaration.

Deklaration for anvendelse af GAI-værktøjer

☐ **Jeg/vi har IKKE benyttet generativ kunstig intelligens til udfærdigelse af denne opgave** *(sæt kryds)*

☒ **Jeg/vi har benyttet generativ kunstig intelligens til udfærdigelse af denne opgave** *(sæt kryds). Oplist, hvilke GAI-værktøjer der er benyttet (husk version):*

Claude Sonnet 4.6
ChatGPT 5.5
Claude Haiku 4.5
Claude Opus 4.7
ChatGPT 4.5

Jeg/vi har brugt GAI-værktøjer på følgende vis*) – en liste over mulige anvendelser er vedlagt til inspiration*

EKSEMPLER PÅ ANVENDELSE
☐ Generering af tekst og parafrasering *I akademiske sammenhænge opfattes generativ AI ikke som en autoritativ kilde. Det anbefales derfor ikke at bruge generativ AI til at generere færdig tekst til akademiske opgaver, medmindre opgaven kræver det (eksempelvis opgaver, der omhandler generativ AI eller prompt engineering).* *Hvis du alligevel indsætter generativ AI genereret tekst i din opgave, så skal det tydeligt deklareres:* *Hvis generativ AI genererer tekststykker, der indsættes direkte, skal du bruge anførselstegn og du skal citere den pågældende generativ AI.* *Bemærk også at dette gælder, hvis du har brugt AI-genererede noter, som inddrages i opgaven.* *Hvis der indsættes tekststykker, hvor du har parafraseret generativ AI output, eller hvis du indsætter tekststykker, hvor generativ AI har parafraseret originalteksten, skal du citere generativ AI.* *Du kan læse mere om citation af generativ AI i Bibliotekets LibGuides.*
☐ Struktur *Generativ AI har hjulpet med at organisere kapitler og sektioner eller øvrige elementer i opgaven.*
☐ Sammenfatning af større mængder information *Generativ AI har kondenseret store mængder data.* *Dokumentér ved hjælp af kildehenvisninger i litteraturlisten, hvilke originaldata der er sammenfattet.*
☐ Analyse og/eller fortolkning af data, litteratur, teorier, m.v. *Generativ AI har udført analyser og fortolkninger, som indgår i din opgave.*

Dokumentér ved hjælp af kildehenvisninger i litteraturlisten, hvilke data der er analyseret og/eller fortolket.
☐ Brainstorming og idegenerering *Generativ AI har hjulpet med at generere nye ideer og perspektiver.*
☒ Kilde- og litteratursøgning *Generativ AI blev brugt til at understøtte din opgave fx ved at identificere relevante kilder, litteratur og materialer.*
☐ Korrektur på sprog eller sproglig feedback *Generativ AI er blevet brugt til korrektur (grammatik, stavning og tegnsætning).* *Generativ AI har bidraget til den sproglige fremstilling af din opgave (fx ift. korrekt brug af fagterminologi, formuleringer, sproglige justeringer, feedback på sprog, oversættelse, osv.).*
☐ Generering af billede, kort, tabeller, lyd, video, m.v.
☒ Programmering *Generativ AI blev brugt til at arbejde med programmering.*
☐ Anden anvendelse (angiv hvilke(n)):

For hvert afkrydset felt forklares kort, hvordan du har brugt de svar AI har genereret. *Beskriv f.eks. kortfattet, hvordan informationen blev genereret, og forklar, hvordan outputtet er anvendt i din/jeres opgave.*

AI er blandt andet blevet brugt til at håndtere fejl, der dukkede op undervejs i projektet. Hvor fejlmeddelelsen blev delt med modellen, og de forslåede løsninger blev gennemgået slavisk.
AI er også blevet anvendt til at hjælpe med at generere kode. Forslag til deciderede kodestykker er blevet gennemgået for at sikre, at de reelt virkede efter hensigten og for at sikre, at den bagvedliggende forståelse også var intakt.

Beskriv hvordan du/I har forholdt dig/jer kritisk til de svar AI har givet.

Vi har sørget for metodisk at gennemgå svarene for at sikre os, at svaret giver mening, og at vi forstår, hvad de forskellige ting nu må betyde. Vi har sørget for ikke at stole blindt på, hvad modellerne nu måtte give af svar. Hvis der har været tvivl omkring nogle af tingene, har vi diskuteret og taget en beslutning i fællesskab. AI er blevet anvendt som et værktøj, ikke som et ekstra gruppemedlem, da vi gerne vil have, at vores speciale reflekterer vores egne evner, refleksioner og kritisk tænkning. Input fra AI er aldrig blevet fulgt blindt; påstande er blevet faktatjekket, kodestykker er blevet testet og ideer til projektmetoder er blevet diskuteret med vejlederne.